



Product Design & Development Handbook

Prepared by: NEXORA Labs

TABLE OF CONTENTS

OhmSim | Product Design & Development Handbook

Chapter	Title	Page
01	Project Overview	3
02	System Overview	8
03	User Roles, Permissions, and System Workflows	14
04	Screen Inventory and Navigation Structure	21
05	Buyer Page Specification	28
06	Administrator Page Specification	33
07	Functional Specifications	38
08	Business Rules	46
09	Technology Stack & System Architecture	51
10	Database Design	57
11	API Design	64
12	UI Component Library & Design System	71
13	Development Guide	82
14	Testing Strategy	95
15	Deployment & Release Guide	107
16	Security & Data Protection	116
17	Maintenance, Monitoring & Project Handover	128
18	User Manual & System Operations	139
19	System Documentation & Technical Reference	150
20	Final Project Standards & Completion Criteria	163

DOCUMENT STRUCTURE

Part	Chapters
Part I — Product Definition	Chapters 01–05
Part II — Design & Architecture	Chapters 06–09
Part III — Application Development	Chapters 10–14
Part IV — Development Operations & Quality	Chapters 15–17
Part V — Operations & Finalization	Chapters 18–20

CHAPTER 1 – PROJECT OVERVIEW

INTRODUCTION

OhmSim is a Progressive Web Application (PWA) developed as an academic software engineering project by NEXORA Labs. The system serves as a specialized electronics e-commerce platform that enables users to browse, search, purchase, and manage electronic components through an intuitive and organized digital environment.

Unlike general online marketplaces, OhmSim focuses exclusively on electronic components and related development hardware. The platform is designed to simplify the purchasing process while providing tools that support engineering students, hobbyists, educators, makers, and electronics enthusiasts throughout their project development lifecycle.

The application combines traditional e-commerce functionality with engineering-oriented features such as parametric component search, Bill of Materials (BOM) management, customer support chat, inventory management, order tracking, and administrative reporting.

The system will be developed using modern web technologies and follow Agile Scrum principles throughout the software development lifecycle.

PROJECT DESCRIPTION

OhmSim is a web-based electronics marketplace that allows customers to discover and purchase electronic components while enabling administrators to efficiently manage products, inventory, customer orders, reports, and technical support.

The platform aims to provide a centralized environment where customers can complete the entire purchasing process—from searching for components to receiving order updates—without relying on multiple communication channels.

The system also provides administrators with comprehensive management tools that simplify inventory maintenance, customer interaction, sales monitoring, and operational reporting.

PURPOSE OF THE SYSTEM

The primary purpose of OhmSim is to provide a specialized platform for electronic component sourcing through an organized, efficient, and user-friendly digital experience.

The system seeks to:

-  Simplify the process of purchasing electronic components.
-  Improve product discoverability through specialized search and filtering.
-  Provide centralized communication between customers and administrators.
-  Streamline inventory and order management.
-  Support project-based purchasing through Bill of Materials (BOM) management.
-  Deliver real-time order notifications and updates.
-  Improve operational efficiency through administrative reporting tools.

TARGET USERS

The system is intended for individuals and organizations involved in electronics and embedded systems development.

Primary users include:

- Computer Engineering students
- Electronics Engineering students
- Electrical Engineering students
- STEM students
- Arduino enthusiasts
- ESP32 developers
- IoT developers
- Robotics hobbyists
- Makers
- Educators
- Academic institutions
- Small electronics suppliers

USER ROLES

OhmSim currently supports two primary user roles.

Buyer

A Buyer is a registered customer who uses the platform to browse products, manage carts, place orders, communicate with support, and monitor order progress.

Buyer Permissions

- Register an account
- Log in securely
- Browse product catalog
- Search products
- Apply parametric filters
- View product specifications
- Download datasheets (if available)
- Add items to Shopping Cart
- Create and manage BOM Cart
- Place orders
- Track order status
- Communicate with admins through chat
- Receive push notifications
- Manage personal profile

Administrator

The Administrator manages all operational activities within the platform.

Administrator Permissions

- Manage products
- Manage product categories
- Update inventory
- Manage customer orders
- Update order statuses
- Communicate with buyers
- View sales analytics
- Generate reports
- Manage notifications
- Manage customer accounts
- Monitor system activity

PROJECT OBJECTIVES

The development of OhmSim aims to achieve the following objectives.

PRIMARY OBJECTIVE

Develop a modern Progressive Web Application that provides a complete electronics e-commerce solution for buyers while offering administrators efficient management tools for daily operations.

SECONDARY OBJECTIVES

- Create an intuitive user interface.
- Improve electronic component discovery.
- Simplify project-based purchasing.
- Reduce manual order processing.
- Improve inventory management.
- Provide efficient customer communication.
- Support mobile and desktop devices.
- Deliver a responsive user experience.

PROJECT SCOPE

The first version (MVP) of OhmSim includes the following modules.

BUYER MODULE

- Authentication
- Home Page
- Product Catalog
- Product Categories
- Product Details
- Search
- Parametric Search
- Shopping Cart
- BOM Cart
- Checkout
- Orders
- Order Tracking
- Customer Support Chat
- Notifications
- User Profile

ADMINISTRATORS MODULE

- Dashboard
- Product Management
- Category Management
- Inventory Management
- Order Management
- Customer Management
- Customer Support Chat
- Sales Reports
- Monthly Liquidation Reports
- Notification Management

PROJECT DELIMITATIONS

The initial version of OhmSim intentionally excludes the following features.

- **Multi-vendor marketplace**
- **AI product recommendations**
- **Online payment gateway integration (Payment strictly supports Cash on Delivery [COD] and In-Store Pay on Pickup)**
- **SMS notifications**
- **Social media login**
- **Product reviews and ratings**
- **Community forum**
- **Product auctions**
- **Live video support**
- **Voice calling**
- **Multi-language support**
- **International shipping (Local fulfillment supports In-Store/Lab Pickup and Region 3 Central Luzon Door-to-Door Delivery)**
- **Supplier management portal**

These features may be considered for future versions but are outside the scope of the current project

PLATFORM

OhmSim will be developed as a Progressive Web Application (PWA).

The application will provide:

- Desktop compatibility
- Mobile compatibility
- Tablet compatibility
- Responsive interface
- Installable application experience
- Push notification support
- Cross-platform accessibility through modern web browsers

A native Android or iOS application is not included in the initial release.

PROJECT TEAM STRUCTURE

The project will be developed by NEXORA Labs using the following team structure.

Role	Responsibility
Project Manager	Project planning, task management, sprint coordination, code review, and overall project supervision
Requirements Analyst	Requirements gathering, documentation, user stories, and business rules
UI Designer	Visual design, branding, components, and design consistency
UX Designer	User flows, wireframes, navigation, and usability
Frontend Developers (2)	User interface implementation, responsive design, and client-side functionality
Backend Developer	Database, APIs, authentication, business logic, chat, and notifications
QA Engineer	Testing, bug reporting, validation, and quality assurance

End of Chapter 1

This chapter establishes the project's foundation. Every team member should understand what OhmSim is, who it serves, what it includes, and the overall objectives before moving on to system design. The next chapter should transition into Chapter 2 – System Overview, where the application's modules and high-level architecture are introduced before diving into individual screens and features.

CHAPTER 2 – SYSTEM OVERVIEW

OVERVIEW

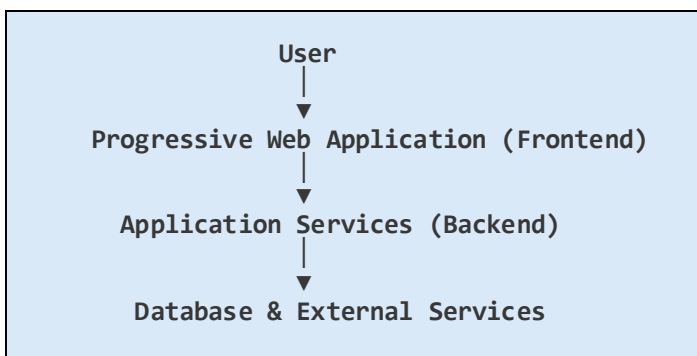
OhmSim is a Progressive Web Application (PWA) that combines an electronics e-commerce platform with inventory management and customer support into a single integrated system. The application consists of two primary modules: the Buyer Module and the Administrator Module.

Each module is designed to support the responsibilities of its respective users while sharing a common database, authentication system, and communication services.

The system follows a modular architecture to improve maintainability, scalability, and future feature expansion.

HIGH-LEVEL ARCHITECTURE

The overall application consists of four major layers.



The frontend provides the user interface and user interactions.

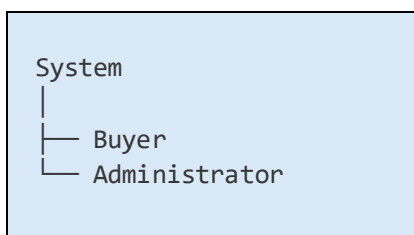
The backend processes business logic, authentication, inventory, order processing, notifications, and communication.

The database stores all application data.

External services provide cloud storage and push notification capabilities.

USER ROLES

OhmSim currently supports two system roles.



Each role has different permissions, interfaces, and responsibilities.

BUYER MODULE

The Buyer Module provides all features required for purchasing electronic components.

The module focuses on product discovery, purchasing, communication, and order monitoring.

INCLUDED FEATURES

- User Registration
- Login
- Home Page
- Product Catalog
- Categories
- Search
- Parametric Search
- Product Details
- Shopping Cart
- BOM Cart
- Checkout
- Order History
- Order Tracking
- Notifications
- Customer Support Chat
- User Profile
- Account Settings

ADMINISTRATOR MODULE

The Administrator Module manages all operational activities.

Administrators control products, inventory, customers, orders, reporting, and customer communication.

INCLUDED FEATURES

- Dashboard
- Product Management
- Category Management
- Inventory Management
- Order Management
- Customer Management
- Customer Support Chat
- Notification Management
- Reports
- Monthly Liquidation
- Account Settings

CORE FUNCTIONAL MODULES

The system is organized into several functional modules.

AUTHENTICATION MODULE

Responsible for secure access to the application.

Functions:

- Registration
- Login
- Logout
- Session Management
- Password Recovery
- Role Authorization

PRODUCT MANAGEMENT MODULE

Responsible for storing and displaying electronic components.

Functions:

- Product Listing
- Product Details
- Categories
- Images
- Specifications
- Datasheets
- Stock Information

PRODUCT DISCOVERY MODULES

Allows buyers to locate products efficiently.

Functions:

- Keyword Search
- Category Filtering
- Parametric Search
- Product Sorting

BOM MANAGEMENT MODULES

Supports project-based purchasing.

Functions:

- Create BOM
- Edit BOM
- Remove Components
- Cost Estimation
- Order Placement
- Availability Checking
- Transfer BOM to Shopping Cart

INVENTORY MODULES

Responsible for stock monitoring.

Functions:

- Add Inventory
- Update Inventory
- Restock Products
- Low Stock Monitoring
- Product Availability

SHOPPING MODULES

Responsible for the purchasing workflow.

Functions:

- Shopping Cart
- Quantity Management
- Voucher Application
- Checkout

ORDER MANAGEMENT MODULES

Responsible for processing customer purchases.

Functions:

- Order Creation
- Status Updates
- Order Tracking
- Pickup Verification
- Order History

CUSTOMER SUPPORT MODULES

Provides communication between buyers and administrators.

Functions:

- Real-time Chat
- Conversation History
- Product Inquiry
- Order Inquiry
- Image Attachment

NOTIFICATION MODULES

Provides user notifications regarding important activities.

Functions:

- Order Updates
- New Messages
- Promotional Announcements (future)
- Push Notifications
- In-App Notifications

REPORTING MODULES

Provides business insights.

Functions:

- Daily Sales
- Weekly Sales
- Monthly Sales
- Inventory Reports
- Revenue Reports
- Monthly Liquidation
- Top Selling Products

SYSTEM NAVIGATION

The application is divided into two navigation structures.

BUYER NAVIGATION

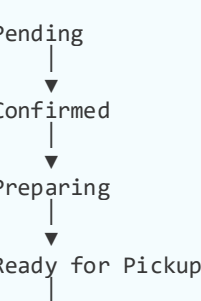


ADMIN NAVIGATION

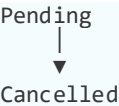


ORDER LIFE CYCLE

Every order follows a predefined workflow.



Alternative paths:

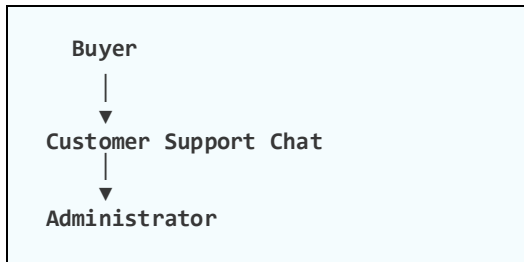


Only administrators can update the order status.
Buyers may only view the current status.

▼
Completed

COMMUNICATION FLOW

Communication occurs only between
Buyers and Administrators.



The chat is intended for:

- Product inquiries
- Order inquiries
- General customer assistance

It is not designed as a public messaging or community platform.

NOTIFICATION FLOW

System events automatically generate notifications.

Examples include:

- New order confirmation
- Order status updates
- New support message
- Account-related notifications

Notifications are delivered through:

- In-app notification center
- Push notifications using Firebase Cloud Messaging (FCM)

SYSTEM WORKFLOW OVERVIEW

The complete high-level workflow is shown below.



Administrators process the transaction through inventory management, order management, customer communication, and reporting.

TECHNOLOGY OVERVIEW

The initial version of OhmSim will use the following technologies.

Layer	Technology
Frontend	Next.js
Language	TypeScript
Styling	Tailwind CSS
UI Components	shadcn/ui
State Management	Zustand
Forms	React Hook Form + Zod
Database	Neon PostgreSQL
ORM	Prisma
Authentication	Better Auth
File Storage	Cloudinary
Real-Time Communication	Socket.IO
Push Notifications	Firebase Cloud Messaging (FCM)
Charts	Recharts
Data Tables	TanStack Table
Toast Notifications	Sonner
Deployment	Vercel
Version Control	GitHub

SUMMARY

The System Overview defines the overall organization of OhmSim before examining individual screens and features. It identifies the major modules, user roles, navigation flows, system architecture, and technology stack that form the foundation of the application.

CHAPTER 3 – USER ROLES, PERMISSIONS, AND SYSTEM WORKFLOWS

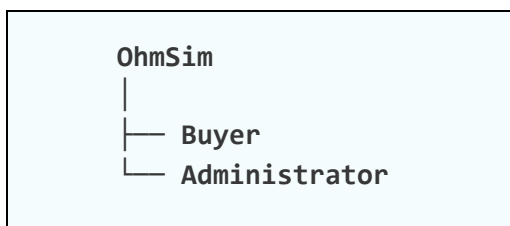
OVERVIEW

The OhmSim system is designed around two primary user roles: Buyer and Administrator. Each role has a distinct set of permissions and responsibilities to ensure a secure, organized, and efficient workflow.

Role-based access control ensures that users can only access the features relevant to their responsibilities.

USER ROLES

The system consists of two primary user roles.



BUYER

DESCRIPTION

A Buyer is a registered user who purchases electronic components through the platform.

The Buyer interacts with the system from product discovery through order completion.

The Buyer does not have permission to modify system data, inventory, products, or administrative information.

BUYER PERMISSIONS

AUTHENTICATION

- Register
- Login
- Logout
- Reset Password
- Update Account Information

PRODUCT BROWSING

- Browse categories
- Search products
- Use parametric search
- View product details

- View specifications
- View product images
- Download datasheets (if available)

SHOPPING

- Add products to Cart
- Update product quantity
- Remove products
- Create BOM Cart
- Edit BOM Cart
- Delete BOM Cart
- Checkout
- View Order Summary

ORDERS

- View Orders
- Track Order Status
- View Order Details
- View Order History
-

COMMUNICATION

- Send messages
- Receive replies
- View conversation history

NOTIFICATIONS

- Receive push notifications
- Receive in-app notifications
- Mark notifications as read

PROFILE

- Edit Profile
- Change Password
- View Purchase History

ADMINISTRATOR

DESCRIPTION

The Administrator is responsible for managing the entire platform.

Administrators control products, categories, inventory, customer orders, customer communication, reports, and system operations.

ADMINISTRATOR PERMISSIONS

PRODUCT MANAGEMENT

- Create Products
- Update Products
- Delete Products
- Archive Products

CATEGORY MANAGEMENT

- Create Categories
- Update Categories
- Delete Categories

INVENTORY MANAGEMENT

- Update Stock
- Restock Inventory
- Monitor Low Stock
- View Inventory Reports

ORDER MANAGEMENT

- View Orders
- Accept Orders
- Reject Orders
- Update Order Status

CUSTOMER MANAGEMENT

- View Customer Information
- Search Customers

COMMUNICATION

- View Conversations
- Reply to Customers
- Resolve Conversations

REPORTING

- Daily Reports
- Weekly Reports
- Monthly Reports
- Monthly Liquidation
- Sales Analytics

NOTIFICATIONS

- Send Announcements
- Send Order Updates

PERMISSION MATRIX

Feature	Buyer	Admin
Register	✓	✗
Login	✓	✓
Browse Products	✓	✓
Search Products	✓	✓
Parametric Search	✓	✓
View Product Details	✓	✓
Add to Cart	✓	✗
Manage BOM Cart	✓	✗
Checkout	✓	✗
View Orders	✓ (Own)	✓ (All)
Update Order Status	✗	✓
Manage Products	✗	✓
Manage Categories	✗	✓
Manage Inventory	✗	✓
View Reports	✗	✓
Chat	✓	✓
Receive Notifications	✓	✓
Manage Users	✗	✓

BUYER WORKFLOW

The Buyer follows a structured purchasing process.



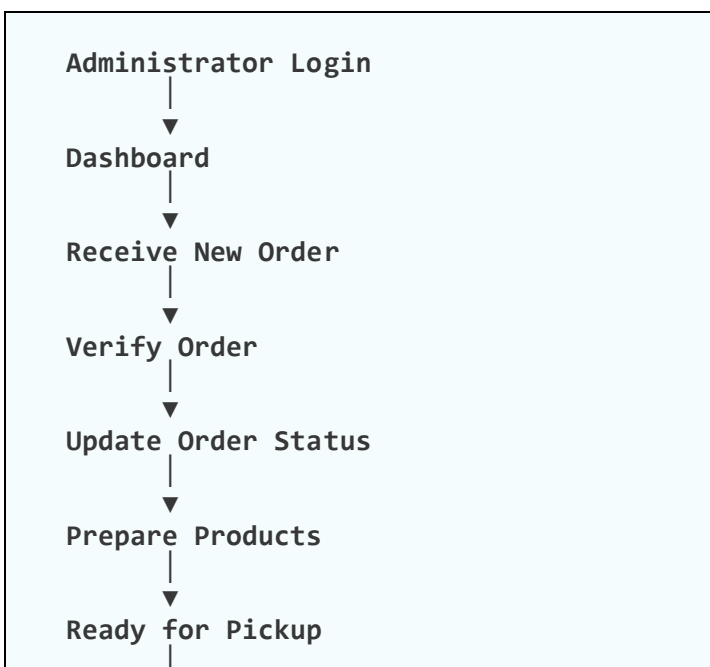
BOM CART WORKFLOW

The Bill of Materials (BOM) Cart is designed for project-based purchasing.



ADMINISTRATOR WORKFLOW

The Administrator manages incoming orders and platform operations.



```
graph TD; A[Buyer Collects Order] --> B[Mark Order Completed]; B --> C[Generate Reports];
```

Buyer Collects Order

Mark Order Completed

Generate Reports

CUSTOMER SUPPORT WORKFLOW

Communication occurs directly between Buyer and Administrator.

```
graph TD; A[Buyer] --> B[Open Chat]; B --> C[Send Message]; C --> D[Administrator Receives Message]; D --> E[Administrator Replies]; E --> F[Buyer Receives Reply];
```

Buyer

Open Chat

Send Message

Administrator Receives Message

Administrator Replies

Buyer Receives Reply

The chat system supports:

- Product inquiries
- Order inquiries
- General customer assistance

ORDER STATUS WORKFLOW

Every order follows a predefined lifecycle.

```
graph TD; A[Pending] --> B[Confirmed]; B --> C[Preparing]; C --> D[Ready for Pickup]; D --> E[Completed];
```

Pending

Confirmed

Preparing

Ready for Pickup

Completed

Alternative status:

```
graph TD; A[Pending] --> B[Cancelled];
```

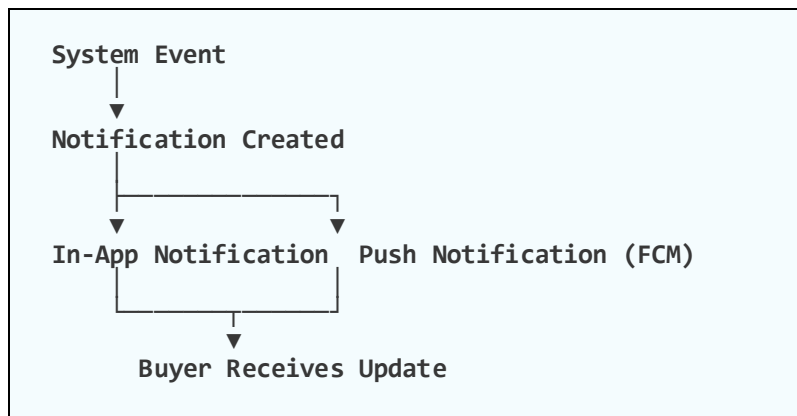
Pending

Cancelled

Status updates are performed only by Administrators.

NOTIFICATION WORKFLOW

System-generated events trigger notifications.



Typical notification events include:

- Order Confirmed
- Order Preparing
- Ready for Pickup
- Order Completed
- New Chat Message

SECURITY PRINCIPLES

To protect user data and maintain system integrity, OhmSim follows these principles:

- Authentication is required for all account-specific actions.
- Users may only access data relevant to their role.
- Buyers may only view and manage their own orders.
- Administrative functions are restricted to Administrator accounts.
- All role-based permissions are enforced on the server side.
- User sessions must be validated before processing requests.

CHAPTER SUMMARY

This chapter defined the two user roles within OhmSim, their permissions, responsibilities, and the complete workflows they follow while interacting with the system. These workflows establish the behavioral foundation of the application and guide the design of the user interface, backend logic, and testing procedures.

CHAPTER 4 – SCREEN INVENTORY AND NAVIGATION STRUCTURE

OVERVIEW

This chapter defines all user interfaces included in OhmSim Version 1.0. It serves as the navigation blueprint for the application by identifying every screen, its purpose, and how users move throughout the system.

This chapter does not describe the detailed functionality of each screen. Those specifications are documented in Chapter 5 – Page Specifications.

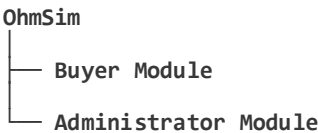
The objectives of this chapter are to:

- Identify every screen within the application.
- Organize screens according to user roles.
- Define the overall navigation hierarchy.
- Establish a consistent navigation structure for desktop and mobile devices.

Provide a reference for UI/UX designers before interface design begins.

APPLICATION NAVIGATION STRUCTURE

OhmSim is divided into two primary application modules.



Each module contains its own set of screens based on the user's permissions and responsibilities.

BUYER SCREEN INVENTORY

The Buyer Module provides all interfaces necessary for browsing products, placing orders, communicating with administrators, and managing personal account information.

No.	Screen	Description
1	Landing Page	Introduces the platform and serves as the public entry point.
2	Login	Authenticates existing buyers.
3	Register	Creates a new buyer account.
4	Forgot Password	Allows buyers to reset their password.

No.	Screen	Description
5	Home	Displays featured products, categories, and announcements.
6	Product Catalog	Displays all available electronic components.
7	Product Details	Displays detailed product information and specifications.
8	Shopping Cart	Displays selected products before checkout.
9	BOM Cart	Organizes components into project-based lists.
10	Checkout	Reviews the order before confirmation.
11	Orders	Displays current and previous orders.
12	Order Details	Displays complete order information and status.
13	Chat	Allows communication with administrators.
14	Notifications	Displays system and order notifications.
15	Profile	Displays buyer information.
16	Account Settings	Allows account management and password updates.

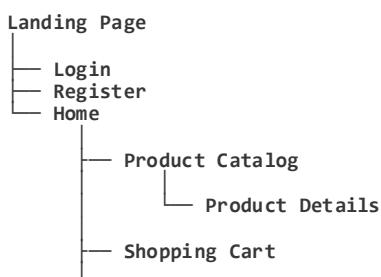
ADMINISTRATOR SCREEN INVENTORY

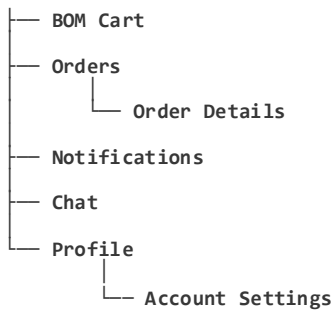
The Administrator Module provides interfaces required to manage daily business operations.

No.	Screen	Description
1	Dashboard	Displays system statistics and recent activity.
2	Product Management	Creates, edits, archives, and removes products.
3	Category Management	Manages product categories.
4	Inventory Management	Updates stock quantities and monitors inventory.
5	Order Management	Processes customer orders.
6	Order Details	Displays detailed order information.
7	Customer Management	Displays registered customer accounts.
8	Chat Center	Handles customer inquiries.
9	Reports	Displays sales and inventory reports.
10	Monthly Liquidation	Generates monthly financial summaries.
11	Notification Management	Sends announcements and notifications.
12	Account Settings	Manages administrator profile information.

BUYER NAVIGATION HIERARCHY

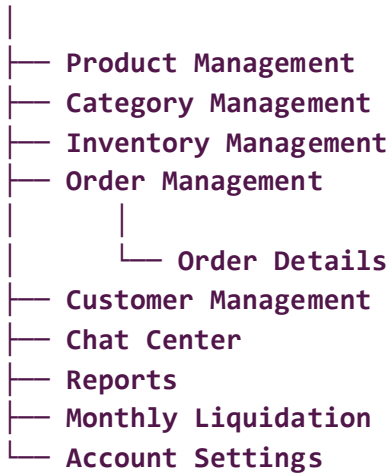
The Buyer Module follows the navigation structure shown below.





ADMIN NAVIGATION HIERARCHY

Dashboard



NAVIGATION PRINCIPLES

The following navigation principles shall be implemented throughout the application.

CONSISTENCY

- └ All pages shall maintain consistent navigation placement and behavior.

SIMPLICITY

- └ Primary actions shall be accessible within a minimal number of interactions.

VISIBILITY

- └ The current page shall always be visually distinguishable within the navigation interface.

ACCESSIBILITY

- └ Navigation components shall remain usable across desktop, tablet, and mobile devices.

RESPONSIVENESS

- └ Navigation layouts shall automatically adapt to different screen sizes.

NAVIGATION COMPONENTS

The Buyer Module contains several persistent navigation components.

HEADER

Appears on all authenticated buyer pages.

Contains:

- OhmSim Logo
- Search Bar
- Notification Icon
- Shopping Cart Icon
- User Profile Menu

SIDEBAR DESKTOP

Displays:

- Home
- Product Catalog
- BOM Cart
- Orders
- Chat
- Notifications
- Profile

BOTTOM NAVIGATION (MOBILE)

Contains:

- Home
- Search
- Cart
- Orders
- Profile

ADMIN NAVIGATION COMPONENTS

The Administrator interface primarily uses a collapsible sidebar.

Sidebar Items:

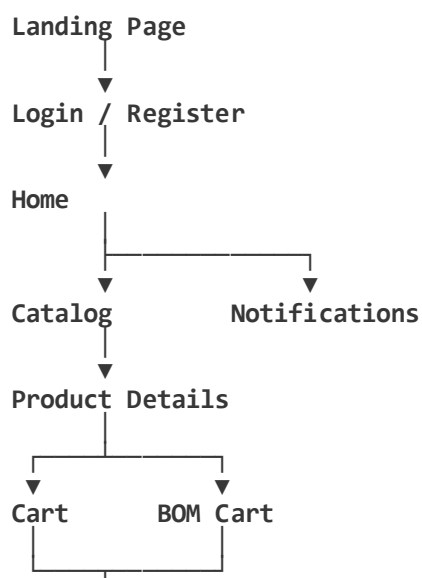
- Dashboard
- Products
- Categories
- Inventory
- Orders
- Customers
- Reports
- Chat
- Monthly Liquidation
- Settings

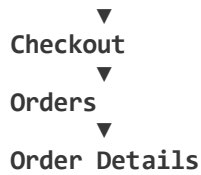
A top navigation bar displays:

- Current page title
- Search (optional)
- Notifications
- Administrator profile

SCREEN RELATIONSHIPS

The following diagram illustrates the relationship between major Buyer screens.





COMMON UI ELEMENTS

The following components are shared across multiple screens to maintain visual consistency.

Component	Purpose
Header	Primary navigation
Sidebar	Desktop navigation
Bottom Navigation	Mobile navigation
Search Bar	Product search
Product Card	Product preview
Category Card	Category selection
Notification Badge	Unread notification count
Shopping Cart Icon	Cart access
Breadcrumb	Navigation hierarchy
Footer	Additional information and links

SCREEN STATES

Each page should support appropriate interface states.

LOADING STATE

Displayed while data is being retrieved.

Examples:

- Skeleton loaders
- Loading indicators
- Progress bars

EMPTY STATE

Displayed when no data exists.

Examples:

- Empty Cart
- No Orders
- No Notifications

- No Search Results
- Empty BOM

Each empty state should provide a descriptive message and a suggested next action.

ERROR STATE

Displayed when the requested operation cannot be completed.

Examples:

- Product Not Found
- Order Not Found
- Network Connection Error
- Unauthorized Access
- Session Expired
- Internal Server Error

Users should always be presented with a recovery option, such as retrying the request or returning to the previous page.

RESPONSIVE DESIGN GUIDELINES

OhmSim shall support responsive layouts across multiple device categories.

Device	Layout
Desktop	Sidebar + Header
Laptop	Sidebar + Header
Tablet	Collapsible Sidebar
Mobile	Bottom Navigation + Header

The interface should maintain consistent functionality regardless of screen size.

DESIGN CONSISTENCY

To provide a cohesive user experience, the following design principles shall be observed throughout the application:

- Consistent spacing and alignment.
- Uniform typography hierarchy.
- Standardized button styles.
- Reusable UI components.
- Consistent iconography.
- Uniform color palette aligned with the NEXORA Labs branding.
- Clear visual feedback for interactive elements.

CHAPTER SUMMARY

This chapter establishes the complete screen inventory and navigation structure for OhmSim Version 1.0. It identifies every screen included in both the Buyer and Administrator modules, defines how users navigate throughout the application, and documents the common navigation components and interface standards.

CHAPTER 5 – BUYER PAGE SPECIFICATION

OVERVIEW

This chapter provides the detailed specification for every page within the Buyer Module. Each page specification defines its purpose, layout, interface components, available user actions, navigation behavior, validation requirements, and expected system responses.

The objective of this chapter is to establish a single reference for the implementation and testing of all buyer-facing interfaces. The specifications described here will be used by the UI Designer, UX Designer, Frontend Developers, Backend Developer, and QA Engineer throughout the development process.

Every page specification follows a standardized structure to ensure consistency across the application.

Each page includes:

- Purpose
- Access Requirements
- Navigation Entry Points
- Layout Structure
- UI Components
- User Actions
- Validation Rules
- Navigation Behavior
- Loading States
- Empty States
- Error States
- Future Enhancements (if applicable)

BUYER PAGES

The Buyer Module consists of the following pages:

No.	Page
5.3	Landing Page
5.4	Login
5.5	Register
5.6	Forgot Password

No.	Page
5.7	Home
5.8	Product Catalog
5.9	Product Details
5.10	Shopping Cart
5.11	BOM Cart
5.12	Checkout
5.13	Orders
5.14	Order Details
5.15	Chat
5.16	Notifications
5.17	Profile
5.18	Account Settings

STANDARD PAGE SPECIFICATION TEMPLATE

Every buyer page documented in this handbook shall follow the same structure.

1. PURPOSE

└ Defines the objective of the page and the problem it solves.

2. ACCESS REQUIREMENTS

└ Defines:

- Guest access
- Authenticated access
- Redirect behavior
- Required permissions

3. ENTRY POINTS

└ Identifies where users can navigate to the page from.

Example:

- Navigation Bar
- Sidebar
- Search Results
- Product Card
- Orders Page

4. LAYOUT STRUCTURE

└ Defines the overall arrangement of the page.

Typical layout includes:

- Header
- Main Content
- Sidebar (if applicable)
- Footer

5. UI COMPONENTS

└ Lists every interface element on the page.

Examples include:

- Buttons
- Cards
- Tables
- Forms
- Inputs
- Search Bars
- Filters
- Dialogs
- Toast Notifications
- Badges
- Pagination
- Breadcrumbs

6. USER ACTIONS

└ Defines every action the user may perform.

Examples:

- Click
- Search
- Submit
- Add
- Remove
- Edit
- Cancel
- Continue

7. VALIDATION RULES

└ Defines client-side validation requirements.

Examples:

- Required fields
- Valid email format
- Password requirements

- Quantity limits
- Stock availability

8. VALIDATION RULES

└ Defines where each action redirects the user.

Examples:

- Successful Login → Home
- Product Click → Product Details
- Checkout Complete → Orders

9. LOADING STATES

└ Defines loading behavior while waiting for data.

Examples:

- Skeleton loaders
- Progress indicators
- Disabled buttons

10. EMPTY STATES

└ Defines the interface when no data is available.

Examples:

- Empty Cart
- No Orders
- No Notifications
- No Search Results

11. ERROR STATES

└ Defines the behavior when an operation fails.

Examples:

- Invalid credentials
- Product unavailable
- Checkout failed
- Network error

12. RELATED APIs

└ This section will be completed after backend planning.

Examples:

- Login API
- Product API
- Cart API
- Order API

13. DATABASE TABLES

└ Lists the database tables involved.

Examples:

- users
- products
- carts
- orders

DOCUMENTATION STANDARD STATES

To ensure consistency across the project:

- Every page shall have a unique specification.
- Every page shall follow the standard template.
- Every interaction shall be documented before implementation.
- Every UI component shall have a defined purpose.
- Every user action shall have a corresponding system response.
- Every page shall define its validation and error handling requirements.

RELATIONSHIP TO FUTURE CHAPTERS

This chapter defines what each page contains.

Subsequent chapters expand on this information:

- Chapter 6 will document each Administrator Page.
- Chapter 7 will describe the functional behavior of features such as Search, Shopping Cart, BOM Cart, Checkout, Chat, Notifications, and Order Management.
- Chapter 8 will define the business rules governing those features.

CHAPTER SUMMARY

This chapter establishes the documentation framework for every buyer-facing page within OhmSim. Rather than describing individual pages immediately, it defines the standard specification format that every page must follow. This ensures consistency across the entire application and provides a clear reference for design, development, backend integration, and testing.

CHAPTER 6 – ADMINISTRATOR PAGE SPECIFICATION

OVERVIEW

This chapter defines the specifications for every page within the Administrator Module of OhmSim. Each page supports administrative operations such as product management, inventory control, order processing, customer support, and reporting.

The purpose of this chapter is to establish a complete reference for the development, testing, and maintenance of all administrator-facing interfaces.

Each page follows the same specification structure introduced in Chapter 5 to maintain consistency across the application.

ADMINISTRATOR PAGES

└─ The Administrator Module consists of the following pages.

No.	Page
6.3	Dashboard
6.4	Product Management
6.5	Category Management
6.6	Inventory Management
6.7	Order Management
6.8	Order Details
6.9	Customer Management
6.10	Chat Center
6.11	Reports
6.12	Monthly Liquidation
6.13	Notification Management
6.14	Administrator Settings

STANDARD SPECIFICATION

└─ Every administrator page shall follow the same documentation format.

PURPOSE

└─ Defines the objective of the page.

ACCESS REQUIREMENTS

Defines:

- Administrator authentication
- Role authorization
- Redirect behavior

ENTRY POINTS

└ Defines where users can access the page.

Examples:

- Sidebar
- Dashboard widgets
- Tables
- Search results
- Notification links

LAYOUT STRUCTURE

└ Typical layout consists of:

- Sidebar
- Top Navigation
- Page Header
- Main Content
- Action Toolbar
- Data Tables
- Footer

UI COMPONENTS

└ Possible components include:

- Data Tables
- Search Bars
- Filters
- Forms
- Charts
- Statistic Cards
- Buttons
- Dialogs
- Confirmation Modals
- Toast Notifications
- Pagination

User Actions

└ Examples include:

- Create
- Update

- Archive
- Delete
- View
- Search
- Filter
- Export
- Print

VALIDATION RULES

Examples:

- Required fields
- Duplicate prevention
- Numeric validation
- Date validation
- Stock validation

NAVIGATION BEHAVIOR

└─ Defines page transitions after user actions.

Examples:

- Save Product → Product List
- View Order → Order Details
- Generate Report → Report Preview

LOADING STATES

Examples:

- Skeleton tables
- Loading spinners
- Disabled action buttons

EMPTY STATES

Examples:

- No Products
- No Orders
- No Reports
- No Messages

ERROR STATES

Examples:

- Validation failed
- Product not found
- Database error
- Permission denied

RELATED APIS

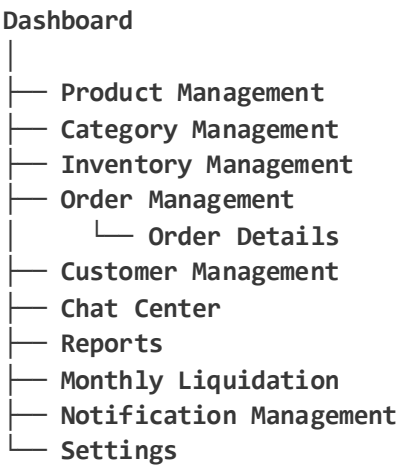
└─ To be completed during backend planning.

DATABASE TABLES

└─ To be completed during database design.

ADMINISTRATOR NAVIGATION

Administrators access all management functions from a centralized dashboard.



DASHBOARD COMPONENTS

└─ The Dashboard serves as the administrator's main workspace.

Typical sections include:

- Summary Cards
- Recent Orders
- Low Stock Products
- Sales Overview
- Monthly Revenue
- Pending Orders
- Notifications
- Quick Actions

SHARED ADMINISTRATOR COMPONENTS

└─ The following components are reused across multiple administrator pages.

Component	Purpose
Sidebar	Primary navigation
Top Navigation	Global actions and profile
Statistics Card	Summary metrics
Data Table	Display records

Component	Purpose
Search Bar	Search records
Filter Panel	Narrow results
Confirmation Dialog	Confirm destructive actions
Toast Notification	Display operation feedback
Pagination	Navigate large datasets

ADMINISTRATIVE OPERATIONS

└ Administrator pages support the following operations.

CREATE

└ Adds new records to the system.

Examples:

- Product
- Category
- Announcement

READ

└ Displays stored information.

Examples:

- Products
- Orders
- Customers
- Reports

UPDATE

└ Modifies existing records.

Examples:

- Product Information
- Inventory
- Order Status

DELETE / ARCHIVE

└ Removes or archives records based on business rules.

EXPORT

└ Allows reports to be exported in supported formats.

ADMINISTRATIVE DESIGN PRINCIPLES

Administrator interfaces should prioritize:

- Efficiency
- Readability
- Fast navigation
- Data accuracy
- Consistency
- Clear visual hierarchy

Unlike buyer interfaces, administrator pages prioritize productivity over marketing or promotional content.

SECURITY CONSIDERATIONS

Administrator pages shall implement additional security controls.

Examples include:

- Authentication required before access.
- Role-based authorization for all administrative functions.
- Confirmation dialogs for destructive actions.
- Audit logging for critical operations (if implemented).
- Automatic session expiration after inactivity.

CHAPTER SUMMARY

This chapter establishes the documentation framework for every administrator-facing page in OhmSim. It identifies the complete set of management interfaces, standardizes how each page will be documented, and defines the shared components, navigation patterns, and administrative operations that will be implemented throughout the system.

CHAPTER 7 – FUNCTIONAL SPECIFICATIONS

OVERVIEW

This chapter defines the functional behavior of every major feature implemented in OhmSim Version 1.0. Unlike the previous chapters, which focused on project structure, navigation, and page organization, this chapter describes how each feature operates from the user's perspective and how the system is expected to respond.

The objective of this chapter is to establish clear functional requirements that guide the implementation of the frontend, backend, and testing processes. Each feature is documented independently to avoid duplication and to provide a centralized reference for developers and testers.

Every functional specification includes:

- Purpose
- Description
- Actors
- Preconditions
- Inputs
- Process Flow
- Outputs
- Exceptions
- Dependencies
- Future Enhancements (if applicable)

AUTHENTICATION

PURPOSE

— Provide secure access to the system while protecting user accounts and restricting access based on assigned roles.

DESCRIPTION

— The Authentication module manages user registration, login, logout, session management, and password recovery.

ACTORS

- Buyer
- Administrator

PRECONDITIONS

- User account exists (Login).
- User has internet connectivity.
- Authentication service is available.

INPUTS

- Email address
- Password
 - Registration additionally requires:
 - Full Name

- Contact Number
- Password Confirmation

PROCESS FLOW

- User enters credentials.
- System validates input.
- Credentials are verified.
- Session is created.
- User is redirected to the appropriate dashboard.

OUTPUTS

- Successful login
- Validation errors
- Invalid credentials message

EXCEPTIONS

- Incorrect password
- Account not found
- Network failure
- Session expired

DEPENDENCIES

- Better Auth
- Database
- User Table

PRODUCT CATALOG

PURPOSE

└ Provide buyers with a centralized list of available electronic components.

DESCRIPTION

└ Displays all products with images, pricing, stock availability, and essential product information.

Supports:

- Pagination
- Searching
- Filtering
- Sorting

ACTORS

└ Buyer

└ Administrator (View Only)

INPUTS

- Search keyword
- Category
- Filter selections
- Sorting option

OUTPUTS

└─ Filtered product list.

EXCEPTIONS

- No products available.
- Search returns no results.

PRODUCT SEARCH

PURPOSE

└─ Allow buyers to quickly locate electronic components.

SEARCH TYPES

KEYWORD SEARCH

Search by:

- Product Name
- Brand
- SKU

PARAMETRIC SEARCH

Filters include:

- Category
- Brand
- Voltage
- Current
- Resistance
- Capacitance
- Tolerance
- Package Type
- Price
- Stock Availability

OUTPUTS

└─ Matching products displayed in the Product Catalog.

PRODUCT DETAILS

PURPOSE

└ Display comprehensive information for a selected electronic component.

DISPLAYS

- Images
- Name
- Price
- Stock
- Description
- Specifications
- Datasheet (if available)
- Related Products

AVAILABLE ACTIONS

- Add to Cart
- Add to BOM
- Contact Administrator

SHOPPING CART

PURPOSE

└ Store products selected for purchase before checkout.

AVAILABLE ACTIONS

- Add Item
- Remove Item
- Update Quantity
- Clear Cart

SYSTEM BEHAVIOR

- Updates subtotal automatically.
- Validates available stock.
- Prevents negative quantities.

BILL OF MATERIALS (BOM) CART

PURPOSE

└ Allow buyers to organize electronic components into project-specific lists before purchasing.

AVAILABLE ACTIONS

- Create BOM
- Rename BOM
- Add Components
- Remove Components
- Edit Quantities
- Calculate Total Cost
- Transfer Components to Shopping Cart

BENEFITS

- Organizes project requirements.
- Reduces repeated searching.
- Simplifies purchasing.

CHECKOUT

PURPOSE

└─ Finalize customer purchases.

REQUIRED INFORMATION

- Fulfillment Method (Pickup / Region 3 Delivery), Delivery Address (Central Luzon), Pickup / Recipient Name
- Contact Number
- Order Notes (Optional)

PROCESS

- Review Order
- Validate Stock
- Confirm Order
- Create Order Record
- Notify Administrator

OUTPUTS

- Order Confirmation
- Order Number

ORDER MANAGEMENT

PURPOSE

Track customer purchases throughout their lifecycle.

ORDER STATUSES

Pending

↓

Confirmed

↓

Preparing

↓

Ready for Pickup

↓

Completed

Alternative Status

Pending

↓

Cancelled

Only administrators may update order statuses.

CUSTOMER SUPPORT CHAT

PURPOSE

└─ Provide direct communication between buyers and administrators.

AVAILABLE FUNCTIONS

- Send Message
- Receive Message
- View Conversation History
- Share Product Reference

NOTIFICATIONS

└─ New messages generate both:

- In-App Notifications

- Push Notifications

NOTIFICATIONS

PURPOSE

└ Inform users about important activities within the system.

NOTIFICATION TYPES

- Order Confirmation
- Order Status Updates
- Chat Messages
- Administrative Announcements

DELIVERY METHODS

- Notification Center
- Firebase Cloud Messaging (FCM)

PRODUCT MANAGEMENT

└ Administrator Functions

- Create Product
- Edit Product
- Archive Product
- Delete Product

CATEGORY MANAGEMENT

└ Administrator Functions

- Create Category
- Edit Category
- Delete Category

INVENTORY MANAGEMENT

└ Administrator Functions

- Update Stock
- Restock Inventory
- Monitor Low Stock
- View Inventory Reports

CUSTOMER MANAGEMENT

└ Administrator Functions

- View Registered Buyers
- Search Customers
- View Customer Purchase History

REPORTS

└ Administrator reports include:

- Daily Sales
- Weekly Sales
- Monthly Sales
- Revenue Reports
- Inventory Reports
- Top Selling Products
- Monthly Liquidation

NOTIFICATION MANAGEMENT

└ Administrator Functions

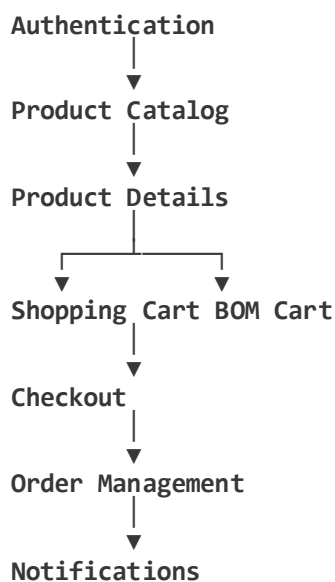
- Create Announcement
- Publish Announcement
- Send Notifications to Buyers

COMMON FUNCTIONAL REQUIREMENTS

└ All system features shall adhere to the following requirements:

- Authentication shall be required for all protected functions.
- The system shall validate all user inputs before processing.
- The system shall display meaningful error messages when operations fail.
- Data shall be retrieved from the database in real time.
- User interface updates shall reflect changes immediately after successful operations.
- The system shall maintain consistent behavior across supported devices.

FUNCTIONAL DEPENDENCY OVERVIEW



The Customer Support Chat operates independently and may be accessed from the Product Details page, Orders page, or the dedicated Chat page.

CHAPTER 8 – BUSINESS RULES

OVERVIEW

This chapter defines the business rules that govern the operation of OhmSim Version 1.0. Business rules establish the conditions, constraints, and policies that the system must enforce to maintain consistency, accuracy, and security.

These rules apply regardless of the technology or implementation used. All frontend interfaces, backend services, and database operations must comply with the business rules described in this chapter.

USER ACCOUNT RULES

BR-001 — Account Registration

└ A buyer must create an account before accessing protected features such as the Shopping Cart, BOM Cart, Checkout, Orders, Chat, and Profile.

BR-002 — Unique Email Address

└ Each email address shall be associated with only one account.

Duplicate registrations are not permitted.

BR-003 — Password Security

└ Passwords shall meet the minimum-security requirements established by the authentication system.

BR-004 — Authentication Required

└ Only authenticated users may access protected pages.

Unauthenticated users attempting to access protected resources shall be redirected to the Login page.

BR-005 — Session Management

└ Authenticated sessions shall expire after a defined period of inactivity.

PRODUCT RULES

BR-006 — Product Availability

└ Only active products shall be displayed in the Product Catalog.

Archived products shall remain in the database but shall not be visible to buyers.

BR-007 — Product Information

└ Each product shall include at least:

- Product Name
- Category
- Price
- Available Stock

Optional information includes:

- Images
- Technical Specifications
- Datasheet

- Brand

BR-008 — Stock Quantity

- └ Product stock shall never become negative.

SHOPPING CART RULES

BR-009 — Cart Ownership

- └ Each Shopping Cart belongs to exactly one buyer.

BR-010 — Quantity Validation

- └ Requested quantities shall not exceed available stock.

BR-011 — Duplicate Products

- └ Adding the same product multiple times shall update its quantity instead of creating duplicate cart entries.

BR-012 — Cart Persistence

- └ Items remain in the buyer's cart until they are removed, purchased, or otherwise cleared by the system.

BOM CART RULES

BR-013 — Multiple BOM Projects

- └ A buyer may create multiple BOM projects.

BR-014 — Project Ownership

- └ Each BOM project belongs to a single buyer.

BR-015 — Independent Management

- └ Changes to one BOM project shall not affect other projects.

BR-016 — Transfer to Shopping Cart

- └ Products transferred from a BOM project shall be validated against current inventory before being added to the Shopping Cart.

CHECKOUT RULES

BR-017 — Valid Cart Required

- └ Checkout requires at least one valid item in the Shopping Cart.

BR-018 — Stock Verification

- └ The system shall verify stock availability immediately before creating an order.

BR-019 — Order Creation & Fulfillment Assignment

- └ A successful checkout shall generate a unique order record, assign the selected fulfillment method (Store Pickup or Region 3 Delivery), apply the appropriate delivery fee (Free for orders ₱1,000.00 and above; ₱80.00 flat fee below ₱1,000.00 for delivery), and set payment method to Cash on Delivery (COD) or Pay on Pickup.

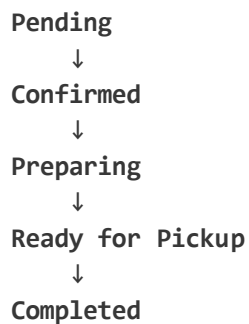
BR-020 — Inventory Update

- └ Product inventory shall be updated only after the order is confirmed by the administrator.

ORDER MANAGEMENT RULES

BR-021 — Order Status Progression (Dual Fulfillment)

└ Orders shall follow the defined lifecycle:



BR-022 — Status Modification

└ Only administrators may change an order's status.

BR-023 — Completed Orders

└ Completed orders shall become read-only and may not be modified.

BR-024 — Cancelled Orders

└ Cancelled orders shall not proceed to any subsequent status.

INVENTORY RULES

BR-025 — Stock Accuracy

└ Inventory quantities shall accurately reflect confirmed stock.

BR-026 — Inventory Updates

└ Only administrators may update inventory records.

BR-027 — Low Stock

└ Products reaching a predefined stock threshold should be identified as low-stock items.

CUSTOMER SUPPORT RULES

BR-028 — Buyer-to-Administrator Communication

└ Chat conversations shall occur only between buyers and administrators.

BR-029 — Conversation History

└ Message history shall be retained and displayed to both participants.

BR-030 — Product References

└ Messages may include references to products or orders to provide context.

NOTIFICATION RULES

BR-031 — Automatic Notifications

└ The system shall automatically generate notifications for significant events, including:

- Order confirmation
- Order status changes
- New chat messages

BR-032 — Notification Delivery

└ Notifications shall be delivered through:

- In-app notifications
- Push notifications (FCM)

ADMINISTRATIVE RULES

BR-033 — Product Management

└ Only administrators may create, modify, archive, or remove products.

BR-034 — Category Management

└ Only administrators may manage product categories.

BR-035 — Customer Records

└ Administrators may view customer information but shall not modify buyer account credentials.

BR-036 — Reports

└ Reports shall reflect the latest available system data.

SECURITY RULES

BR-037 — Role-Based Access Control

└ System permissions shall be enforced based on user roles.

BR-038 — Server-Side Validation

└ Critical business rules shall be validated on the server regardless of client-side validation.

BR-039 — Input Validation

└ All user input shall be validated before processing.

BR-040 — Data Integrity

└ The system shall prevent operations that would leave the database in an inconsistent state.

8.13 GENERAL SYSTEM RULES

BR-041 — Consistent Error Handling

— The system shall display clear, user-friendly error messages when an operation cannot be completed.

BR-042 — Data Consistency

— All users shall view the most recent committed system data.

BR-043 — Responsive Operation

— Core system functionality shall be available across supported desktop and mobile devices.

BR-044 — Auditability (Future Enhancement)

Future versions may record administrative actions such as product updates, inventory changes, and order status modifications.

BR-045 — Delivery Coverage (Region 3 - Central Luzon)

— Door-to-door delivery is strictly supported within Region 3 (Central Luzon provinces: Aurora, Bataan, Bulacan, Nueva Ecija, Pampanga, Tarlac, and Zambales).

BR-046 — Delivery Fee Calculation

— Delivery orders with a merchandise subtotal of ₱1,000.00 and above qualify for Free Delivery (₱0.00). Delivery orders below ₱1,000.00 incur a standard flat delivery fee of ₱80.00. In-store/lab pickup orders incur ₱0.00 fee regardless of amount.

BR-047 — Payment Method Policy (Cash on Delivery / Pay on Pickup)

— Orders strictly support Cash on Delivery (COD) for delivery orders and In-Person Pay on Pickup for store pickup orders. Online payment gateway integration is excluded in Version 1.0.

BR-048 — Saved Delivery Address Limit

— Registered buyers may save and manage a maximum of two (2) delivery addresses within Region 3 in their account profile.

BUSINESS RULE SUMMARY

Rule Category	Rule IDs
User Accounts	BR-001 – BR-005
Products	BR-006 – BR-008
Shopping Cart	BR-009 – BR-012
BOM Cart	BR-013 – BR-016
Checkout	BR-017 – BR-020
Orders	BR-021 – BR-024
Inventory	BR-025 – BR-027
Customer Support	BR-028 – BR-030
Notifications	BR-031 – BR-032
Administration	BR-033 – BR-036
Security	BR-037 – BR-040
General	BR-041 – BR-044
Fulfillment & Delivery	BR-045 – BR-048

CHAPTER SUMMARY

This chapter establishes the business policies that govern the behavior of OhmSim. These rules define the conditions under which users, administrators, products, orders, inventory, and notifications operate, ensuring consistent behavior throughout the application. The business rules documented here serve as the basis for backend validation, frontend behavior, database constraints, and quality assurance testing.

CHAPTER 9 – TECHNOLOGY STACK & SYSTEM ARCHITECTURE

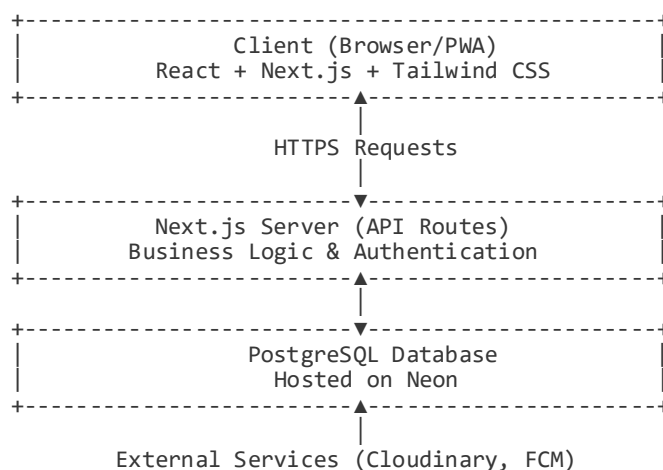
OVERVIEW

This chapter defines the technologies, architectural patterns, and software components used to develop OhmSim Version 1.0. It establishes the technical foundation of the system by identifying the frameworks, libraries, external services, and infrastructure required for development and deployment.

The chosen technology stack prioritizes performance, scalability, maintainability, and ease of collaboration among the development team.

SYSTEM ARCHITECTURE

OhmSim follows a modern web application architecture consisting of four primary layers.



ARCHITECTURAL PATTERN

└─ OhmSim adopts a client–Server Architecture.

Client

Responsible for:

- Rendering the user interface
- Managing user interactions
- Form validation
- Sending requests to the server

Server

Responsible for:

- Business logic
- Authentication
- Database operations
- API responses
- Authorization
- Validation

Database

Responsible for:

- Persistent storage
- Relationships
- Data integrity
- Transaction management

TECHNOLOGY STACK

Layer	Technology
Frontend Framework	Next.js
UI Library	React
Language	TypeScript
Styling	Tailwind CSS
Component Library	shadcn/ui
Icons	Lucide React
Notifications	Sonner
Authentication	Better Auth
Database	PostgreSQL
Database Hosting	Neon
ORM	Prisma ORM
Image Storage	Cloudinary

Layer	Technology
Push Notifications	Firebase Cloud Messaging (FCM)
Version Control	Git
Repository Hosting	GitHub
Deployment	Vercel
Package Manager	npm

FRONTEND ARCHITECTURE

└─ The frontend is responsible for presenting the application interface and handling user interactions.

Major responsibilities include:

- Rendering pages
- Form validation
- Navigation
- State management
- API communication
- Responsive layouts

The frontend shall be developed using React with the Next.js App Router.

BACKEND ARCHITECTURE

└─ The backend handles all application logic.

Responsibilities include:

- Authentication
- Authorization
- Database queries
- Order processing
- Inventory updates
- Notification triggering
- Input validation

The backend is implemented using Next.js Route Handlers.

DATABASE LAYER

└─ The database stores all persistent application data.

Primary responsibilities:

- User accounts
- Products
- Categories
- Shopping carts

- BOM projects
- Orders
- Messages
- Notifications

PostgreSQL hosted on Neon provides relational storage and transaction support.

9.8 EXTERNAL SERVICES

Cloudinary

Used for:

- Product images
- User-uploaded images (if supported)
- Image optimization
- CDN delivery

Firebase Cloud Messaging (FCM)

Used for:

- Push notifications
- Order updates
- Chat notifications
- Announcements

Better Auth

Used for:

- Registration
- Login
- Session management
- Password security
- Authentication middleware

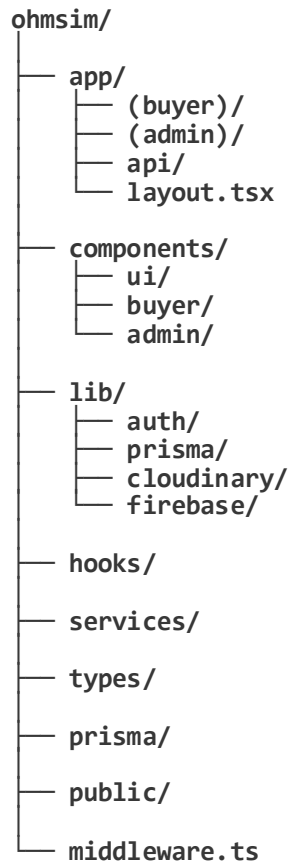
DEVELOPMENT ENVIRONMENT

Recommended tools:

Tool	Purpose
Visual Studio Code	Code editor
Git	Version control
GitHub	Collaboration
Postman or Bruno	API testing
Prisma Studio	Database inspection
Chrome DevTools	Debugging
Figma	UI/UX design

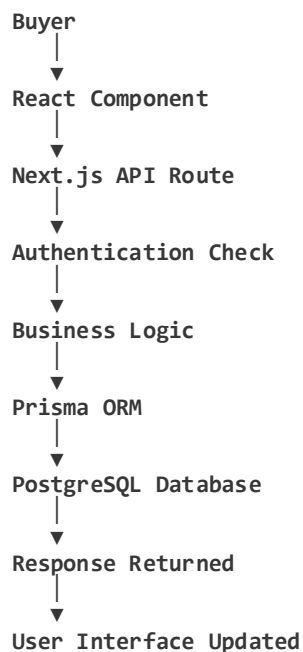
PROJECT FOLDER STRUCTURE

└ A recommended project structure is shown below.



REQUEST FLOW

└ The following diagram illustrates a typical request lifecycle.



SECURITY ARCHITECTURE

└ The application incorporates several security mechanisms.

- HTTPS for all communication.

- Password hashing through Better Auth.
- Role-based access control.
- Server-side authorization.
- Input validation.
- SQL injection protection through Prisma ORM.
- Secure session management.
- Environment variables for sensitive credentials.

PERFORMANCE CONSIDERATIONS

└─ To improve application performance:

- Optimize images using Cloudinary.
- Use server-side rendering where appropriate.
- Implement pagination for large datasets.
- Lazy-load non-critical components.
- Cache static assets.
- Minimize unnecessary API requests.

SCALABILITY CONSIDERATIONS

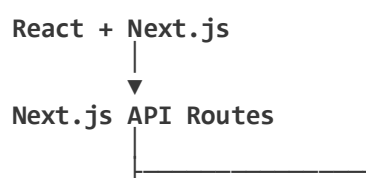
└─ The architecture should support future expansion.

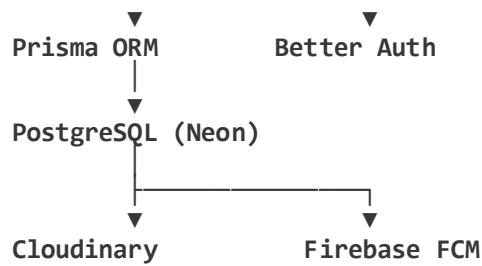
Potential future enhancements include:

- Online payment gateways
- Supplier management
- Multi-administrator support
- Barcode scanning
- QR code pickup verification
- Product reviews and ratings
- Wishlist functionality
- Advanced analytics
- Multi-store support

These features are outside the scope of Version 1.0 but the architecture should allow them to be integrated with minimal restructuring.

TECHNOLOGY INTERACTION DIAGRAM





CHAPTER SUMMARY

This chapter defines the technical foundation of OhmSim by documenting its architecture, development stack, project structure, external services, and system interactions. These decisions establish a consistent environment for development and ensure that every team member works with the same technologies and architectural principles.

CHAPTER 10 – DATABASE DESIGN

OVERVIEW

This chapter defines the database architecture of OhmSim Version 1.0. It identifies the entities, relationships, constraints, and normalization strategy used to support the application's functionality.

The database is implemented using PostgreSQL hosted on Neon, with Prisma ORM serving as the data access layer.

The primary objectives of the database are to:

- Store application data securely.
- Maintain data integrity.
- Reduce redundancy through normalization.
- Support efficient querying.
- Preserve relationships between system entities.

DATABASE MANAGEMENT SYSTEM

Property	Value
Database Management System	PostgreSQL
Hosting Provider	Neon

Property	Value
ORM	Prisma ORM
Database Type	Relational Database

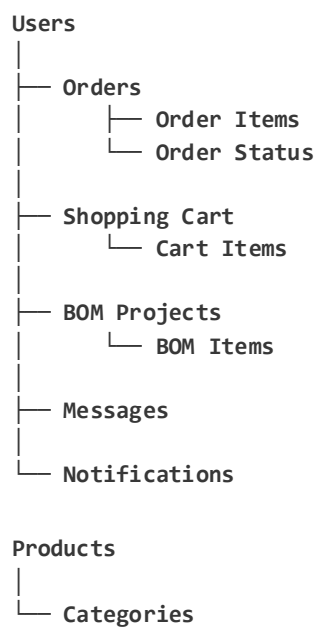
DATABASE DESIGN PRINCIPLE

The database follows these principles:

- Entity normalization to reduce redundancy.
- Primary keys for unique identification.
- Foreign keys to maintain relationships.
- Cascading rules where appropriate.
- Server-side validation before persistence.
- Timestamp tracking for important records.

ENTITY RELATIONSHIP OVERVIEW

The core entities of OhmSim are:



CORE DATABASE TABLE

Table	Purpose
users	Stores buyer and administrator accounts
categories	Stores product categories
products	Stores electronic component information
carts	Stores shopping carts
cart_items	Stores products inside carts
bom_projects	Stores buyer project lists
bom_items	Stores components inside BOM projects
orders	Stores customer orders
order_items	Stores purchased products
messages	Stores buyer-administrator conversations
notifications	Stores in-app notifications
user_addresses	Stores buyer saved delivery addresses (max 2 per buyer within Region 3)

ENTITY DESCRIPTIONS

USERS

Stores account information for all authenticated users.

Example fields:

- id
- full_name
- email
- password_hash
- role
- created_at
- updated_at

CATEGORIES

Groups products into logical classifications.

Example fields:

- id
- name
- description

PRODUCTS

Stores electronic component information.

Example fields:

- id
- category_id

- name
- sku
- brand
- description
- specifications
- price
- stock
- image_url
- created_at
- updated_at

SHOPPING CART

Stores active shopping carts.

Example fields:

- id
- user_id
- created_at

CART ITEMS

Stores products selected by buyers.

Example fields:

- id
- cart_id
- product_id
- quantity

BOM PROJECTS

Stores project-based component collections.

Example fields:

- id
- user_id
- project_name
- created_at

BOM ITEMS

Stores components assigned to each BOM project.

Example fields:

- id
- project_id
- product_id
- quantity

ORDERS

Stores customer purchase information.

Example fields:

- id
- user_id
- order_number
- total_amount
- status
- fulfillment_type
- pickup_name
- recipient_name
- delivery_address
- delivery_fee
- payment_method
- contact_number
- created_at

ORDER ITEMS

Stores products associated with an order.

Example fields:

- id
- order_id
- product_id
- quantity
- unit_price

MESSAGES

Stores chat conversations.

Example fields:

- id
- sender_id
- receiver_id
- message
- created_at

NOTIFICATIONS

Stores system-generated notifications.

Example fields:

- id
- user_id
- title
- message
- type
- is_read
- created_at

USER ADDRESSES

Stores saved delivery addresses for registered buyers (maximum 2 addresses within Region 3: Central Luzon).

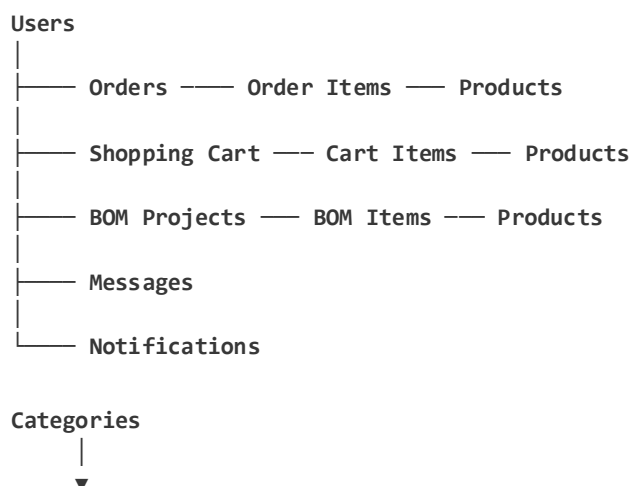
Example fields:

- id
- user_id
- recipient_name
- contact_number
- region (Region III)
- province
- city_municipality
- barangay
- street_address
- postal_code
- is_default

PRIMARY RELATIONSHIPS

Parent	Child	Relationship
Users	Orders	One-to-Many
Users	Shopping Cart	One-to-One
Shopping Cart	Cart Items	One-to-Many
Products	Cart Items	One-to-Many
Users	BOM Projects	One-to-Many
BOM Projects	BOM Items	One-to-Many
Products	BOM Items	One-to-Many
Categories	Products	One-to-Many
Orders	Order Items	One-to-Many
Products	Order Items	One-to-Many
Users	Messages	One-to-Many
Users	Notifications	One-to-Many

ENTITY RELATIONSHIP DIAGRAM (CONCEPTUAL)



PRIMARY KEYS

Every table shall contain a unique primary key.

Table	Primary Key
users	id
categories	id
products	id
carts	id
cart_items	id
bom_projects	id
bom_items	id
orders	id
order_items	id
messages	id
notifications	id

FOREIGN KEYS

Table	Foreign Key	References
products	category_id	categories.id
carts	user_id	users.id
cart_items	cart_id	carts.id
cart_items	product_id	products.id
bom_projects	user_id	users.id
bom_items	project_id	bom_projects.id
bom_items	product_id	products.id
orders	user_id	users.id
order_items	order_id	orders.id
order_items	product_id	products.id
messages	sender_id	users.id
messages	receiver_id	users.id
notifications	user_id	users.id

NAMING CONVENTIONS

The following conventions shall be used throughout the database.

- Table names use lowercase plural nouns.
- Primary keys use id.
- Foreign keys use the format <entity>_id.
- Timestamp columns use created_at and updated_at.
- Boolean fields begin with is_.

DATA INTEGRITY RULES

The database shall enforce the following:

- Primary key uniqueness.
- Foreign key constraints.
- Required fields for critical data.
- Valid data types.
- Transaction support for multi-step operations such as checkout.

INDEXING STRATEGY

Indexes should be created for frequently queried columns.

Recommended indexes include:

- users.email
- products.name
- products.sku
- orders.order_number
- orders.status
- notifications.user_id
- messages.sender_id
- messages.receiver_id

BACKUP AND RECOVERY

Database backups are managed through the hosting provider.

Recommendations:

- Enable automatic backups.
- Monitor storage usage.
- Test restoration procedures before production deployment.

CHAPTER SUMMARY

This chapter defines the relational database design for OhmSim, including its entities, relationships, keys, constraints, and data management principles. The database structure presented here provides the foundation for implementing the application's backend services and ensures that data is stored consistently, securely, and efficiently.

CHAPTER 11 – API DESIGN

OVERVIEW

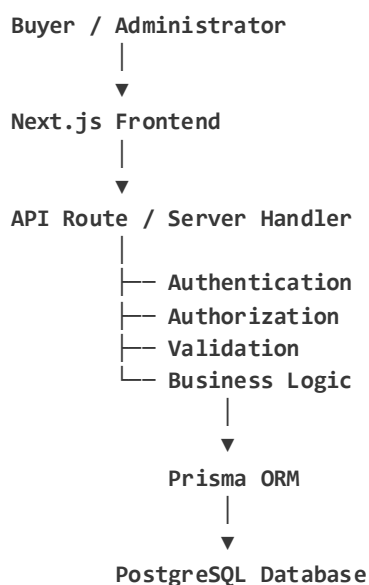
This chapter defines the Application Programming Interface (API) structure for OhmSim Version 1.0.

The API serves as the communication layer between the frontend application and backend services. It handles authentication, product data, shopping carts, BOM projects, orders, messaging, notifications, inventory, and administrative operations.

The API design follows a REST-style approach and uses JSON for request and response data.

API ARCHITECTURE

The general communication flow is:



The frontend must not communicate directly with the PostgreSQL database.

All protected operations must pass through the backend.

API DESIGN PRINCIPLES

The API shall follow these principles:

- Use HTTP methods according to the operation being performed.
- Return JSON responses.
- Validate incoming data on the server.
- Authenticate protected requests.
- Authorize requests according to the user's role.
- Return appropriate HTTP status codes.
- Keep endpoint naming consistent.

- Avoid exposing sensitive database information.
- Return predictable response structures.

HTTP METHODS

Method	Purpose	Example
GET	Retrieve data	Get products
POST	Create data	Create order
PATCH	Modify existing data	Update order status
DELETE	Remove data	Delete cart item

HTTP STATUS CODES

Status	Meaning	Example
200	Successful request	Product retrieved
201	Resource created	Order created
400	Invalid request	Missing required field
401	Unauthenticated	Login required
403	Unauthorized	Buyer attempting admin action
404	Resource not found	Product does not exist
409	Conflict	Duplicate email
422	Validation failure	Invalid quantity
500	Server error	Unexpected backend failure

API ENDPOINT STRUCTURE

OhmSim APIs shall use a consistent structure.

/api/<resource>

Examples:

```
/api/products
/api/orders
/api/cart
/api/bom
/api/messages
/api/notifications
```

Resource-specific operations use the resource identifier where appropriate.

```
/api/products/:id
/api/orders/:id
/api/cart/items/:id
```

AUTHENTICATION API

Method	Endpoint	Purpose	Access
POST	/api/auth/register	Create buyer account	Public
POST	/api/auth/login	Authenticate user	Public
POST	/api/auth/logout	End session	Authenticated
POST	/api/auth/forgot-password	Request password recovery	Public
POST	/api/auth/reset-password	Reset password	Public
GET	/api/auth/session	Retrieve current session	Authenticated

Authentication implementation will follow the selected authentication library and its supported session mechanism.

PRODUCT API

Method	Endpoint	Purpose	Access
GET	/api/products	Retrieve products	Public
GET	/api/products/:id	Retrieve product details	Public
POST	/api/products	Create product	Admin
PATCH	/api/products/:id	Update product	Admin
DELETE	/api/products/:id	Archive/delete product	Admin

CATEGORY API

Method	Endpoint	Purpose	Access
GET	/api/categories	Retrieve categories	Public
POST	/api/categories	Create category	Admin
PATCH	/api/categories/:id	Update category	Admin
DELETE	/api/categories/:id	Delete category	Admin

SHOPPING CART API

Method	Endpoint	Purpose	Access
GET	/api/cart	Retrieve buyer cart	Buyer
POST	/api/cart/items	Add product	Buyer
PATCH	/api/cart/items/:id	Update quantity	Buyer
DELETE	/api/cart/items/:id	Remove item	Buyer
DELETE	/api/cart	Clear cart	Buyer

BOM API

Method	Endpoint	Purpose	Access
GET	/api/bom	Retrieve buyer BOM projects	Buyer
POST	/api/bom	Create BOM project	Buyer
GET	/api/bom/:id	Retrieve BOM project	Owner
PATCH	/api/bom/:id	Rename/update BOM	Owner
DELETE	/api/bom/:id	Delete BOM	Owner
POST	/api/bom/:id/items	Add component	Owner
PATCH	/api/bom/:id/items/:itemId	Update quantity	Owner
DELETE	/api/bom/:id/items/:itemId	Remove component	Owner

ORDER API

Method	Endpoint	Purpose	Access
POST	/api/orders	Create order	Buyer
GET	/api/orders	Retrieve own orders	Buyer
GET	/api/orders/:id	Retrieve order details	Owner/Admin
PATCH	/api/orders/:id/status	Update order status	Admin

The backend must verify stock availability and order ownership before processing applicable operations.

INVENTORY API

Method	Endpoint	Purpose	Access
GET	/api/inventory	Retrieve inventory	Admin
GET	/api/inventory/low-stock	Retrieve low-stock items	Admin
PATCH	/api/inventory/:productId	Update stock	Admin

NOTIFICATION API

Method	Endpoint	Purpose	Access
GET	/api/notifications	Retrieve notifications	Authenticated
PATCH	/api/notifications/:id/read	Mark notification as read	Owner
PATCH	/api/notifications/read-all	Mark all as read	Owner

ADMINISTRATOR NOTIFICATION API

Method	Endpoint	Purpose	Access
POST	/api/admin/notifications	Create announcement	Admin
POST	/api/admin/notifications/send	Send notification	Admin

REPORTS API

Method	Endpoint	Purpose	Access
GET	/api/admin/reports/sales	Retrieve sales data	Admin
GET	/api/admin/reports/inventory	Retrieve inventory report	Admin
GET	/api/admin/reports/products	Retrieve product statistics	Admin
GET	/api/admin/reports/liquidation	Retrieve monthly liquidation data	Admin

REQUEST VALIDATION

Every API endpoint accepting user input shall validate the request before processing it.

Validation should verify:

- Required fields
- Data types
- String length
- Numeric ranges
- Valid identifiers
- User permissions
- Product availability
- Order ownership

For example, an order quantity must satisfy:

```
quantity > 0
```

and:

```
quantity <= available_stock
```

API RESPONSE STRUCTURE

Successful responses should follow a consistent structure.

```
{
  "success": true,
  "data": {
    "id": "product-id",
    "name": "ESP32 Development Board",
    "price": 350
  }
}
```

Error responses should provide a consistent structure.

```
{
  "success": false,
  "error": {
```

```

    "code": "PRODUCT_NOT_FOUND",
    "message": "The requested product was not found."
  }
}

```

AUTHENTICATION AND AUTHORIZATION

The API shall distinguish between authentication and authorization.

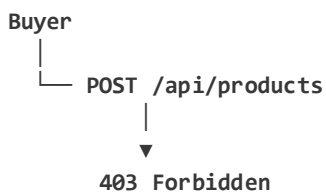
AUTHENTICATION

Determines: Who is making the request?

AUTHORIZATION

Determines: Is this user allowed to perform this operation?

Example:



An authenticated Buyer must not receive access to administrative endpoints.

API SECURITY

The backend shall enforce:

- Authentication checks.
- Role-based authorization.
- Server-side validation.
- Resource ownership checks.
- Secure session handling.
- Rate limiting where appropriate.
- Protection against unauthorized data access.
- Secure handling of environment variables.

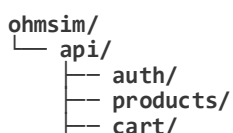
Sensitive credentials and service keys must never be included in frontend code.

API TESTING

The development team will use Bruno for API testing.

API collections shall be stored within the project repository so the team can version-control endpoint tests alongside the application.

Example structure:



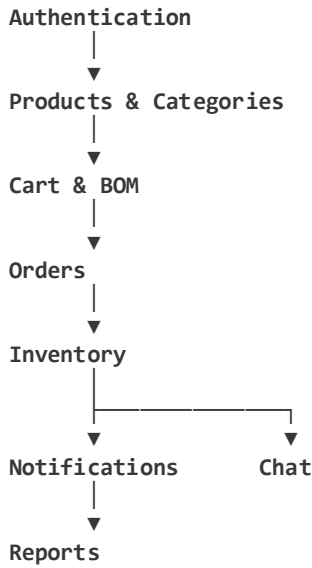
Each collection should contain tests for:

- Successful requests
- Invalid requests
- Unauthorized requests
- Missing resources
- Validation failures

- bom/
- orders/
- inventory/
- chat/
- notifications/

API DEVELOPMENT SEQUENCE

Backend development should follow the system dependencies rather than implementing endpoints randomly.



This sequence allows the team to establish the core data and authentication layers before implementing features that depend on them.

CHAPTER SUMMARY

This chapter establishes the API structure for OhmSim Version 1.0. It defines the endpoint organization, HTTP methods, access requirements, validation standards, response formats, security requirements, and API testing approach.

The API serves as the communication layer between the frontend, backend logic, database, and external services.

CHAPTER 12 – UI COMPONENT LIBRARY & DESIGN SYSTEM

OVERVIEW

This chapter defines the reusable interface components and visual standards used throughout OhmSim.

The design system exists to keep the Buyer and Administrator interfaces consistent while allowing the development team to build pages from reusable components instead of creating separate UI implementations for every screen.

The design system covers:

- Typography
- Colors
- Spacing
- Buttons
- Forms
- Cards
- Tables
- Navigation
- Feedback components
- Product components
- Responsive behavior
- Accessibility requirements

DESIGN SYSTEM OBJECTIVES

The OhmSim interface shall prioritize:

- Consistency – Similar actions should look and behave the same throughout the application.
- Usability – Users should understand available actions without unnecessary interaction.
- Responsiveness – Interfaces must adapt to desktop, tablet, and mobile screens.
- Reusability – Common interface elements should exist as reusable components.
- Accessibility – Components should remain usable with different input methods and screen sizes.
- Maintainability – UI changes should be manageable from centralized component implementations.

UI TECHNOLOGY

Technology	Purpose
React	Component architecture
Next.js	Application framework

Technology	Purpose
TypeScript	Type-safe development
Tailwind CSS	Styling
shadcn/ui	Reusable UI components
Lucide React	Icons
Sonner	In-app toast notifications

DESIGN TOKENS

Design tokens establish the reusable values used throughout the application.

Tokens should be defined centrally rather than repeatedly hard-coded inside individual components.

Primary token categories include:

- Colors
- Typography
- Spacing
- Border radius
- Shadows
- Breakpoints
- Component dimensions

COLOR SYSTEM

OhmSim shall maintain a consistent color hierarchy.

PRIMARY COLOR

Used for:

- Primary actions
- Important links
- Active navigation states
- Selected elements

SECONDARY COLOR

Used for:

- Secondary actions
- Supporting interface elements

SUCCESS

Used for:

- Successful operations
- Completed orders
- Successful confirmations

DESTRUCTIVE

Used for:

- Delete actions
- Cancellation
- Irreversible operations
- Error states

NEUTRAL

Used for:

- Backgrounds
- Borders
- Secondary text
- Disabled elements

The exact color values will be finalized during UI design

WARNING

Used for:

- Low-stock warnings
- Pending states
- Important notices

TYPOGRAPHY

Level	Usage
Display	Major landing-page headings
H1	Page title
H2	Major section
H3	Subsection
Body	Standard content
Small	Supporting information
Caption	Metadata

Typography should provide clear visual hierarchy without excessive font-size variation.

SPACING SYSTEM

Spacing shall use a consistent scale rather than arbitrary values.

The system should apply spacing consistently to:

- Page sections
- Cards
- Forms
- Navigation
- Tables
- Buttons
- Dialogs

Tailwind CSS spacing utilities will provide the primary implementation mechanism.

BUTTONS

Buttons represent actions throughout the application.

PRIMARY BUTTON

Used for the main action of a page.

Examples:

- Add to Cart
- Checkout
- Save Product

- Confirm Order

SECONDARY BUTTON

Used for supporting actions.

Examples:

- Cancel
- Back
- View Details

DESTRUCTIVE BUTTON

Used for actions involving deletion or cancellation.

Examples:

- Delete Product
- Cancel Order

ICON BUTTON

Used for compact actions.

Examples:

- Search
- Edit
- Delete
- Notifications
- More Options

Every button must provide a visible state for: Default, Hover, Focus, Active, Disabled, and Loading.

FORM COMPONENTS

Forms will use standardized components.

Required components include:

- Input
- Textarea
- Select
- Checkbox
- Radio Group
- Switch
- Date Picker
- Search Input

Every form should provide:

- Label
- Input
- Validation feedback

- Required-field indication where applicable
- Error state
- Submission state

PRODUCT CARD

The Product Card is a primary component within the Buyer Module.

It should display:

- Product image
- Product name
- Price
- Stock status
- Category or relevant metadata
- Primary action

Example:

Product Image
ESP32 Development Board ₱350.00 In Stock [View] [Add to Cart]

The component should remain reusable across Product Catalog, Search Results, Related Products, and BOM interfaces.

PRODUCT DETAIL COMPONENTS

The Product Details interface may contain:

- Product image gallery
- Product title
- Price
- Stock indicator
- Quantity selector
- Product description
- Technical specifications
- Datasheet link
- Add to Cart button
- Add to BOM button

Technical information should use structured layouts instead of large blocks of text.

NAVIGATION

BUYER NAVIGATION

The Buyer interface should provide access to major functions such as:

Home
Products
BOM
Cart
Orders
Chat
Notifications
Profile

The exact navigation arrangement will depend on the final responsive design.

ADMINISTRATOR NAVIGATION

The Administrator interface should provide access to:

Dashboard
Products
Categories
Inventory
Orders
Customers
Chat
Reports
Notifications
Settings

CARDS

Cards provide visual grouping for related information.

Common uses include:

- Product information
- Dashboard statistics
- Order summaries
- Notifications
- BOM projects

Cards should not replace tables when users need to compare large amounts of structured data.

DATA TABLES

Administrator interfaces will use tables for structured management data.

Examples:

- Products
- Inventory
- Orders
- Customers
- Reports

Standard table functionality includes:

- Column headers
- Row actions
- Sorting where required
- Filtering where required
- Pagination
- Loading state
- Empty state

BADGES

Badges communicate compact status information.

Status	Example
Active	Product is available
In Stock	Stock available
Low Stock	Stock below threshold
Out of Stock	No available stock
Pending	Order awaiting processing
Preparing	Order being prepared
Ready	Order ready for pickup
Completed	Order completed
Cancelled	Order cancelled

The same status should use the same visual treatment throughout the application.

DIALOGS AND MODALS

Dialogs should handle focused interactions without requiring navigation away from the current page.

Examples:

- Confirm deletion
- Confirm order cancellation
- Edit product
- Update inventory
- Create BOM

Destructive actions should require confirmation.

TOAST NOTIFICATIONS

Sonner will handle temporary in-app feedback.

Product added to cart.
Order placed successfully.
Inventory updated.
Unable to update product.

Toast notifications should communicate the result of an operation without replacing persistent system information.

LOADING STATES

The interface must provide visual feedback when data or operations require processing.

Possible implementations include:

- Skeleton components
- Loading indicators
- Disabled buttons
- Progress indicators

Skeleton loading should be preferred for content-heavy areas such as product lists and tables when practical.

EMPTY STATES

Every data-driven component should define an empty state.

EMPTY CART

Your cart is empty.
[Browse Products]

NO ORDERS

You don't have any orders yet.
[Browse Products]

NO SEARCH RESULTS

No products match your search.
Try changing your search or filters.

Empty states should explain what happened and provide an appropriate next action when one exists.

ERROR STATES

Errors must communicate:

- What failed.
- Whether the user needs to take action.
- What action the user should take when applicable.

Example:

Unable to load products.
[Try Again]

Technical error details should not be exposed to ordinary users.

RESPONSIVE DESIGN

OhmSim is a web/PWA application and must support different screen sizes.

Primary responsive targets:

- Mobile
- Tablet
- Desktop

Responsive behavior includes:

- Collapsing navigation
- Responsive product grids
- Scrollable data tables where necessary
- Stacked forms
- Adaptive dialogs
- Mobile-friendly touch targets

The interface must not depend on desktop-only interactions.

MOBILE NAVIGATION

On smaller screens, the primary navigation should transition into a compact navigation pattern.

Possible implementations include:

- Bottom navigation
- Collapsible sidebar
- Mobile drawer

The final pattern will be determined during UI/UX design based on the number and priority of buyer functions.

ACCESSIBILITY

Components should follow basic accessibility requirements.

The development team shall:

- Use semantic HTML.
- Associate labels with form controls.
- Provide keyboard-accessible interactions.
- Maintain visible focus states.
- Provide meaningful button labels.

- Provide alternative text for meaningful images.
- Avoid relying exclusively on color to communicate status.

COMPONENT NAMING CONVENTION

Reusable components shall use descriptive PascalCase names.

```
ProductCard
ProductGrid
ProductTable
OrderCard
OrderStatusBadge
CartItem
BomItem
NotificationItem
ChatMessage
SearchBar
ConfirmDialog
```

Components should represent a clear UI responsibility.

COMPONENT ORGANIZATION

A recommended structure is:

```
components /
├── ui /
│   ├── Button
│   ├── Input
│   ├── Dialog
│   ├── Table
│   └── Badge
├── buyer /
│   ├── ProductCard
│   ├── CartItem
│   ├── BomItem
│   ├── OrderCard
│   └── ChatMessage
├── admin /
│   ├── ProductTable
│   ├── InventoryTable
│   └── OrderTable
└── CustomerTable
```

The exact folder implementation can change during development as long as the team maintains consistent organization.

COMPONENT REUSABILITY RULES

Developers should reuse existing components before creating new versions.

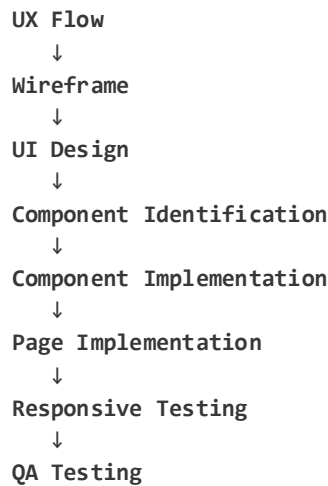
For example, the same ProductCard should be used wherever the application displays products in card format.

Developers should create a new component when:

- Existing components do not meet the required behavior.
- The new component represents a distinct reusable pattern.
- Reusing an existing component would create excessive conditional logic.

DESIGN-TO-DEVELOPMENT WORKFLOW

UI implementation should follow this sequence:



The UI and UX designers should coordinate with frontend developers before finalizing complex interactions.

DESIGN SYSTEM OWNERSHIP

The UI Designer and UX Designer are responsible for maintaining the design direction.

Frontend Developers are responsible for translating approved designs into reusable components.

The Project Manager coordinates changes when a design decision affects:

- Development scope
- Existing pages
- Development timeline
- Feature requirements

Major design changes should be discussed before implementation.

CHAPTER SUMMARY

This chapter establishes the UI component library and design system for OhmSim. It defines the visual structure, reusable components, responsive behavior, feedback patterns, accessibility requirements, and component organization that the frontend team will follow.

The design system provides a shared reference for the UI Designer, UX Designer, and Frontend Developers, reducing inconsistent implementations across the application.

CHAPTER 13 – DEVELOPMENT GUIDE

OVERVIEW

This chapter defines the development workflow for OhmSim Version 1.0.

The development process uses Agile methodology and GitHub-based version control. The workflow establishes how the team plans work, creates branches, writes commits, submits changes, performs reviews, integrates code, and moves completed work to QA.

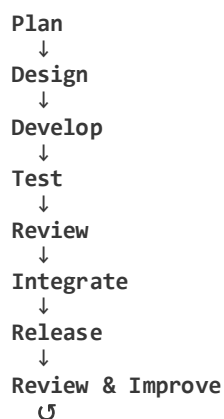
The goal is to prevent conflicting changes, keep the repository organized, and allow team members with different levels of development experience to contribute safely.

DEVELOPMENT METHODOLOGY

OhmSim will use an Agile development approach.

The project will be divided into smaller development tasks rather than attempting to complete the entire application at once.

The general cycle is:



The team will continuously evaluate completed work and adjust upcoming tasks based on project requirements and development progress.

DEVELOPMENT TEAM STRUCTURE

Role	Responsibility
Project Manager	Coordinates the project and development process
Data Gatherer	Collects and organizes required product/system data

Role	Responsibility
UI Designer	Designs the visual interface
UX Designer	Designs user flows and interaction structure
Frontend Developer 1	Implements frontend features
Frontend Developer 2	Implements frontend features
Backend Developer	Implements APIs, business logic, and database integration
QA Manager	Tests features and verifies requirements

PROJECT MANAGER

The Project Manager coordinates the overall development process.

Responsibilities include:

- Assigning development tasks.
- Maintaining the project backlog.
- Monitoring progress.
- Coordinating team communication.
- Resolving task conflicts.
- Reviewing major changes.
- Coordinating frontend, backend, design, and QA work.
- Managing development priorities.
- Approving major changes before implementation.

The Project Manager does not need to implement every feature personally. The role focuses on coordination, planning, and maintaining project direction.

DATA GATHERER

The Data Gatherer is responsible for preparing the information required by the application.

Responsibilities include:

- Collecting product information.
- Organizing product specifications.
- Preparing category information.
- Checking data completeness.
- Identifying missing information.
- Coordinating required data with the development team.

The Data Gatherer should provide structured data rather than sending unorganized information directly to developers.

UI DESIGNER

The UI Designer is responsible for the visual design of OhmSim.

Responsibilities include:

- Page layouts.
- Typography.
- Colors.
- Component appearance.
- Product cards.
- Forms.
- Tables.
- Buttons.
- Responsive layouts.
- Design system consistency.

The UI Designer provides approved designs that frontend developers can implement.

UX DESIGNER

The UX Designer is responsible for the user's interaction with the system.

Responsibilities include:

- User flows.
- Navigation structure.
- Interaction patterns.
- Information hierarchy.
- User journey analysis.
- Identifying usability problems.
- Mobile interaction planning.

The UX Designer works closely with the UI Designer before frontend implementation begins.

FRONTEND DEVELOPERS

The two Frontend Developers implement the client-side application.

Responsibilities include:

- Next.js development.
- React components.
- TypeScript implementation.
- Tailwind CSS styling.
- UI component integration.
- Page implementation.
- API integration.

- Form validation.
- Responsive behavior.
- Client-side state management.
- Frontend error handling.

Frontend Developers should follow the approved UI/UX designs rather than independently changing the application's structure.

BACKEND DEVELOPER

The Backend Developer is responsible for the server-side system.

Responsibilities include:

- API development.
- Authentication integration.
- Authorization.
- Database integration.
- Prisma implementation.
- Business rules.
- Order processing.
- Inventory operations.
- Chat functionality.
- Notification integration.
- Server-side validation.

The Backend Developer coordinates with the Frontend Developers when API requirements affect frontend implementation.

QA MANAGER

The QA Manager verifies whether implemented features meet their requirements.

Responsibilities include:

- Creating test cases.
- Testing completed features.
- Testing different user roles.
- Testing error conditions.
- Testing responsive behavior.
- Reporting defects.
- Verifying fixes.

- Performing regression testing.

QA should test completed functionality rather than waiting until the entire application is finished.

GITHUB REPOSITORY

The OhmSim source code will be maintained in a GitHub repository.

The repository serves as the central location for:

- Source code.
- Documentation.
- API collections.
- Database configuration.
- Development configuration.
- Version history.

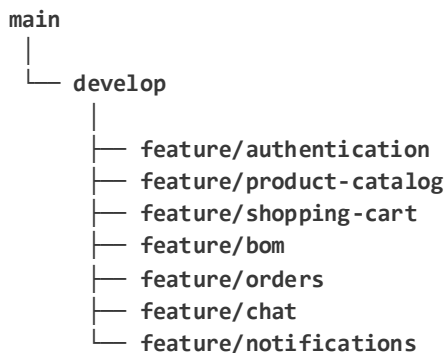
The repository should contain only project-related files. Sensitive information must never be committed.

Examples include:

```
.env
API_SECRET
DATABASE_PASSWORD
PRIVATE_KEY
FIREBASE_PRIVATE_KEY
```

BRANCH STRATEGY

The project will use a controlled branch structure.



MAIN

Contains stable versions of the application. Only reviewed and tested changes should reach main.

DEVELOP

Contains the integrated development version. Feature branches merge into develop after review and testing.

FEATURE BRANCHES

Each feature receives its own branch.

Examples:

```
feature/login
feature/product-catalog
feature/cart
feature/order-management
feature/chat
```

Developers should avoid working directly on main.

BRANCH OWNERSHIP

A developer who creates a feature branch is responsible for keeping it updated and resolving conflicts within their work.

```
Frontend Developer 1
└─ feature/product-catalog

Frontend Developer 2
└─ feature/shopping-cart

Backend Developer
└─ feature/product-api
```

Multiple developers should not independently create competing branches for the same feature without coordination.

TASK ASSIGNMENT

Tasks should be assigned through the project's chosen project-management board.

Each task should include:

- Task title.
- Description.
- Assigned member.
- Priority.
- Related feature.
- Acceptance criteria.
- Current status.

Example:

Task: Implement Product Catalog

Assigned:

Frontend Developer 1

Requirements:

- Product grid
- Search
- Category filter
- Product cards

- Loading state
- Empty state
- Responsive layout

Status:
In Progress

DEVELOPMENT WORKFLOW

Every feature should follow this workflow:



After sufficient testing, the stable version can be merged into main.

COMMIT STANDARDS

Commits should describe the change clearly.

Recommended format:

`type: description`

Examples:

```

feat: add product catalog
feat: implement shopping cart
fix: correct order quantity validation
fix: resolve mobile navigation issue
refactor: simplify product service
style: update product card layout
docs: update API documentation
test: add order validation tests
  
```

Common commit types:

Type	Purpose
feat	New functionality

Type	Purpose
fix	Bug fix
refactor	Code restructuring
style	UI/style changes
docs	Documentation
test	Tests
chore	Maintenance/configuration

Commits should represent a focused change rather than unrelated modifications.

EXAMPLE COMMIT WORKFLOW

A developer completes the shopping cart feature.

```
git checkout develop
git pull origin develop

git checkout -b feature/shopping-cart
```

After implementation:

```
git add .
git commit -m "feat: implement shopping cart"
git push -u origin feature/shopping-cart
```

The developer then creates a Pull Request targeting:

```
feature/shopping-cart
↓
develop
```

PULL REQUESTS

Every significant feature should use a Pull Request.

A Pull Request should include:

- What was changed.
- Why it was changed.
- Features implemented.
- Known limitations.
- Testing performed.
- Screenshots for significant UI changes.

Example:

Title:
feat: implement shopping cart

Description:

Implemented:

- Add item
- Remove item
- Quantity update
- Empty cart state
- Responsive cart layout

Tested:

- Desktop
- Mobile
- Quantity validation
- Empty cart

CODE REVIEW

At least one other developer should review significant code before merging it into develop.

Reviewers should check:

- Functionality.
- Code organization.
- Naming.
- TypeScript usage.
- Security.
- Validation.
- Duplicate code.
- Responsive behavior.
- Compatibility with existing features.

The reviewer should request changes when necessary rather than approving incomplete work.

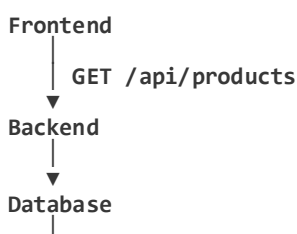
MERGE RULES

The team should follow these rules:

- Do not push directly to main.
- Do not merge unfinished features.
- Resolve merge conflicts before merging.
- Review significant Pull Requests.
- Test the feature before merging.
- Keep commits related to the feature.
- Do not commit secrets.

FRONTEND AND BACKEND COORDINATION

Frontend and backend development should use clearly defined API contracts.





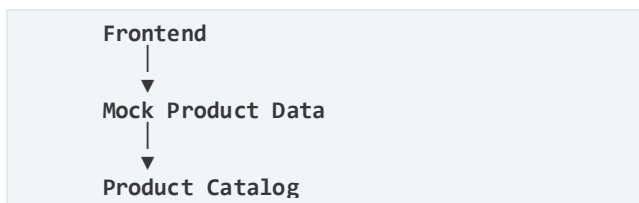
Before frontend integration begins, the team should agree on:

- Endpoint.
- HTTP method.
- Request format.
- Response format.
- Error format.
- Authentication requirements.

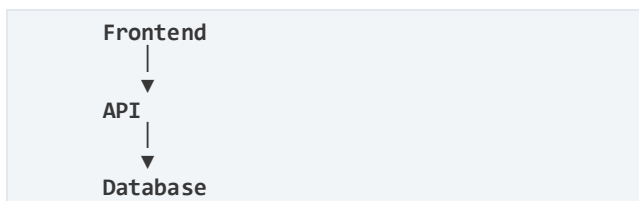
Bruno will be used to test the API independently before frontend integration.

FRONTEND-FIRST DEVELOPMENT

Because the project requires the frontend to be developed first, the team should initially use realistic mock data.



Once the backend becomes available:

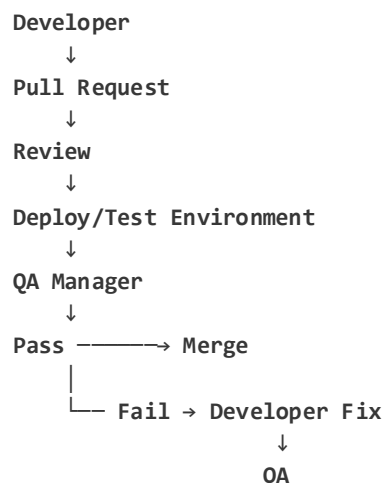


This allows UI development to continue without waiting for the entire backend to be completed.

The frontend team should avoid permanently embedding mock data into production functionality.

QA WORKFLOW

Once a feature reaches a testable state:



A failed test should return to the developer with a clear defect description.

BUG REPORTING

Every reported bug should include:

- Bug title.
- Description.
- Steps to reproduce.
- Expected result.
- Actual result.
- Severity.
- Screenshots or recordings when useful.
- Environment.

Example

Bug:

Cart quantity exceeds available stock

Steps:

1. Open product with 5 units available.
2. Add 5 units to cart.
3. Increase quantity to 6.

Expected:

System prevents quantity from exceeding stock.

Actual:

Cart accepts 6 units.

Severity:

High

DEVELOPMENT ENVIRONMENTS

The team should distinguish between:

LOCAL DEVELOPMENT

Used by individual developers.

localhost

development environment

Used for integration and team testing.

develop branch

production

Used for the deployed application.

main branch

Changes should move through these environments rather than going directly from local development to production.

ENVIRONMENT VARIABLES

Environment-specific configuration should be stored using environment variables.

```
DATABASE_URL
AUTH_SECRET
CLOUDINARY_CLOUD_NAME
CLOUDINARY_API_KEY
CLOUDINARY_API_SECRET
FIREBASE_PROJECT_ID
```

Actual values must not be committed to GitHub.

Developers should maintain a safe example file such as:

```
.env.example
```

Example:

```
DATABASE_URL=
AUTH_SECRET=
CLOUDINARY_CLOUD_NAME=
CLOUDINARY_API_KEY=
CLOUDINARY_API_SECRET=
```

DEVELOPMENT RULES FOR NEW TEAM MEMBERS

Team members who are unfamiliar with Git or the project should follow a controlled onboarding process.

- Step 1 — Clone the repository.
- Step 2 — Install dependencies.
- Step 3 — Configure environment variables.
- Step 4 — Run the application locally.
- Step 5 — Understand the folder structure.
- Step 6 — Create a small practice branch.
- Step 7 — Make a controlled change.
- Step 8 — Create a Pull Request.
- Step 9 — Receive review feedback.
- Step 10 — Proceed to assigned feature development.

This reduces the risk of inexperienced contributors accidentally modifying the stable codebase.

RECOMMENDED REPOSITORY STRUCTURE

```
ohmsim/
|
├─ app/
├─ components/
├─ hooks/
├─ lib/
├─ services/
```

```

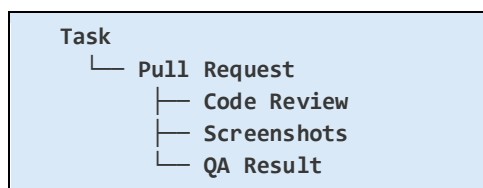
|— types/
|— prisma/
|— public/
|— api/
|— docs/
|
|— .env.example
|— .gitignore
|— package.json
|— README.md
|— ...

```

The docs/ directory can contain project documentation, while api/ contains the Bruno API collection.

DEVELOPMENT COMMUNICATION

The development team should keep feature-related communication connected to the relevant task or Pull Request.



This creates a traceable record of why a change was implemented and how it was verified.

DEFINITION OF DONE

A feature should not be considered complete merely because the developer finished coding it.

A feature is Done when:

- Required functionality works.
- UI matches the approved design.
- Responsive behavior works.
- Validation works.
- Error states work.
- Code has been reviewed.
- QA has tested it.
- Identified defects have been resolved.
- Documentation has been updated when required.
- The feature has been merged into the appropriate branch.

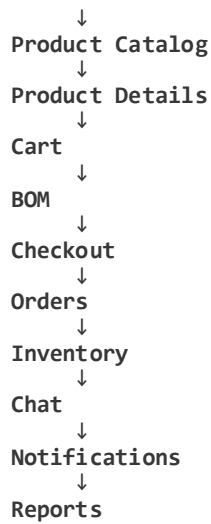
DEVELOPMENT PRIORITY

Development should follow system dependencies.

```

Project Setup
  ↓
Design System
  ↓
Authentication

```



This order allows the team to establish foundational functionality before implementing dependent features.

CHAPTER 14 – TESTING STRATEGY

OVERVIEW

This chapter defines the testing strategy for OhmSim Version 1.0.

Testing verifies that the application meets its functional requirements, follows the defined business rules, handles invalid operations correctly, and provides a consistent experience across supported devices.

Testing will occur throughout development rather than only after the entire application has been completed.

The QA process will involve:

- Functional testing
- Integration testing
- API testing
- Database testing
- UI testing
- Responsive testing
- Security testing
- Regression testing
- User acceptance testing

TESTING OBJECTIVES

The testing process aims to verify that:

- Features operate according to their specifications.
- User inputs are validated correctly.
- Unauthorized users cannot access restricted functionality.
- Data remains consistent across related features.
- API endpoints return correct responses.
- Database operations preserve data integrity.

- Interfaces work across supported screen sizes.
- Errors are handled appropriately.
- Fixed defects do not reappear.
- The final system satisfies the project's acceptance criteria.

QA RESPONSIBILITY

The QA Manager coordinates the testing process.

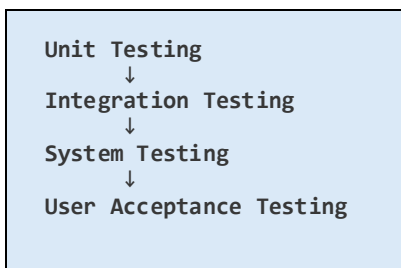
Responsibilities include:

- Preparing test cases.
- Executing tests.
- Recording defects.
- Assigning severity.
- Verifying fixes.
- Performing regression tests.
- Maintaining testing documentation.
- Conducting final acceptance testing.

Developers remain responsible for testing their own code before submitting it for QA.

TESTING LEVELS

OhmSim will use several levels of testing.



Each level examines the system from a different perspective.

UNIT TESTING

Unit testing verifies individual functions or components independently.

Examples:

- Quantity validation.
- Price calculation.
- Order status validation.
- Form validation.
- Utility functions.

Example:

Input:
Price = ₱100
Quantity = 3

Expected:
Subtotal = ₱300

Unit tests should focus on predictable logic and edge cases.

- Product filtering.

COMPONENT TESTING

Frontend components should be tested independently when they contain meaningful behavior.

Examples:

- ProductCard
- CartItem
- SearchBar
- OrderStatusBadge
- LoginForm
- CheckoutForm

Testing should verify:

- Correct rendering.
- User interaction.
- Validation.
- State changes.
- Error behavior.

INTEGRATION TESTING

Integration testing verifies whether multiple parts of the system work together correctly.

Examples:

```
graph TD; A[Login Form] --> B[Authentication API]; B --> C[Database]; C --> D[Session]
```

Another example:

```
graph TD; A[Add to Cart] --> B[Cart API]; B --> C[Database]; C --> D[Updated Cart]; D --> E[Frontend]
```

Integration testing should identify problems between components that individual unit tests cannot detect.

API TESTING

API testing will use Bruno.

Each API endpoint should be tested for:

- Successful request.
- Invalid request.
- Missing fields.
- Invalid identifiers.
- Unauthorized access.
- Incorrect role.
- Missing resource.
- Database failure.

- Boundary conditions.

Product API

GET /api/products

Expected:
HTTP 200

GET /api/products/nonexistent

Expected:
HTTP 404

POST /api/products

Expected:
HTTP 401 or 403

The expected status depends on the authentication state.

DATABASE TESTING

Database operations must be tested to verify data integrity.

Testing should include:

- Record creation.
- Record updates.
- Record deletion/archive.
- Foreign-key relationships.
- Duplicate prevention.
- Required fields.
- Transaction behavior.
- Concurrent updates where relevant.

Critical operations such as checkout require special attention because multiple database records may change during one operation.

AUTHENTICATION TESTING

REGISTRATION

Test:

- Valid registration.
- Duplicate email.
- Invalid email.
- Missing required fields.
- Weak password.
- Password mismatch.

LOGIN

Test:

- Correct credentials.
- Incorrect password.
- Unknown account.
- Empty fields.
- Session creation.

PASSWORD RECOVERY

Test:

LOGOUT

Test:

- Session termination.
- Access to protected pages after logout.

- Valid account.
- Unknown account.
- Invalid reset token.
- Expired reset token.

AUTHORIZATION TESTING

Authorization testing verifies role restrictions.

BUYER

A Buyer should be able to:

- Browse products.
- Manage their cart.

A Buyer must not be able to:

- Create products.
- Modify inventory.
- Change another user's order.
- Access administrator reports.

- Manage their BOM projects.
- Place orders.
- View their orders.
- Use chat.
- Manage their own profile.

ADMINISTRATOR

An Administrator should be able to access administrative functionality according to the defined permissions.

PRODUCT TESTING

Product functionality should be tested for:

- Product listing.
- Product search.
- Category filtering.
- Product details.
- Product availability.
- Product creation.
- Product modification.
- Product archival.
- Invalid product data.

Example:

Stock = 0

Expected:
Product displays "Out of Stock"

SHOPPING CART TESTING

Test cases should include:

Test	Expected Result
Add product	Product appears in cart
Add same product again	Quantity increases
Remove product	Product disappears
Increase quantity	Quantity updates
Set quantity to 0	Invalid or removes item according to implementation
Exceed stock	Operation rejected
Clear cart	Cart becomes empty

BOM TESTING

Test:

- Create BOM.
- Rename BOM.
- Add product.
- Remove product.
- Update quantity.
- Create multiple BOM projects.
- Transfer BOM items to cart.
- Verify ownership restrictions.

A Buyer must not access another Buyer's BOM project.

CHECKOUT TESTING

Checkout represents a critical transaction and requires extensive testing.

Test:

- Empty cart.
- Valid cart.
- Out-of-stock product.
- Insufficient stock.
- Invalid customer information.
- Successful order creation.
- Duplicate submission.
- Database failure during transaction.

Expected successful flow:

```

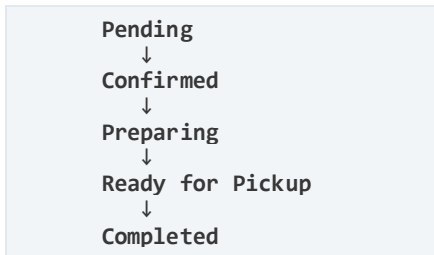
Cart
↓
Stock Validation
↓
Order Creation
↓
Order Items Creation
↓
Required Inventory Update
↓
Cart Update/Clear
↓
Order Confirmation

```

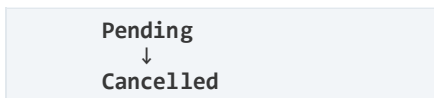
The system must avoid creating

ORDER TESTING

Test every supported status transition.



Also test:



Invalid transitions must be rejected.

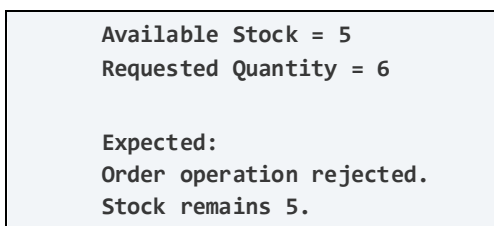
For example, Completed → Preparing should not be permitted.

INVENTORY TESTING

Inventory tests should verify:

- Correct stock display.
- Stock updates.
- Restocking.
- Low-stock detection.
- Out-of-stock detection.
- Prevention of negative stock.
- Synchronization with order processing.

Example:



CHAT TESTING

Test:

- Creating a conversation.
- Sending messages.
- Receiving messages.
- Message ordering.
- Conversation history.
- Empty messages.
- Unauthorized conversations.

NOTIFICATION TESTING

Notifications require testing through two channels.

IN-APP NOTIFICATIONS

Verify:

- Notification creation.
- Display.
- Read/unread state.
- Mark as read.
- Mark all as read.

PUSH NOTIFICATIONS

Using Firebase Cloud Messaging, test:

- Permission request.
- Token registration.
- Notification delivery.
- Order status notifications.
- Chat notifications.
- Behavior when the application is not active.

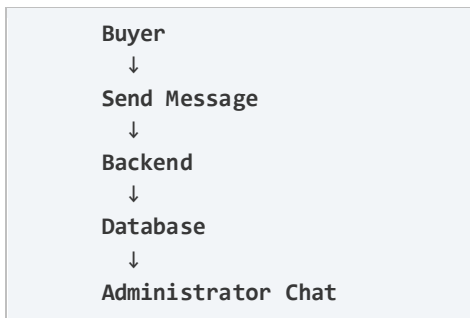
UI TESTING

UI testing verifies that interfaces match the approved designs and behave correctly.

Test:

- Buttons.
- Forms.
- Navigation.
- Dialogs

- Network interruption.



RESPONSIVE TESTING

The application must be tested on:

MOBILE

- Small smartphones
- Large smartphones

TABLET

- Portrait
- Landscape

DESKTOP

- Standard laptop
- Larger desktop display

The QA Manager should verify:

- Navigation.
- Product grids.
- Forms.
- Tables.
- Dialogs.
- Chat.
- Checkout.
- Touch interactions.

BROWSER TESTING

The application should be tested on the primary browsers supported by the project.

Recommended baseline:

- Google Chrome
- Microsoft Edge
- Mozilla Firefox
- Safari where available for testing

The team should prioritize browsers used by the project's target users.

PERFORMANCE TESTING

Performance testing should evaluate:

- Initial page loading.
- Product catalog loading.

- Search response.
- API response time.
- Image loading.
- Large product lists.
- Large order lists.
- Chat loading.

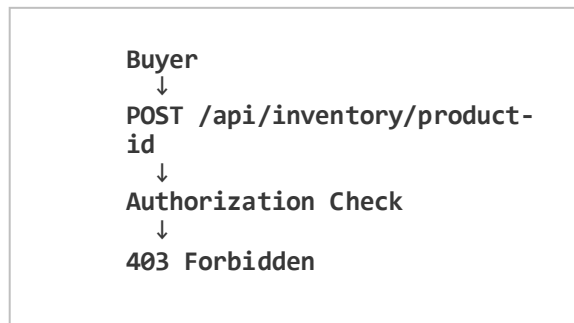
Potential optimization areas include:

- Image optimization.
- Pagination.
- Database indexing.
- API query optimization.
- Lazy loading.
- Caching.

SECURITY TESTING

Security testing should verify:

- Authentication enforcement.
- Authorization enforcement.
- Session handling.
- Input validation.
- Resource ownership.
- Protected API endpoints.
- Sensitive information handling.
- Environment variable protection.



ERROR HANDLING TESTING

The QA Manager should deliberately test failure conditions.

Examples:

- Server unavailable.
- Database unavailable.
- Invalid API response.
- Network interruption.
- Invalid form data.
- Expired session.
- Missing resource.
- Duplicate operation.

The application should display an appropriate user-facing response rather than exposing raw server errors.

REGRESSION TESTING

Regression testing verifies that previously working features remain functional after changes.

Regression testing should occur after:

- Bug fixes.
- Major feature additions.
- Database changes.
- Authentication changes.
- API changes.
- Major UI refactoring.

A regression checklist should cover critical flows.

CRITICAL USER FLOW TEST

The following flow should be included in every major regression cycle:

```
graph TD
    Register --> Login
    Login --> BrowseProducts[Browse Products]
    BrowseProducts --> ViewProduct[View Product]
    ViewProduct --> AddToCart[Add to Cart]
    AddToCart --> Checkout
    Checkout --> ViewOrder[View Order]
    ViewOrder --> AdminUpdates[Administrator Updates Order]
    AdminUpdates --> BuyerNotification[Buyer Receives Notification]
    BuyerNotification --> OrderCompleted[Order Completed]
```

Chat and BOM functionality should also receive separate regression coverage.

TEST CASE FORMAT

Each test case should use a consistent format.

Field	Description
Test ID	Unique identifier
Feature	Feature being tested
Preconditions	Required starting state
Test Steps	Actions performed
Test Data	Values used
Expected Result	Correct system behavior

Field	Description
Actual Result	Observed behavior
Status	Pass/Fail
Severity	If failed
Tester	Person performing test
Date	Testing date

TC-CART-001

Feature: Shopping Cart

Precondition: User is logged in.

Steps: 1. Open Product Catalog. 2. Select a product. 3. Click Add to Cart. 4. Open Cart.

Expected Result: Product appears in the cart with the correct quantity and price.

Status: Pass / Fail

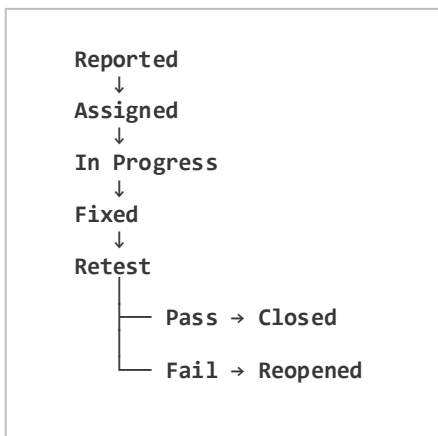
DEFECT SEVERITY

Defects should be categorized according to their impact.

Severity	Description
Critical	Prevents major system operation
High	Major feature does not work
Medium	Feature works incorrectly but has a workaround
Low	Minor issue with limited impact

Examples: Critical — Checkout creates duplicate orders. High — Users cannot place orders. Medium — Cart quantity display does not update immediately. Low — Minor spacing issue on a product card.

DEFECT LIFECYCLE



A defect should not be marked as closed until QA verifies the fix.

USER ACCEPTANCE TESTING

User Acceptance Testing (UAT) determines whether the completed system satisfies the project's practical requirements.

UAT should focus on real user scenarios rather than implementation details.

Example Buyer scenario:

```
graph TD; A[Login] --> B[Find electronic component]; B --> C[View specifications]; C --> D[Add component to cart]; D --> E[Checkout]; E --> F[Track order]; F --> G[Receive ready-for-pickup notification];
```

Example Administrator scenario:

```
graph TD; A[Login] --> B[View pending order]; B --> C[Confirm order]; C --> D[Update preparation status]; D --> E[Mark order ready]; E --> F[Complete order];
```

ACCEPTANCE CRITERIA

A feature should pass QA when:

- Required functionality works.
- Expected outputs are produced.
- Invalid operations are handled.
- Authorization works.
- Data is stored correctly.
- UI states work.
- Responsive behavior works.
- No unresolved Critical defects remain.

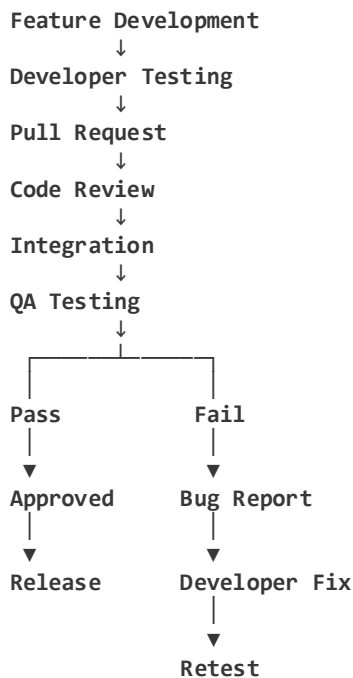
RELEASE CRITERIA

A production release should require:

- All critical features implemented.
- Critical and High defects resolved.
- Required regression tests passed.
- API tests passed.
- Database operations verified.
- Authentication verified.
- Authorization verified.
- Responsive testing completed.
- Production environment verified.
- Required documentation updated.

TESTING WORKFLOW

The complete QA workflow is:



CHAPTER SUMMARY

This chapter establishes the testing and quality assurance strategy for OhmSim. It defines testing levels, feature-specific testing requirements, API and database testing, responsive testing, security testing, defect management, regression testing, user acceptance testing, and release criteria.

Testing will occur continuously throughout development. The QA Manager will coordinate formal testing, while developers remain responsible for validating their own changes before submitting them for review.

CHAPTER 15 – DEPLOYMENT & RELEASE GUIDE

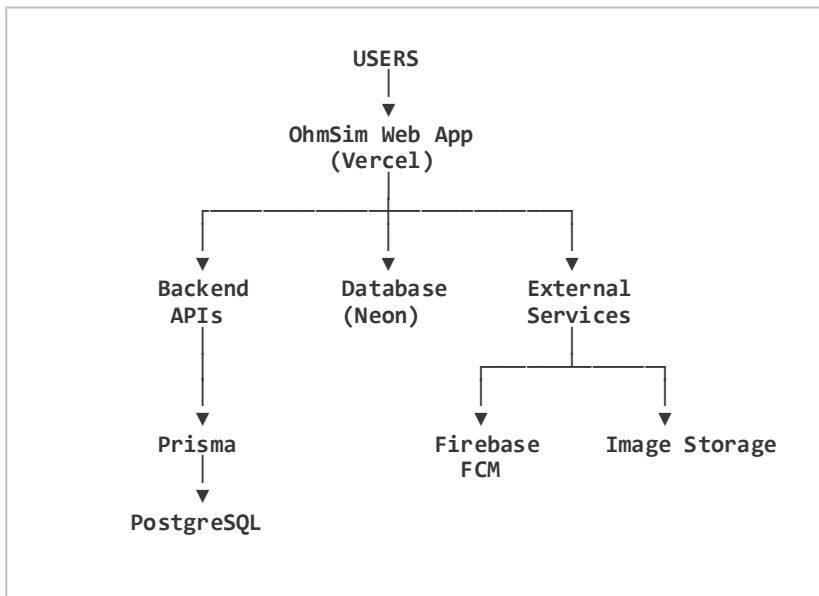
OVERVIEW

This chapter defines how OhmSim moves from local development to a deployed production application.

The deployment process covers application deployment, database deployment, environment variables, external services, production configuration, build verification, release workflow, post-deployment testing, and rollback procedures.

The deployment process separates development from production so unfinished features do not directly affect the live application.

DEPLOYMENT ARCHITECTURE



The exact external services used by the final implementation should match the approved project stack.

DEPLOYMENT ENVIRONMENTS

Environment	Purpose
Local	Individual development
Development	Team integration and testing
Production	Final deployed application

The general flow is:



PRODUCTION HOSTING

The OhmSim web application will be deployed using Vercel.

Vercel will host the Next.js application and provide the production deployment environment.

The deployment process should connect the GitHub repository to the Vercel project. A successful push to the configured production branch should trigger the appropriate deployment process.

GITHUB-TO-VERCEL WORKFLOW



The team should not use production deployment as a substitute for QA testing.

BUILD PROCESS

Before production deployment, the application must successfully complete its build process.

The build process should verify:

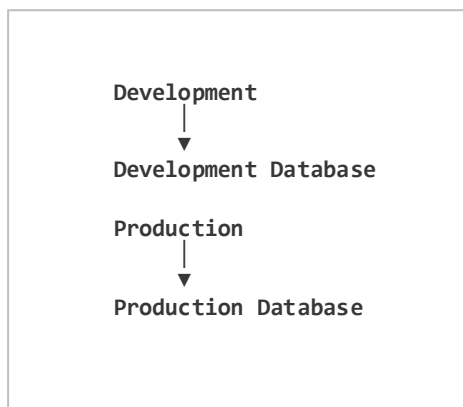
- TypeScript compilation.
- Dependency resolution.
- Next.js build.
- API compilation.
- Database client generation.
- Required environment variables.
- Production configuration.

A failed build must prevent the release from being considered complete.

PRODUCTION DATABASE

The production database will use PostgreSQL hosted through Neon.

The production database must remain separate from local development databases.



Developers must not use the production database for ordinary experimentation.

DATABASE MIGRATION WORKFLOW

```
Developer
↓
Modify Prisma Schema
↓
Create Migration
↓
Test Locally
↓
Review
↓
Apply to Development
↓
QA
↓
Apply to Production
```

Examples of schema changes include: Adding a table, adding a field, creating an index, changing a relationship, or adding a constraint.

Database migrations should be reviewed before production deployment.

ENVIRONMENT VARIABLES

Production credentials must be configured through the deployment platform rather than committed to GitHub.

```
DATABASE_URL
AUTH_SECRET
NEXT_PUBLIC_APP_URL

FIREBASE_PROJECT_ID
FIREBASE_CLIENT_EMAIL
FIREBASE_PRIVATE_KEY
```

Additional variables will depend on the external services selected for the final implementation. Sensitive values must remain private.

.ENV MANAGEMENT

Local development may use:

```
.env.local
```

A template should be maintained:

```
.env.example
```

Example:

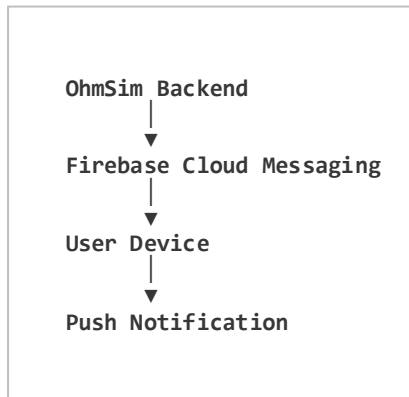
```
DATABASE_URL=
AUTH_SECRET=
NEXT_PUBLIC_APP_URL=
```

The .env.example file contains variable names but not production credentials.

FIREBASE CLOUD MESSAGING DEPLOYMENT

OhmSim uses Firebase Cloud Messaging (FCM) for push notifications.

The production implementation must configure the Firebase project and notification credentials correctly.



Production Firebase credentials must not be exposed through publicly accessible frontend code.

PUSH NOTIFICATION VERIFICATION

After deployment, the team should test:

- User grants notification permission.
- Device/browser receives a notification token.
- Token is associated with the appropriate user.
- Backend sends a notification.
- Firebase processes the notification.
- User receives the notification.

Notification behavior works when the application is not actively open.

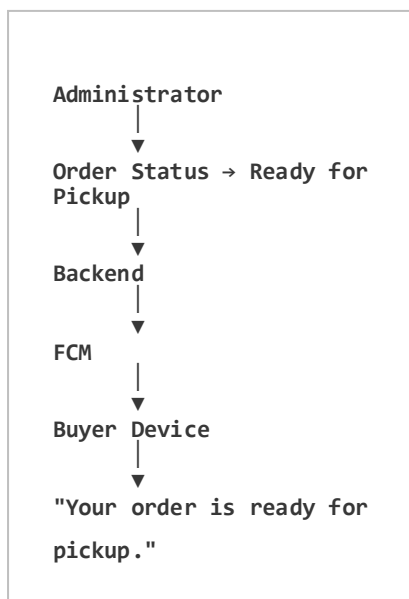


IMAGE STORAGE

If OhmSim uses external image storage, production credentials and configuration must be stored as environment variables.

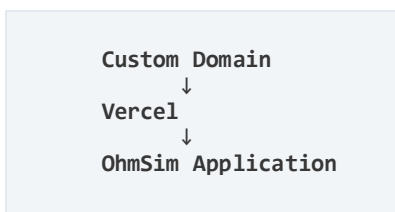
Product images should not rely on local development storage.



The database should store the required image reference rather than unnecessarily storing large image files directly inside PostgreSQL.

15.15 DOMAIN CONFIGURATION

If the project uses a custom domain, the domain should point to the production deployment.



The production URL should use HTTPS. The final domain will be configured after the application deployment is stable.

PRODUCTION CONFIGURATION CHECKLIST

APPLICATION

- ☐ Production build succeeds.
- ☐ No development-only configuration remains.
- ☐ Required environment variables exist.
- ☐ Debugging output is removed or disabled.
- ☐ Production API configuration is correct.

UI

- ☐ Responsive layout works.
- ☐ Loading states work.
- ☐ Empty states work.
- ☐ Error states work.
- ☐ Navigation works.

DATABASE

- ☐ Production database exists.
- ☐ Prisma configuration is correct.
- ☐ Required migrations are ready.
- ☐ Database connection works.

AUTHENTICATION

- ☐ Login works.
- ☐ Registration works.
- ☐ Sessions work.
- ☐ Logout works.
- ☐ Protected routes work.

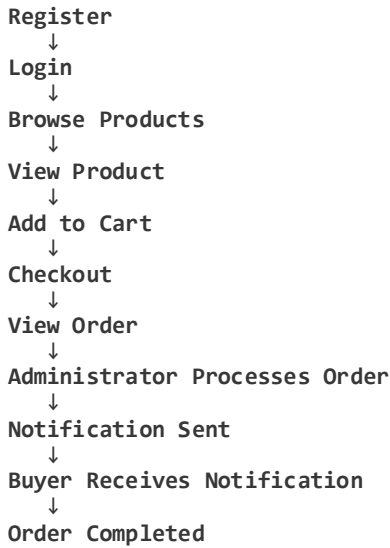
FIREBASE

- ☐ Firebase project is configured.
- ☐ Push notification configuration is correct.
- ☐ Notification permissions work.
- ☐ Test notification is delivered.

PRE-DEPLOYMENT TESTING

The QA Manager should perform a final production-readiness test before release.

The minimum critical flow is:



A failure in a critical step should block the production release until resolved.

PRODUCTION DEPLOYMENT

The recommended release process is:

- Step 1 — Complete Development: Feature development must be completed and merged into develop.
- Step 2 — QA: QA tests the integrated application.
- Step 3 — Fix Defects: Developers resolve identified issues.
- Step 4 — Regression Testing: QA repeats critical tests to ensure fixes did not introduce new problems.
- Step 5 — Merge to Main: Approved changes are merged into main.
- Step 6 — Vercel Deployment: Vercel builds and deploys the production application.
- Step 7 — Production Verification: The team verifies the deployed application using the production environment.

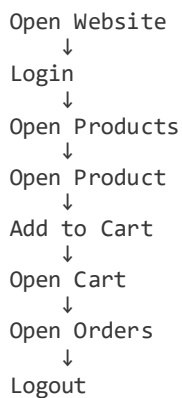
POST-DEPLOYMENT VERIFICATION

After deployment, the team should immediately verify:

- Application loads.
- Login works.
- Registration works.
- Product catalog loads.
- Product details load.
- Cart operations work.
- Checkout works.
- Orders appear correctly.
- Administrator functions work.
- Chat works.
- Notifications work.
- Database operations work.
- No critical production errors appear.

PRODUCTION SMOKE TEST

A smoke test provides a short verification after deployment.



```
graph TD; A[Open Website] --> B[Login]; B --> C[Open Products]; C --> D[Open Product]; D --> E[Add to Cart]; E --> F[Open Cart]; F --> G[Open Orders]; G --> H[Logout];
```

Open Website
↓
Login
↓
Open Products
↓
Open Product
↓
Add to Cart
↓
Open Cart
↓
Open Orders
↓
Logout

The Administrator should perform a separate administrative smoke test.

MONITORING

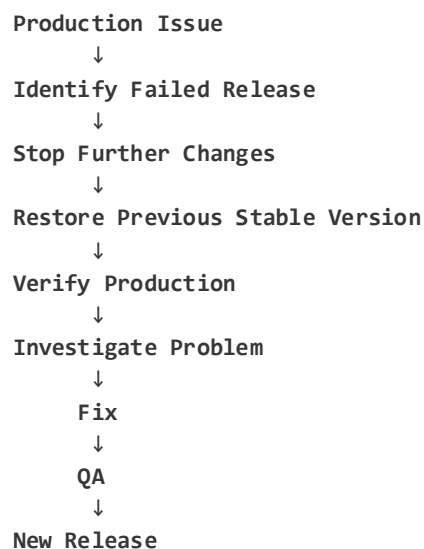
The team should monitor:

- Deployment status.
- Application errors.
- API failures.
- Database errors.
- Authentication failures.
- Notification failures.
- Performance problems.

The team should investigate repeated failures rather than treating them as isolated incidents.

ROLLBACK STRATEGY

If a production deployment introduces a critical problem, the team should revert to the previous stable version.



```
graph TD; A[Production Issue] --> B[Identify Failed Release]; B --> C[Stop Further Changes]; C --> D[Restore Previous Stable Version]; D --> E[Verify Production]; E --> F[Investigate Problem]; F --> G[Fix]; G --> H[QA]; H --> I[New Release];
```

Production Issue
↓
Identify Failed Release
↓
Stop Further Changes
↓
Restore Previous Stable Version
↓
Verify Production
↓
Investigate Problem
↓
Fix
↓
QA
↓
New Release

The team should avoid applying untested emergency changes directly to production.

DATABASE ROLLBACK CONSIDERATIONS

Application rollback and database rollback are separate concerns.

A previous application version might depend on a database structure introduced by a newer migration.

Therefore:

- Database migrations must be planned before deployment.
- Destructive schema changes require additional review.
- Production database backups should be available.
- The team should test migration behavior before release.

RELEASE VERSIONING

OhmSim should use a consistent versioning system.

v1.0.0

Version structure:

MAJOR.MINOR.PATCH

MAJOR

Used for significant incompatible changes.

v2.0.0

MINOR

Used for new backward-compatible functionality.

v1.1.0

PATCH

Used for bug fixes.

v1.0.1

RELEASE NOTES

Each production release should document the major changes.

OhmSim v1.0.0

Added:

- User authentication
- Product catalog
- Shopping cart
- BOM management
- Order management
- Administrator inventory
- Buyer-administrator chat
- Push notifications

Fixed:

- Initial production defects

Known Issues:

- None

The release notes should remain concise and focused on changes relevant to the project.

DEPLOYMENT RESPONSIBILITIES

Activity	Responsible Role
Prepare feature	Assigned Developer
Code review	Developer / Project Manager
QA testing	QA Manager
Production approval	Project Manager
Application deployment	Assigned Developer / Project Manager
Database migration	Backend Developer
Production verification	QA Manager
Release documentation	Project Manager

PRODUCTION ACCESS

Production credentials and deployment permissions should be restricted to authorized team members.

The team should avoid sharing:

- Database passwords.
- Authentication secrets.
- Firebase private credentials.
- Hosting credentials.
- API secrets.

Team members who only need to develop the frontend should not receive production database credentials.

DEPLOYMENT FAILURE HANDLING



Developers should not repeatedly redeploy without identifying the cause of the failure.

RELEASE CHECKLIST

CODE

- ☐ Required features complete.
- ☐ Code reviewed.
- ☐ No unresolved Critical defects.
- ☐ No known blocking High defects.

DATABASE

- ☐ Migration tested.
- ☐ Production database verified.
- ☐ Backup/recovery procedure available.

API

- ☐ API tests passed.
- ☐ Authentication tested.
- ☐ Authorization tested.
- ☐ Error responses verified.

FRONTEND

- ☐ UI verified.
- ☐ Responsive behavior verified.
- ☐ Forms tested.
- ☐ Loading states tested.
- ☐ Error states tested.

EXTERNAL SERVICES

- ☐ Firebase configured.
- ☐ Push notifications tested.
- ☐ Image storage tested if applicable.

DEPLOYMENT

- ☐ Production build successful.
- ☐ Environment variables verified.
- ☐ Production smoke test passed.
- ☐ Release documented.

CHAPTER 16 – SECURITY AND DATA PROTECTION

OVERVIEW

This chapter defines the security requirements for OhmSim Version 1.0.

Security must be considered throughout development rather than added after implementation. The system must protect user accounts, personal information, product data, orders, messages, administrative functions, database credentials, and external-service credentials.

The security strategy focuses on:

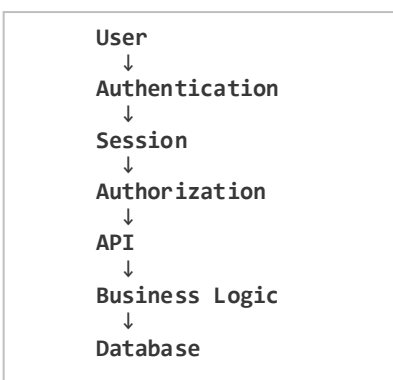
- Authentication
- Authorization
- Password protection
- Session security
- API security
- Input validation
- Database security
- Data protection
- Secret management
- File and image security
- Push notification security
- Administrative access
- Security testing
- Incident response

SECURITY OBJECTIVES

OhmSim should maintain the following security objectives:

- Confidentiality – Prevent unauthorized access to protected information.
- Integrity – Prevent unauthorized modification of system data.
- Availability – Keep core application functions available to legitimate users.
- Account Protection – Protect user credentials and sessions.
- Access Control – Restrict functions according to user roles.
- Data Protection – Store and transmit information securely.
- Secret Protection – Prevent credentials and private keys from entering the source repository.

SECURITY ARCHITECTURE

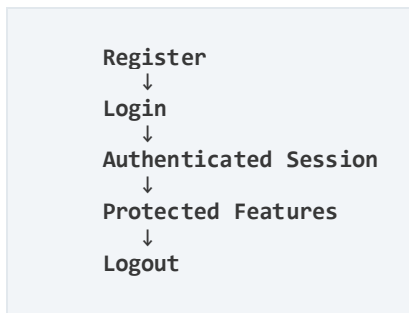


Every protected operation should pass through the appropriate authentication and authorization checks.

The frontend must not serve as the only security layer. Server-side APIs must enforce access restrictions independently.

AUTHENTICATION

Authentication determines whether a user has a valid account and session.



Protected functionality should require an authenticated session.

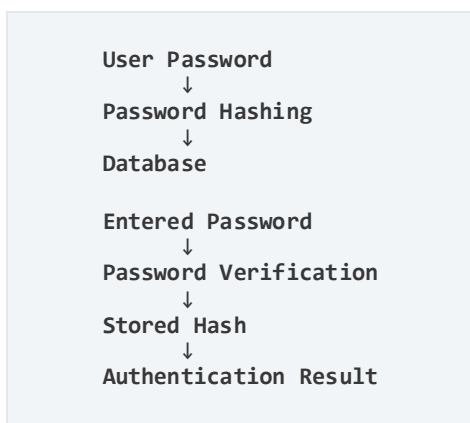
Examples include:

- Cart operations tied to an account.
- BOM management.
- Order history.
- Chat.
- Profile management.
- Administrative functions.

PASSWORD SECURITY

User passwords must never be stored as plain text.

Passwords should be processed using a secure password-hashing mechanism before storage. The database should contain a password hash rather than the original password.



The system must not log passwords.

PASSWORD REQUIREMENTS

The final password policy should define:

- Minimum password length.
- Required account fields.
- Password confirmation during registration when applicable.
- Password reset requirements.
- Rate limiting for repeated login attempts.

The application should provide clear validation messages without exposing sensitive authentication information.

SESSION SECURITY

Authenticated sessions must be protected from unauthorized use.

The authentication implementation should consider:

- Secure cookies where applicable.
- HttpOnly cookies for sensitive session information.
- Appropriate SameSite configuration.
- Session expiration.
- Logout behavior.
- Session invalidation.

The application should not store sensitive authentication credentials in insecure client-side storage.

AUTHORIZATION

Authorization determines what an authenticated user is allowed to do.

OhmSim uses role-based access control.

Buyer
Administrator

The backend must enforce role permissions.

BUYER PERMISSIONS

A Buyer should have access to:

- Product browsing.
- Product searching.
- Product details.
- Shopping cart.
- BOM management.
- Checkout.
- Personal orders.
- Personal chat conversations.
- Personal account information.

A Buyer must not access administrative functions.

ADMINISTRATOR PERMISSIONS

An Administrator should have access to authorized management functions, including:

- Product management.
- Category management.
- Inventory management.
- Order management.
- Customer management where required.
- Chat management.
- Reports.
- Administrative settings.

Administrative access must require an authenticated administrator account.

RESOURCE OWNERSHIP

Authorization must also verify ownership.

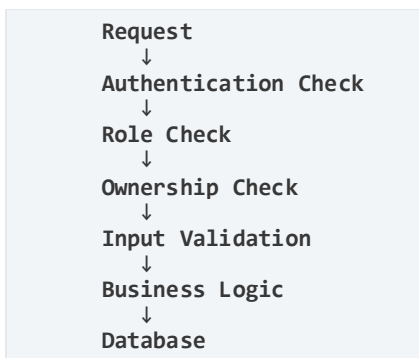
For example, if Buyer A requests GET /api/orders/order-B and order-B belongs to Buyer B, the backend must reject the request.

The same principle applies to Orders, BOM projects, Conversations, Profile information, and other user-owned resources.

The backend must verify ownership rather than trusting an ID supplied by the frontend.

API SECURITY

Every protected API endpoint must perform the appropriate security checks.



An API must not assume the frontend has already validated a request.

INPUT VALIDATION

All user-controlled input must be validated.

Examples include:

- Product information.
- Search terms.
- Quantities.
- Addresses.
- Account information.
- Chat messages.
- BOM names.
- Order information.

Validation should occur on the server even when the frontend already performs validation.

INPUT SANITIZATION

User-provided content should be handled safely before being displayed or processed.

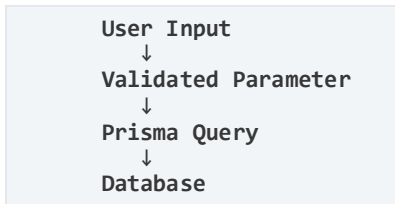
Particular attention should be given to Chat messages, Product descriptions, User profile information, Search input, and Form fields.

The application should prevent user-controlled content from being interpreted as executable code.

16.15 SQL INJECTION PROTECTION

OhmSim must prevent SQL injection attacks.

Database operations should use Prisma's parameterized query mechanisms rather than constructing SQL queries from raw user input.



Raw SQL should only be used when necessary and must follow secure parameterization practices.

CROSS-SITE SCRIPTING PROTECTION

The application must prevent malicious scripts from being executed through user-controlled content.

Potential sources include Product descriptions, Chat messages, User profile fields, and Search parameters.

The frontend should avoid unsafe HTML rendering unless the content has been explicitly sanitized and the implementation requires HTML.

CROSS-SITE REQUEST FORGERY PROTECTION

Authentication and state-changing operations should use appropriate CSRF protections based on the authentication architecture.

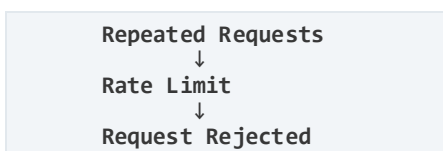
The team must evaluate Cookie-based authentication, SameSite settings, CSRF tokens where required, and Origin checks where appropriate.

The final implementation should match the selected authentication framework.

RATE LIMITING

Rate limiting should protect sensitive endpoints from excessive requests.

Priority endpoints include Login, Registration, Password reset, Authentication-related APIs, Chat message submission, and other operations susceptible to abuse.



Rate limiting values should be determined during implementation and testing.

DATABASE SECURITY

The production database must use controlled access.

Database credentials should remain private, exist only in secure environment configuration, never appear in source code or committed logs, and not be shared with unauthorized team members.

Application users should access database functionality through the backend rather than connecting directly from the browser.

DATABASE INTEGRITY

Database constraints should enforce important system rules.

Examples include Required fields, Unique identifiers, Foreign-key relationships, Valid relationships, Appropriate indexes, and Valid status values.

Business rules should not rely exclusively on frontend validation.

TRANSACTION SECURITY

Operations involving multiple related database changes should use appropriate transactions. Checkout represents a major example.

```
graph TD; A[Validate Stock] --> B[Create Order]; B --> C[Create Order Items]; C --> D[Update Inventory]; D --> E[Complete Transaction];
```

A vertical flowchart showing the steps of a checkout transaction: Validate Stock, Create Order, Create Order Items, Update Inventory, and Complete Transaction, connected by downward arrows.

If a critical operation fails, the system should avoid leaving partially completed transaction data.

INVENTORY SECURITY

Inventory operations must be restricted to authorized administrators.

```
graph TD; A[Buyer] --> B["POST /api/inventory/update"]; B --> C[Authorization]; C --> D["403 Forbidden"];
```

A vertical flowchart showing an unauthorized inventory update attempt: Buyer, POST /api/inventory/update, Authorization, and 403 Forbidden, connected by downward arrows.

The backend must enforce this restriction regardless of the frontend interface.

ORDER SECURITY

Orders contain user and transaction information and require ownership controls.

Buyers should only access their own orders. Administrators should access orders according to their assigned permissions. Order status changes should also be restricted.

```
graph TD; A[Buyer] --> B[Change Order Status]; B --> C[Authorization Check]; C --> D[Rejected];
```

A vertical flowchart showing an unauthorized order status change attempt: Buyer, Change Order Status, Authorization Check, and Rejected, connected by downward arrows.

Order status changes should occur through authorized administrative functionality.

CHAT SECURITY

Chat data must remain accessible only to participants and authorized administrators.

The backend should verify Conversation membership, Sender identity, Recipient/conversation relationship, and Message ownership where applicable.

A user must not gain access to another user's conversation by modifying a conversation ID.

PUSH NOTIFICATION SECURITY

Push notification tokens should be treated as application data and stored securely.

The backend should associate tokens with the appropriate user or device.

Notification content should avoid unnecessarily exposing sensitive information.

Preferred: Your OhmSim order has been updated. Avoid exposing sensitive account information directly inside a notification.

FIREBASE SECURITY

Firebase configuration must distinguish between public client configuration and private server credentials.

Private Firebase credentials must never be committed to GitHub or exposed through frontend source code.

The backend should use the appropriate secure Firebase credentials when sending server-generated push notifications.

IMAGE UPLOAD SECURITY

If administrators can upload product images, the upload process should validate File type, File size, File extension, Content type, and Storage destination.

The application should reject unsupported file formats. Uploaded files should not be treated as executable application code.

FILE NAMING

Uploaded files should use controlled identifiers rather than relying entirely on user-provided filenames.

```
product-8f31a2.webp
```

```
instead of:
```

```
my-product-final-final-version.php
```

The application should generate safe storage names where the storage provider does not already provide this behavior.

ENVIRONMENT SECRET MANAGEMENT

The following information must remain outside the Git repository:

```
Database passwords
Authentication secrets
Firebase private keys
API secrets
Storage credentials
Deployment credentials
```

The repository should contain .env.example but never production values.

GITHUB SECURITY

The development team should maintain repository security through:

- Protected production branch.
- Pull Request review.
- Restricted production access.
- Secret scanning where available.
- .gitignore configuration.
- Removal of accidental secrets.
- Controlled repository permissions.

If a secret is accidentally committed, deleting the file from the latest commit is not sufficient. The affected credential must be considered compromised and replaced.

DEPENDENCY SECURITY

Third-party packages introduce potential security risks.

The development team should:

- Keep dependencies reasonably updated.
- Review security alerts.
- Remove unused packages.
- Avoid unnecessary dependencies.
- Investigate vulnerable packages before deployment.

Dependency updates should be tested before merging into the production branch.

ERROR MESSAGE SECURITY

Error messages should provide useful information without exposing internal system details.

Avoid exposing Database connection strings, File paths, Stack traces, Authentication secrets, and Internal API details.

Unable to complete the request.

Please try again.

Detailed technical information should remain available through controlled development or server logs.

LOGGING

Logs should help developers diagnose problems without exposing sensitive information.

Logs should not contain Passwords, Authentication tokens, Private keys, Full payment credentials, or Unnecessary personal information.

Important system events include Authentication failures, Authorization failures, Critical API errors, Database errors, and Production deployment failures.

DATA PRIVACY

OhmSim should collect only information required for its functionality.

Potential user information includes Account information, Contact information, Order information, Chat messages, and Device notification information.

The application should define why each category of information is collected and where it is stored.

DATA ACCESS PRINCIPLES

The system should follow the principle of least privilege. Each user and service should receive only the access required for its function.



Production credentials should remain restricted.

SECURE COMMUNICATION

Production communication should use HTTPS.

The application should avoid transmitting sensitive information through unencrypted connections.

The production environment should use secure connections between Browser ↔ Application, Application ↔ Database, and Application ↔ External Services.

SECURITY TESTING

Security testing should occur during QA.

Tests should include:

- Unauthorized API access.
- Unauthorized page access.
- Incorrect role access.
- Resource ownership violations.
- Invalid input.
- SQL injection attempts.
- XSS attempts.
- Excessive login attempts.
- Invalid session behavior.
- Expired authentication.
- Malicious file uploads.

SECURITY TEST EXAMPLE

Test ID: SEC-AUTH-001

Scenario: Buyer attempts to access an administrator endpoint.

Steps: 1. Log in as Buyer. 2. Obtain authenticated session. 3. Send request to an administrator-only API endpoint. 4. Observe response.

Expected Result: The server rejects the request.

HTTP 403 Forbidden

The frontend must not be the only mechanism preventing access.

SECURITY TEST EXAMPLE – OWNERSHIP

Test ID: SEC-ORDER-001

Scenario: Buyer attempts to access another Buyer's order.

Steps: 1. Log in as Buyer A. 2. Obtain an order identifier belonging to Buyer B. 3. Request the order through the API.

Expected Result: The server rejects the request and does not expose Buyer B's order information.

SECURITY TEST EXAMPLE – INVENTORY

Test ID: SEC-INV-001

Scenario: Buyer attempts to modify inventory.

Steps: 1. Log in as Buyer. 2. Send an inventory modification request. 3. Observe response.

Expected Result: The request is rejected by backend authorization.

SECURITY INCIDENT RESPONSE

```
graph TD; Detect --> Contain; Contain --> Investigate; Investigate --> RotateCredentials[Rotate Credentials]; RotateCredentials --> FixVulnerability[Fix Vulnerability]; FixVulnerability --> Test; Test --> Deploy; Deploy --> Monitor;
```

Examples of incidents include Exposed credentials, Unauthorized account access, Database exposure, Malicious file upload, and Unauthorized administrative access.

The Project Manager should coordinate the response with the relevant developer.

CREDENTIAL ROTATION

Credentials should be replaced when:

- A secret is accidentally exposed.
- A team member with access leaves the project.
- A production credential is suspected to be compromised.
- An external service reports a security issue.

Rotating a credential should include updating the relevant deployment environment and verifying dependent services.

SECURITY RESPONSIBILITIES

Security Area	Responsible Role
Authentication implementation	Backend Developer
Authorization	Backend Developer
Frontend security behavior	Frontend Developers

Security Area	Responsible Role
UI security states	UI/UX Designers
Security testing	QA Manager
Repository access	Project Manager
Production credentials	Authorized Project/Development Personnel
Data requirements	Data Gatherer + Project Manager
Final security review	Project Manager + QA Manager

Security remains a shared responsibility across the team.

SECURITY CHECKLIST

AUTHENTICATION

- ☐ Passwords are securely hashed.
- ☐ Login works correctly.
- ☐ Logout invalidates the session.
- ☐ Protected routes require authentication.
- ☐ Authentication errors do not expose sensitive information.

AUTHORIZATION

- ☐ Buyer permissions work.
- ☐ Administrator permissions work.
- ☐ API authorization works.
- ☐ Resource ownership checks work.

DATABASE

- ☐ Production credentials are protected.
- ☐ Database constraints are configured.
- ☐ Transactions are used where required.
- ☐ No direct database access exists from the browser.

API

- ☐ Inputs are validated.
- ☐ Unauthorized requests are rejected.
- ☐ Invalid resources return appropriate errors.
- ☐ Rate limiting is applied where required.

REPOSITORY

- ☐ .env files are ignored.
- ☐ No secrets exist in Git history.
- ☐ Production branch is protected.
- ☐ Repository permissions are controlled.

EXTERNAL SERVICES

- ☐ Firebase credentials are protected.
- ☐ Image-storage credentials are protected.
- ☐ API keys are stored securely.

TESTING

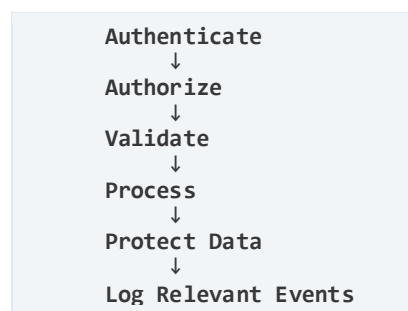
- [] Authentication tests passed.
- [] Authorization tests passed.
- [] Ownership tests passed.
- [] Input-validation tests passed.
- [] Security regression tests passed.

CHAPTER SUMMARY

This chapter establishes the security and data-protection requirements for OhmSim.

The system must enforce security on the backend, not only through the frontend. Authentication protects accounts, authorization controls access, ownership checks protect user-specific resources, validation protects APIs, and secure environment configuration protects system credentials.

The core security model is:



Security testing must occur before production release and after major changes affecting authentication, authorization, APIs, databases, or external services.

CHAPTER 17 – MAINTENANCE, MONITORING & PROJECT HANDOVER

OVERVIEW

This chapter defines how OhmSim will be maintained after initial development and deployment.

Deployment does not mark the end of the project. The system requires continuous monitoring, bug fixing, dependency updates, database maintenance, documentation updates, and controlled feature changes.

This chapter also defines the process for transferring project knowledge between team members so the project remains maintainable even when the original developers are no longer actively working on a particular feature.

MAINTENANCE OBJECTIVES

The maintenance process aims to:

- Keep OhmSim operational after deployment.
- Detect and resolve production problems.
- Protect application and database integrity.
- Keep dependencies reasonably updated.
- Maintain compatibility with external services.
- Preserve project documentation.
- Prevent undocumented changes.
- Maintain code quality.
- Support future feature development.
- Ensure another developer can understand and maintain the system.

TYPES OF MAINTENANCE

OhmSim maintenance will follow four main categories.

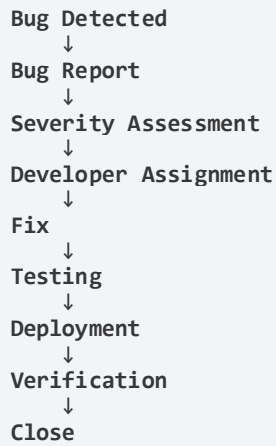
Type	Purpose
Corrective	Fix existing defects
Adaptive	Adjust the system to environmental changes
Perfective	Improve existing functionality
Preventive	Reduce the likelihood of future problems

17.4 CORRECTIVE MAINTENANCE

Corrective maintenance addresses problems discovered after deployment.

Examples:

- Login failure.
- Incorrect order calculation.
- Broken product filtering.
- Notification failure.
- Chat malfunction.
- Inventory synchronization issue.
- UI rendering problem.



Critical production problems receive higher priority than cosmetic issues.

ADAPTIVE MAINTENANCE

Adaptive maintenance modifies OhmSim when its environment changes.

Examples include Next.js updates, Node.js version changes, database platform changes, Firebase API changes, browser compatibility changes, hosting configuration changes, and external API changes.

Before making an adaptive change, the team should evaluate its effect on existing functionality.

17.6 PERFECTIVE MAINTENANCE

Perfective maintenance improves existing functionality without changing the application's fundamental purpose.

Examples:

- Improving product search.
- Improving checkout usability.
- Reducing page loading time.
- Improving mobile layouts.
- Improving chat usability.
- Improving administrator dashboards.

These changes should still follow the project's normal development workflow.

PREVENTIVE MAINTENANCE

Preventive maintenance reduces future problems.

Examples:

- Refactoring duplicated code.
- Removing unused dependencies.
- Updating vulnerable packages.
- Improving database indexes.
- Cleaning obsolete files.
- Improving error handling.
- Updating documentation.
- Reviewing unused API endpoints.

Preventive maintenance should not bypass code review or QA.

MONITORING STRATEGY

The team should monitor the production application for operational problems.

Important areas include:

- Deployment status.
- Application errors.
- API failures.
- Database errors.
- Authentication failures.
- Notification failures.
- Performance degradation.
- Unexpected user behavior.

Monitoring should focus on issues capable of affecting actual users or system integrity.

APPLICATION MONITORING

The team should monitor important application events.



Authentication Failure
API Error
Database Error
Order Processing Failure
Notification Failure
Unexpected Server Error

Repeated errors should be investigated rather than ignored.

DATABASE MONITORING

The production database should be monitored for:

- Connection failures.
- Query failures.
- Storage usage.
- Slow queries.
- Migration problems.
- Unexpected data growth.

Database monitoring becomes increasingly important as product, order, user, and chat data increases.

PERFORMANCE MONITORING

The team should periodically evaluate:

- Page loading speed.
- API response time.
- Database query performance.
- Image loading.
- Search performance.
- Product catalog performance.
- Chat loading.
- Administrator dashboard performance.

Performance problems should be investigated using measurable data rather than assumptions.

BACKUP STRATEGY

Production data should have an appropriate backup and recovery strategy.

Important data includes User accounts, Product records, Inventory, Orders, BOM records, Conversations, and Notification-related records.

The team should verify whether the selected database provider's backup and recovery capabilities meet the project's requirements. Backups are useful only when the team knows how to restore them.

BACKUP VERIFICATION

A backup process should be periodically verified.

- 1. Backup exists.
- 2. Backup contains expected data.
- 3. Backup is accessible to authorized personnel.
- 4. Restoration procedures work.
- 5. Restored data remains consistent.

A backup that has never been tested should not be treated as a verified recovery solution.

DATABASE MAINTENANCE

Database maintenance includes:

- Reviewing indexes.
- Monitoring database size.
- Checking slow queries.
- Removing unnecessary test data.
- Reviewing relationships.
- Maintaining migrations.
- Checking data integrity.

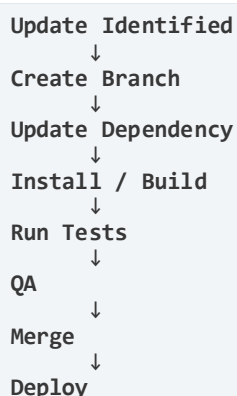
Production data should not be deleted simply to reduce database size without understanding its effect on the application.

DEPENDENCY MAINTENANCE

OhmSim depends on external libraries and frameworks, including Next.js, React, TypeScript, Prisma, authentication libraries, Firebase packages, UI libraries, and utility packages.

Dependencies should be reviewed periodically. The team should prioritize updates involving security vulnerabilities, critical bugs, compatibility issues, and required framework changes.

17.16 DEPENDENCY UPDATE WORKFLOW



The team should avoid updating large numbers of unrelated dependencies simultaneously unless there is a clear reason.

PRODUCTION ISSUE CLASSIFICATION

Production issues should be classified by severity.

Priority	Description	Example
P0	System-critical	Application unavailable
P1	Major functionality affected	Checkout completely unavailable
P2	Significant issue	Chat fails for some users
P3	Minor issue	Small UI defect

The team can adjust these categories based on the project's actual operational requirements.

PRODUCTION INCIDENT WORKFLOW



The team should document major incidents so similar problems become easier to diagnose in the future.

INCIDENT DOCUMENTATION

A production incident record should contain Incident ID, Date and time, Affected feature, Severity, Description, Root cause, Resolution, Affected users, and Preventive action.

Incident ID: INC-001

Feature:
Checkout

Severity:
P1

Problem:
Users could not complete checkout.

Cause:
Production database connection configuration was incorrect.

Resolution:
Corrected production environment configuration and redeployed.

Preventive Action:
Added production configuration verification to the release checklist.

CHANGE MANAGEMENT

Changes to OhmSim should follow a controlled process.

A significant change should include Change description, Reason for change, Affected features, Developer responsible, Testing requirements, Potential risks, Approval, and Deployment plan.

```
Change:
Add product stock reservation.

Affected:
Cart
Checkout
Inventory
Orders

Testing:
Unit
Integration
API
Regression
```

FEATURE REQUESTS

New features should enter the project backlog rather than being implemented immediately.

```
Feature Request
↓
Requirement Review
↓
Priority Assignment
↓
Task Creation
↓
Design
↓
Development
↓
Testing
↓
Release
```

This prevents unplanned changes from disrupting existing development.

TECHNICAL DEBT

Technical debt refers to shortcuts or temporary implementation decisions that increase future maintenance effort.

Examples:

- Duplicated logic.
- Poorly structured components.
- Temporary mock implementations.
- Missing tests.
- Unclear API contracts.
- Hard-coded configuration.
- Unused dependencies.

Technical Debt:
Replace temporary mock notification service with
production implementation.

Priority:
Medium

Owner:
Backend Developer

Technical debt should be recorded rather than forgotten.

CODE QUALITY MAINTENANCE

The team should periodically review:

- Folder structure.
- Component organization.
- API organization.
- Naming conventions.
- Duplicate code.
- Unused code.
- TypeScript types.
- Database access patterns.

Refactoring should preserve existing behavior unless the change explicitly modifies functionality.

DOCUMENTATION MAINTENANCE

Documentation should remain synchronized with the application.

Documents that require updates include the Product Design & Development Handbook, API documentation, Database documentation, Environment setup guide, Deployment guide, Testing documentation, User documentation, and Troubleshooting guide.

A feature should not remain undocumented when its implementation significantly changes system behavior.

API DOCUMENTATION MAINTENANCE

Every production API should have documentation describing Endpoint, HTTP method, Authentication requirements, Request body, Parameters, Response, Error responses, and Example request.

POST /api/orders	Cr	Bu
	ea	yer
	te	
	or	
	de	
	r	
	(Pi	
	ck	
	up	
	/	
	Re	
	gio	
	n 3	
	De	
	liv	
	ery	
	wit	
	h	
	C	
	O	
	D)	

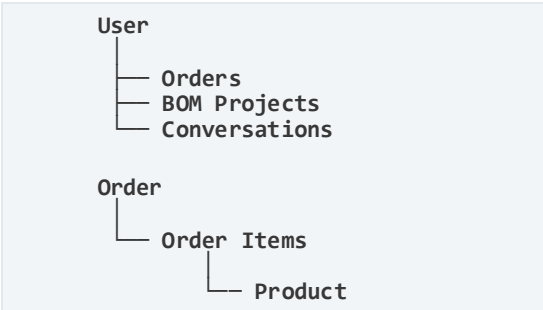
GET /api/user/addresses	Retriever Retrieve buyer saved delivery addresses (Max 2)	Buyer
POST /api/user/addresses	Address New Registration 3 delivery addresses (Max 2)	Buyer Authentication: Required Purpose: Creates an order from the user's current cart. Request: { ... } Response:

	{
	...
	}

The Bruno collection should also remain synchronized with the documented API.

DATABASE DOCUMENTATION

The team should maintain documentation covering Main tables, Important fields, Relationships, Enumerations, Important constraints, and Migration history.



The database documentation should reflect the actual Prisma schema.

TROUBLESHOOTING DOCUMENTATION

Common problems should receive documented solutions.

Problem:
 Application cannot connect to database.

Checks:
 1. Verify DATABASE_URL.
 2. Check database availability.
 3. Verify environment configuration.
 4. Test database connection.

Problem:
 Push notifications are not received.

Checks:
 1. Verify notification permission.
 2. Verify device token.
 3. Check Firebase configuration.
 4. Check backend notification request.
 5. Check Firebase logs.

PROJECT HANDOVER

A project handover occurs when responsibility moves from one developer or team member to another.

The handover should include Source code, GitHub repository, Environment setup instructions, Architecture documentation, Database documentation, API documentation, Deployment instructions, Testing documentation, Known issues, Technical debt, External service configuration, and Current project status.

DEVELOPER HANDOVER PROCESS



The incoming developer should successfully run and understand the assigned feature before the handover is considered complete.

KNOWLEDGE TRANSFER

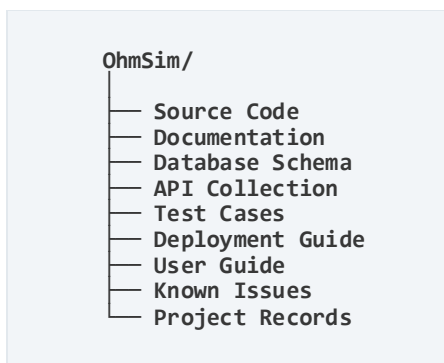
Knowledge should not remain exclusively with one person.

For important features, at least one additional team member should understand Purpose, Architecture, Main files, Database relationships, API endpoints, Common errors, and Testing process.

This reduces dependency on a single developer.

FINAL PROJECT HANDOVER

At the end of the academic project, the team should prepare a complete project package.



The final repository should contain the project documentation required for future maintenance.

ACCESS HANDOVER

Before project completion, verify access to GitHub repository, Vercel project, Neon database, Firebase project, Image-storage service, Domain management, and Project documentation.

Access should be transferred only to authorized project personnel. The team should not rely on a single person's personal account for critical project infrastructure.

PROJECT CLOSURE CHECKLIST

SOURCE CODE

- ☐ Production code is available.

TESTING

- ☐ Test cases are preserved.
- ☐ Final QA results are documented.
- ☐ Known issues are listed.

- ☐ Repository is organized.
- ☐ Main branch contains the final release.
- ☐ Documentation exists.

DATABASE

- ☐ Production schema is documented.
- ☐ Migration history is preserved.
- ☐ Backup/recovery process is documented.

API

- ☐ API endpoints are documented.
- ☐ Bruno collection is available.
- ☐ Authentication requirements are documented.

DEPLOYMENT

- ☐ Production deployment is operational.
- ☐ Environment configuration is documented.
- ☐ Deployment process is documented.

LONG-TERM MAINTENANCE WORKFLOW



The team should avoid modifying production code directly.

MAINTENANCE RESPONSIBILITIES

Responsibility	Primary Role
Issue coordination	Project Manager
Frontend maintenance	Frontend Developers
Backend maintenance	Backend Developer
Database maintenance	Backend Developer
QA verification	QA Manager
Documentation	Project Manager + Assigned Developer
UI improvements	UI Designer
UX improvements	UX Designer
Data maintenance	Data Gatherer

Responsibility	Primary Role
Deployment	Authorized Developer / Project Manager

Responsibilities can change after the academic project depending on who continues maintaining OhmSim.

FINAL MAINTENANCE PRINCIPLES

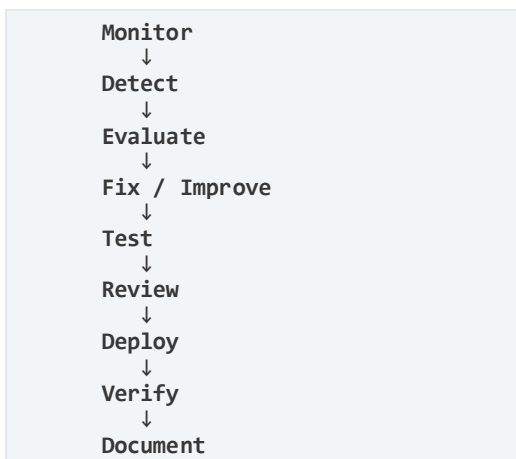
The OhmSim team should follow these principles:

- 1. Do not modify production directly.
- 2. Document significant changes.
- 3. Test before deployment.
- 4. Keep documentation synchronized with the system.
- 5. Protect production credentials.
- 6. Maintain backups and verify recovery procedures.
- 7. Track technical debt.
- 8. Keep dependencies maintained.
- 9. Share technical knowledge.
- 10. Keep the repository as the authoritative source for project code and documentation.

CHAPTER SUMMARY

This chapter establishes the post-development maintenance and project handover process for OhmSim.

The system should follow a controlled lifecycle after deployment:



The project handover process ensures the source code, infrastructure, database, APIs, testing materials, deployment procedures, documentation, and technical knowledge remain accessible to the next developer or project team.

With this process, OhmSim remains maintainable beyond its initial development period rather than depending on the original developers to understand undocumented implementation details.

CHAPTER 18 – USER MANUAL & SYSTEM OPERATIONS

OVERVIEW

This chapter defines how users interact with OhmSim after deployment.

It focuses on the operational use of the system rather than its internal implementation.

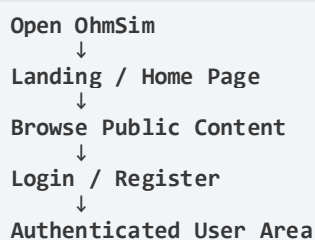
OhmSim has two primary user roles: Buyer and Administrator.

The Buyer uses OhmSim to browse products, manage electronic-component projects through BOMs, purchase products, monitor orders, communicate through chat, and receive notifications.

The Administrator manages products, inventory, orders, customer-related operations, and other authorized administrative functions.

SYSTEM ACCESS

Users access OhmSim through the deployed web application.



Some product information can remain publicly accessible, while account-specific functionality requires authentication.

USER ROLES

Function	Buyer	Administrator
Browse products	✓	✓
Search products	✓	✓
View product details	✓	✓
Manage cart	✓	—
Manage BOM	✓	—
Checkout	✓	—
View own orders	✓	✓
Manage products	—	✓
Manage inventory	—	✓
Manage orders	—	✓
Customer communication	✓	✓
Administrative dashboard	—	✓

The exact permissions should follow the authorization rules implemented in the system.

BUYER WORKFLOW

Register / Login
↓
Browse Products
↓
View Product
↓
Add to Cart / BOM
↓
Review Items
↓
Checkout
↓
Order Created
↓
Monitor Order
↓
Receive Notifications
↓
Complete Order

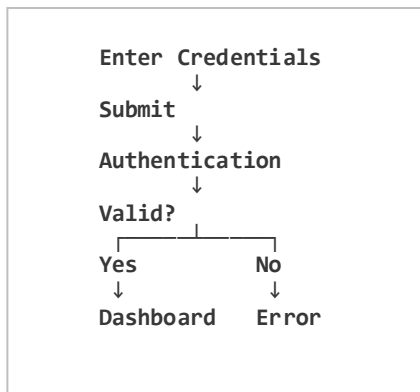
BUYER REGISTRATION

A new Buyer begins by creating an account.

Registration Page
↓
Enter Required Information
↓
Submit
↓
Validation
↓
Account Creation
↓
Login

The registration form should reject missing required information, invalid input, invalid email format, duplicate account information, and passwords that do not satisfy the defined requirements.

BUYER LOGIN



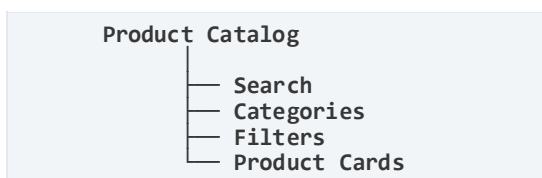
A failed login should display an appropriate error without exposing sensitive authentication information.

BUYER DASHBOARD

After authentication, the Buyer should have access to account information, orders, cart, BOM projects, conversations, and notifications. The exact dashboard layout follows the approved UI design.

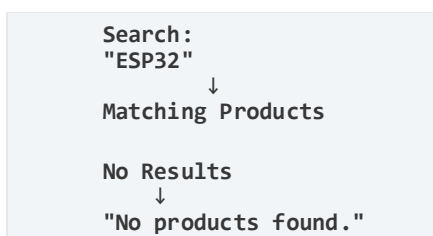
PRODUCT CATALOG

The Product Catalog allows Buyers to browse available products. A product listing can contain Product image, Product name, Category, Price, Availability, Relevant specifications, and Add-to-cart functionality.



PRODUCT SEARCH

The Buyer can search for products using relevant keywords.



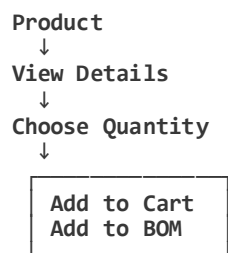
The system should not display unrelated results as though they matched the query.

PRODUCT FILTERING

Products can be filtered according to Category, Availability, Price range, Component type, and other product-specific attributes. The final filter set should follow the approved system requirements.

PRODUCT DETAILS

Selecting a product opens its detailed information page containing Product name, Images, Description, Specifications, Price, Stock availability, Quantity selector, Add to Cart, and Add to BOM.



PRODUCT AVAILABILITY

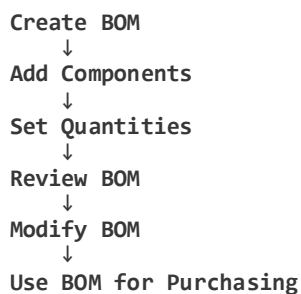
The system should clearly indicate whether a product is available.

In Stock
Low Stock
Out of Stock

A Buyer should not be allowed to complete a purchase exceeding available inventory.

BOM MANAGEMENT

BOM stands for Bill of Materials. The BOM feature allows Buyers to organize electronic components required for a project.



CREATING A BOM

The Buyer creates a new BOM project by providing the required project information.

BOM Name:
Arduino Line-Following Robot
↓
BOM Created

The Buyer can then add products to the BOM.

ADDING PRODUCTS TO A BOM

```
Product Details
↓
Add to BOM
↓
Select BOM
↓
Specify Quantity
↓
Save
```

The system should update the BOM immediately after a successful operation.

MANAGING BOM QUANTITIES

Buyers should be able to modify quantities of components in their BOM.

```
ESP32
Quantity: 1

Resistor
Quantity: 10

LED
Quantity: 8
```

Quantity changes should be validated against the system's defined rules.

BOM TO CART

```
BOM
↓
Review Components
↓
Check Availability
↓
Add Selected Items to Cart
↓
Review Cart
↓
Checkout
```

The system should indicate components that are unavailable or have insufficient stock.

SHOPPING CART

The Shopping Cart contains products the Buyer intends to purchase. A cart item includes Product, Quantity, Unit price, and Subtotal.

```
ESP32
₱250 × 2
= ₱500

LED
₱10 × 10
= ₱100
```

The cart should display the applicable total.

CART OPERATIONS

The Buyer should be able to add products, increase or decrease quantity, remove products, review totals, and proceed to checkout. The system should validate inventory whenever required.

CHECKOUT

```
Cart
↓
Checkout
↓
Review Information
↓
Confirm Order
↓
Validate Stock
↓
Create Order
↓
Update Inventory
↓
Order Confirmation
```

The system should prevent checkout when required information is invalid or inventory is insufficient.

ORDER CONFIRMATION

After successful checkout, the Buyer should receive the Order identifier, Ordered items, Quantities, Total amount, and Current order status.

```
Order #OHM-0001

Status:
Pending

Items:
ESP32 × 2
LED × 10
Resistor × 10
```

ORDER MANAGEMENT

The Buyer can view their own orders. The order list can display Order ID, Date, Total, Status, and Available actions.

```
Order #OHM-0001
Status: Preparing

Order #OHM-0002
Status: Completed
```

ORDER STATUS

```
Pending
↓
Confirmed
↓
Preparing
↓
Ready for Pickup
↓
Completed
```

An order can also be cancelled where the system's cancellation rules permit it.

ORDER TRACKING

Order #OHM-0001

- ✓ Order Placed
- ✓ Confirmed
- ✓ Preparing
- Ready for Pickup
- Completed

The exact visual implementation follows the UI design.

ORDER NOTIFICATIONS

The Buyer receives notifications when important order events occur.

```
Order Confirmed
↓
Notification
"Your order has been confirmed."

Ready for Pickup
↓
Push Notification
"Your order is ready for pickup."
```

The notification system uses the configured Firebase Cloud Messaging implementation.

NOTIFICATION CENTER

The Buyer should have access to an in-app notification center for Order updates, Chat notifications, and other system events. Notifications should support read/unread states.

Notifications

- Your order is ready.
- Order #OHM-0001 was confirmed.
- New message received.

BUYER-ADMINISTRATOR CHAT

OhmSim provides a normal messaging interface between Buyers and authorized administrators. Its purpose is to allow questions about products, orders, or other supported concerns.

```
Buyer
↓
Conversation
↕
Administrator
```

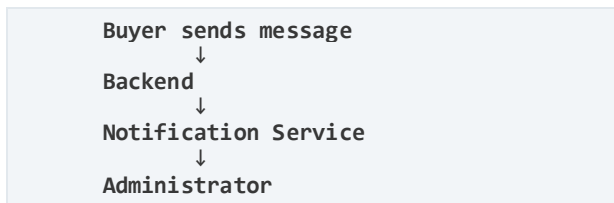
Messages should appear in chronological order.

SENDING A MESSAGE

```
Conversation
↓
Type Message
↓
Send
↓
Message Stored
↓
Administrator Receives Message
```

The system should prevent empty messages from being submitted.

CHAT NOTIFICATIONS

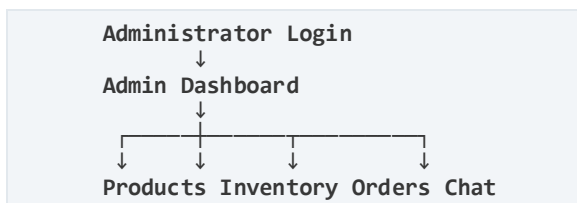


The same mechanism applies when an Administrator sends a message to a Buyer.

BUYER PROFILE

The Buyer can access account information such as Name, Email, Contact information, and other required account information. Users should only be allowed to modify fields permitted by the system.

ADMINISTRATOR WORKFLOW



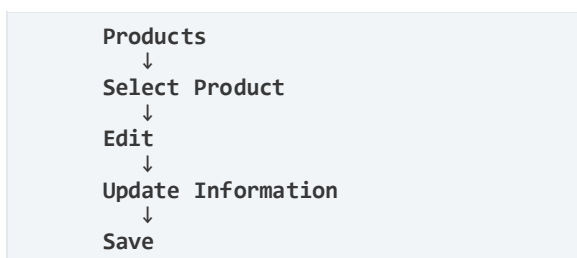
Additional administrative functions can be added according to the final system scope.

ADMINISTRATOR DASHBOARD

The dashboard provides an overview of Pending orders, Inventory status, Low-stock products, Recent orders, Recent conversations, and other approved statistics. It should prioritize information requiring administrative action.

PRODUCT MANAGEMENT

Administrators can create, view, edit, archive, and update products and upload product images.



CREATING A PRODUCT

Possible fields include Product name, Category, Description, Price, Stock, Specifications, and Product image. The system validates the information before creating the product.

EDITING A PRODUCT



Changes should be reflected in the Buyer-facing product catalog after successful completion.

PRODUCT ARCHIVING

```
graph TD
    A[Active Product] --> B[Archive]
    B --> C[Hidden from Normal Catalog]
```

Historical order records should remain consistent.

INVENTORY MANAGEMENT

Administrators manage inventory through View stock, Add stock, Adjust stock, Identify low-stock products, and Identify out-of-stock products.

```
ESP32
Current Stock: 12

Restock:
+20

New Stock:
32
```

Inventory changes must be validated by the backend.

LOW-STOCK MANAGEMENT

Product	Stock	Status
ESP32	12	Normal
OLED Display	3	Low Stock
Servo Motor	0	Out of Stock

The threshold should be configurable according to the final system requirements.

ADMINISTRATOR ORDER MANAGEMENT

```
graph TD
    A[Pending Order] --> B[Review]
    B --> C[Confirm]
    C --> D[Prepare]
    D --> E[Ready for Pickup]
    E --> F[Complete]
```

The Administrator should verify relevant order information before changing the status.

UPDATING ORDER STATUS

An Administrator selects an order and changes its status according to the allowed workflow. The system should validate permissions, validate the status transition, update the order, record the change where required, and trigger a notification when applicable.

ADMINISTRATOR CHAT

Administrators can access authorized Buyer conversations to view messages, send replies, review history, and respond to product or supported order-related concerns. Administrators should not access conversations outside their authorized scope.

ADMINISTRATOR NOTIFICATIONS

Administrators can receive notifications for New order, New Buyer message, Low-stock event, and other administrative alerts. The notification system should prevent unnecessary duplicate notifications.

OPERATIONAL ERROR HANDLING

Users may encounter Network Error, Server Error, Invalid Input, Out of Stock, Unauthorized Action, or Session Expired conditions.

Unable to load products.

[Retry]

The interface should display a clear message and provide an appropriate next action when available.

SESSION EXPIRATION

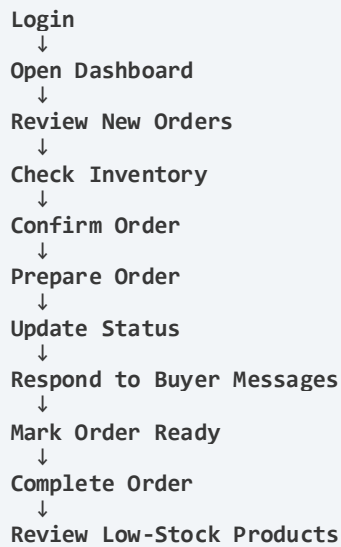
```
graph TD
    A[User Performs Action] --> B[Session Expired]
    B --> C[Authentication Required]
    C --> D[Login]
```

The application should not silently perform protected operations using an invalid session.

BUYER DAILY OPERATION EXAMPLE

```
graph TD
    A[Login] --> B[Search for ESP32]
    B --> C[Open Product]
    C --> D[Add to BOM]
    D --> E[Add Required Components]
    E --> F[Review BOM]
    F --> G[Move Items to Cart]
    G --> H[Checkout]
    H --> I[Receive Order Confirmation]
    I --> J[Administrator Processes Order]
    J --> K[Receive Status Notification]
    K --> L[Open Chat if Assistance Is Needed]
    L --> M[Receive Ready-for-Pickup Notification]
    M --> N[Complete Order]
```

ADMINISTRATOR DAILY OPERATION EXAMPLE



```
graph TD; A[Login] --> B[Open Dashboard]; B --> C[Review New Orders]; C --> D[Check Inventory]; D --> E[Confirm Order]; E --> F[Prepare Order]; F --> G[Update Status]; G --> H[Respond to Buyer Messages]; H --> I[Mark Order Ready]; I --> J[Complete Order]; J --> K[Review Low-Stock Products];
```

Login
↓
Open Dashboard
↓
Review New Orders
↓
Check Inventory
↓
Confirm Order
↓
Prepare Order
↓
Update Status
↓
Respond to Buyer Messages
↓
Mark Order Ready
↓
Complete Order
↓
Review Low-Stock Products

SYSTEM RULES FOR BUYERS

Buyers must use their own account, provide valid information, keep account credentials private, purchase only available quantities, access only their own protected information, and follow the defined ordering process. The system enforces these rules through validation and authorization.

SYSTEM RULES FOR ADMINISTRATORS

Administrators must use authorized accounts, manage only permitted resources, verify information before major changes, follow the defined order workflow, avoid exposing Buyer information unnecessarily, and maintain accurate inventory data.

USER SUPPORT WORKFLOW



```
graph TD; A[Problem] --> B[Check Error Message]; B --> C[Retry / Correct Input]; C --> D[If unresolved]; D --> E[Contact Administrator]; E --> F[Chat]; F --> G[Issue Resolution];
```

Problem
↓
Check Error Message
↓
Retry / Correct Input
↓
If unresolved
↓
Contact Administrator
↓
Chat
↓
Issue Resolution

The chat feature provides the primary in-system communication method for supported Buyer inquiries.

USER MANUAL CHECKLIST

BUYER

- ☐ Register account.
- ☐ Log in.
- ☐ Browse products.
- ☐ Search products.
- ☐ Filter products.
- ☐ View product details.
- ☐ Add products to BOM.
- ☐ Manage BOM quantities.
- ☐ Add products to cart.
- ☐ Modify cart.
- ☐ Checkout.
- ☐ View orders.
- ☐ Track order status.
- ☐ Receive notifications.
- ☐ Send chat messages.
- ☐ Manage profile.
- ☐ Log out.

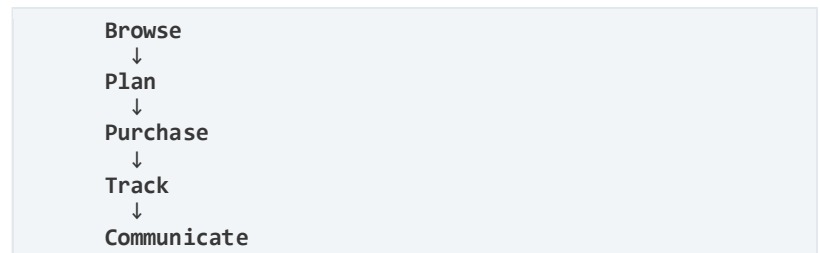
ADMINISTRATOR

- ☐ Log in.
- ☐ Access dashboard.
- ☐ Manage products.
- ☐ Manage inventory.
- ☐ Review orders.
- ☐ Update order status.
- ☐ Respond to Buyers.
- ☐ Review notifications.
- ☐ Manage authorized administrative functions.
- ☐ Log out.

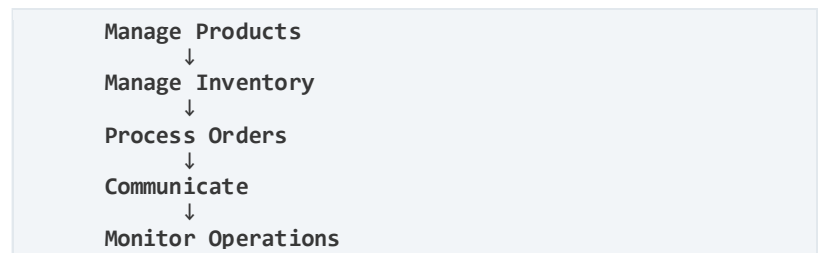
CHAPTER SUMMARY

This chapter defines the operational use of OhmSim for both Buyers and Administrators.

The Buyer workflow centers around:



The Administrator workflow centers around:



The system connects these workflows through the product catalog, BOM, shopping cart, checkout, order management, chat, inventory, and notification systems.

CHAPTER 19 – SYSTEM DOCUMENTATION & TECHNICAL REFERENCE

OVERVIEW

This chapter defines the technical documentation structure for OhmSim.

The purpose of this documentation is to make the project understandable, maintainable, testable, and transferable to another developer or team.

The technical reference should describe the actual implementation rather than planned functionality. It should remain synchronized with the source code, database schema, APIs, deployment configuration, and external services.

DOCUMENTATION OBJECTIVES

OhmSim technical documentation should allow a developer to answer:

- What does this system do?
- How is the system structured?
- Where is each feature implemented?
- How does the frontend communicate with the backend?
- How does the backend communicate with the database?
- What database tables exist?
- What APIs exist?
- What environment variables are required?
- How do I run the project locally?
- How do I test the system?
- How do I deploy the system?
- How do I troubleshoot common problems

DOCUMENTATION HIERARCHY



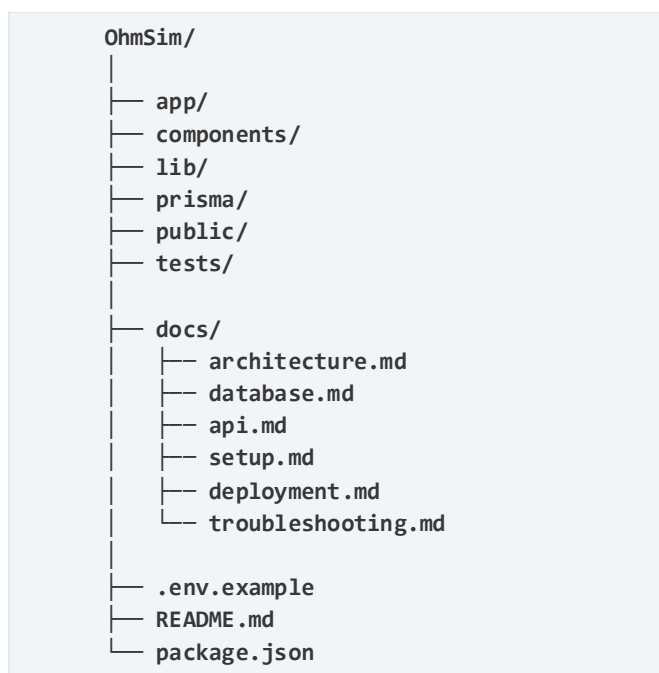
The complete Product Design & Development Handbook provides high-level project documentation, while technical documentation provides implementation-level details.

SOURCE OF TRUTH

Information	Source of Truth
Application behavior	Source code
Database structure	Prisma schema + migrations
API implementation	Backend source code
API testing	Bruno collection
Environment variables	.env.example + deployment configuration
UI implementation	Frontend source code
Deployment configuration	Hosting configuration
Project requirements	Approved project documentation

Documentation must not describe functionality differently from the actual implementation.

REPOSITORY DOCUMENTATION STRUCTURE



The actual directory structure should follow the project's implementation.

README

The root README.md should provide the fastest way for a new developer to understand and run OhmSim.

It should contain Project name, Short project description, Main technologies, Requirements, Installation instructions, Environment setup, Development commands, Testing commands, Build commands, Deployment reference, and Documentation links.

```

# OhmSim

## Overview

## Technology Stack

## Requirements

## Installation

## Environment Variables

## Development

## Testing

## Production Build

## Deployment

## Documentation

```

LOCAL DEVELOPMENT DOCUMENTATION

The setup documentation should allow a new developer to configure the project without relying on verbal instructions.

Required information includes Node.js version, package manager, repository setup, dependency installation, environment variables, database setup, Prisma setup, development server, testing, and production build.

```

Clone Repository
↓
Install Dependencies
↓
Configure .env.local
↓
Configure Database
↓
Run Prisma Setup
↓
Start Development Server
↓
Open OhmSim

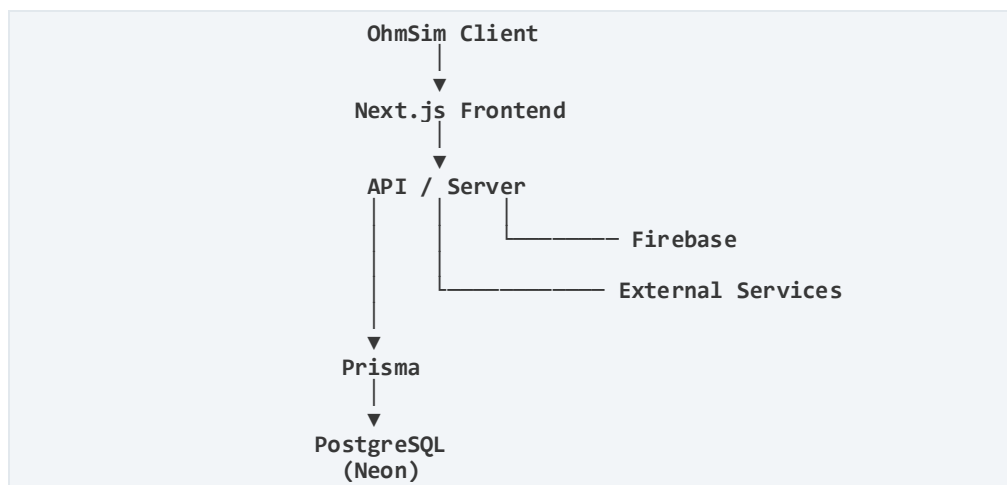
```

TECHNOLOGY STACK REFERENCE

Layer	Technology
Frontend	Next.js / React
Language	TypeScript
Backend	Next.js server-side/API implementation
Database	PostgreSQL
Database Hosting	Neon
ORM	Prisma
Notifications	Firebase Cloud Messaging
API Testing	Bruno
Repository	GitHub
Deployment	Vercel

The final document should identify the exact versions used by the project and update this table whenever the technology stack changes.

19.9 SYSTEM ARCHITECTURE REFERENCE



The architecture diagram should be updated when a major system component changes.

FRONTEND DOCUMENTATION

Frontend documentation should explain Application routes, Pages, Components, Layouts, Forms, State management, API communication, Authentication state, Loading states, Error states, and Responsive behavior.

FRONTEND ROUTE REFERENCE

Route	Purpose	Access
/	Home	Public
/products	Product catalog	Public
/products/[id]	Product details	Public
/login	Authentication	Public
/register	Registration	Public
/cart	Shopping cart	Buyer
/orders	Buyer orders	Buyer
/bom	BOM management	Buyer
/chat	Messaging	Authenticated
/admin	Admin dashboard	Administrator

The final route list must match the implemented application.

COMPONENT REFERENCE

```
components /
├── Navbar
├── ProductCard
├── ProductGrid
├── SearchBar
├── CartItem
├── OrderCard
├── BOMTable
├── ChatWindow
├── NotificationPanel
└── AdminTable
```

The documentation should describe the purpose of each major reusable component.

COMPONENT DOCUMENTATION FORMAT

```
Component:
ProductCard

Purpose:
Displays a product in the catalog.

Inputs:
Product information

Actions:
View product
Add to cart
Add to BOM

Dependencies:
Product data
Cart functionality
BOM functionality
```

The exact format can vary as long as the information remains clear.

BACKEND DOCUMENTATION

Backend documentation should describe API routes, Authentication, Authorization, Validation, Business logic, Database access, Error handling, and External service integration.

BACKEND REQUEST FLOW

```
Frontend
↓
HTTP Request
↓
API Route
↓
Authentication
↓
Authorization
↓
Validation
↓
Business Logic
↓
Prisma
↓
PostgreSQL
↓
Response
↓
Frontend
```

This structure should remain consistent across protected API operations.

API REFERENCE

Method	Endpoint	Purpose	Auth
GET	/api/products	List products	Public
GET	/api/products/[id]	Get product	Public
POST	/api/cart	Add cart item	Buyer
GET	/api/orders	Get own orders	Buyer
POST	/api/orders	Create order	Buyer
PATCH	/api/orders/[id]	Update order	Authorized
POST	/api/chat/messages	Send message	Authenticated

The final table must reflect the actual API implementation.

API ENDPOINT DOCUMENTATION

Each endpoint should document HTTP method, URL, Purpose, Authentication requirement, Authorization requirement, Request parameters, Request body, Validation rules, Success response, Error responses, Database operations, and External services used.

POST /api/orders	Create order (Pick up / Region 3 Delivery with COD)	Buyer
GET /api/user/addresses	Retrieve buyer saved	Buyer

	delivery addresses (Max 2)	
POST /api/user/addresses	Address new registration 3 delivery addresses (Max 2)	Buyer Purpose: Create a new order. Authentication: Required. Authorization: Buyer. Input: Cart items Required order in

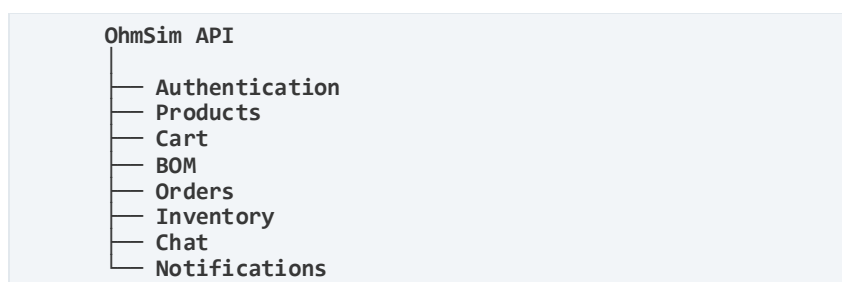
	fo rm at io n
	Pr oc es s:
	1. Au th en ti ca te us er .
	2. Va li da te re qu es t.
	3. Va li da te in ve nt or y.
	4. Cr ea te or de r.
	5. Cr ea te or de r it em s.
	6. Up da te in ve nt or y.
	7. Re tu rn or de r re su lt .

API ERROR REFERENCE

Status	Meaning
400	Invalid request
401	Authentication required
403	Insufficient permissions
404	Resource not found
409	Conflict
429	Too many requests
500	Server error

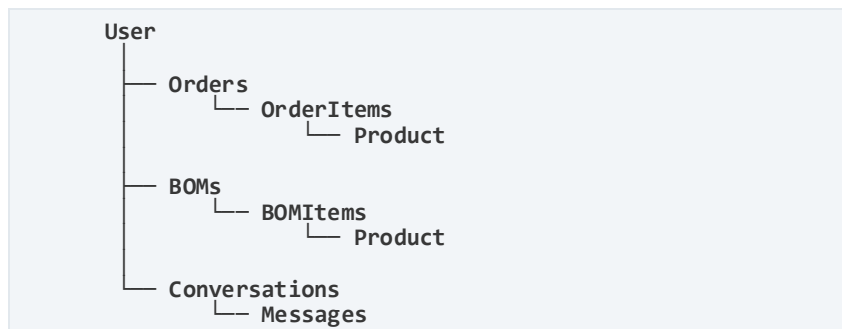
The actual API implementation should remain consistent with the documented behavior.

BRUNO API COLLECTION



Each request should use the correct environment and authentication configuration.

DATABASE REFERENCE



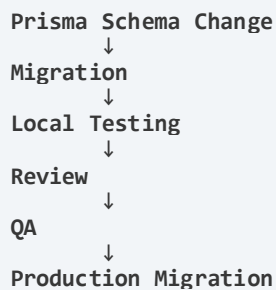
The final diagram must match the actual Prisma schema.

DATABASE TABLE DOCUMENTATION

Table: Product
Purpose: Stores products available through OhmSim.
Important Fields: - id - name - description - price - stock - categoryId - createdAt - updatedAt

Relationships should also be documented.

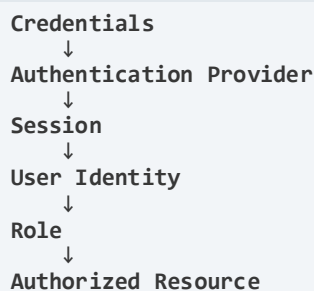
DATABASE MIGRATION REFERENCE



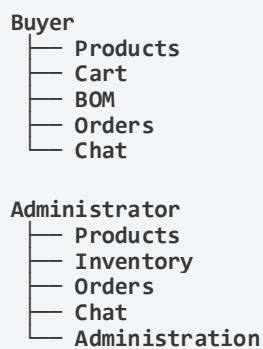
Every production schema change should have a corresponding migration.

AUTHENTICATION REFERENCE

The technical documentation should explain Registration, Login, Logout, Password hashing, Session management, Protected routes, Role verification, and Password reset if implemented.



AUTHORIZATION REFERENCE



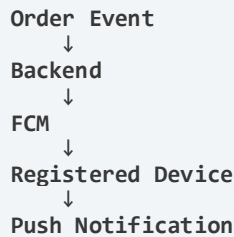
The backend must enforce these permissions.

EXTERNAL SERVICE REFERENCE

Service	Purpose
Vercel	Application hosting
Neon	PostgreSQL hosting
Firebase	Push notifications
GitHub	Source control
Bruno	API testing

FIREBASE REFERENCE

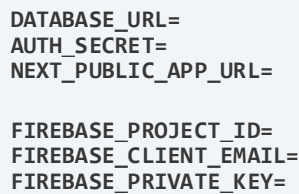
The Firebase documentation should identify Firebase project, FCM configuration, Notification flow, Required environment variables, Token handling, Server-side notification process, and Testing procedure.



```
graph TD; A[Order Event] --> B[Backend]; B --> C[FCM]; C --> D[Registered Device]; D --> E[Push Notification];
```

Private Firebase credentials must never appear in the documentation. Only variable names and configuration requirements should be documented.

ENVIRONMENT CONFIGURATION REFERENCE



```
DATABASE_URL=  
AUTH_SECRET=  
NEXT_PUBLIC_APP_URL=  
  
FIREBASE_PROJECT_ID=  
FIREBASE_CLIENT_EMAIL=  
FIREBASE_PRIVATE_KEY=
```

ENVIRONMENT SEPARATION



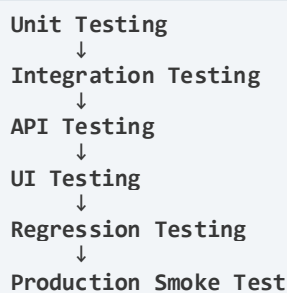
```
graph TD; A[Development] --> B[.env.local]; B --> C[Production]; C --> D[Vercel Environment Variables];
```

Production values must not be copied into public documentation or committed to GitHub.

DEPLOYMENT REFERENCE

The deployment documentation should describe Repository connection, Environment configuration, Build command, Database configuration, Migration process, Deployment process, Production verification, and Rollback procedure. The deployment process should reference Chapter 15 for the complete release workflow.

TESTING REFERENCE



```
graph TD; A[Unit Testing] --> B[Integration Testing]; B --> C[API Testing]; C --> D[UI Testing]; D --> E[Regression Testing]; E --> F[Production Smoke Test];
```

Each layer should have a defined purpose.

TEST CASE DOCUMENTATION

Important test cases should contain Test ID, Feature, Preconditions, Steps, Expected result, Actual result, and Status.

Test ID:
ORD-001

Feature:
Checkout

Precondition:
Buyer has products in cart.

Steps:
1. Open cart.
2. Select checkout.
3. Review order.
4. Confirm.

Expected:
Order is created and displayed in order history.

TROUBLESHOOTING REFERENCE

Problem:
Local application does not start.

Check:

1. Node.js version.
2. Dependencies.
3. Environment variables.
4. Database connection.
5. Terminal error output.

Problem:
Database query fails.

Check:

1. DATABASE_URL.
2. Database availability.
3. Prisma Client.
4. Migration state.
5. API logs.

CHANGE LOG

Version: 1.1.0

Added:

- Product filtering
- Improved BOM interface
- Notification center

Changed:

- Product catalog layout

Fixed:

- Cart quantity validation

The changelog should focus on changes relevant to the application.

ARCHITECTURE DECISION RECORDS

Decision:
Use Neon PostgreSQL for production database.

Context:
OhmSim requires relational storage for users, products, BOMs, orders, inventory, and messages.

Decision:
Use PostgreSQL hosted through Neon.

Reason:
Fits the relational data model and integrates with the selected application architecture.

ADRs prevent important technical decisions from becoming dependent on undocumented team conversations.

DOCUMENTATION VERSION CONTROL

Documentation Change
↓
Git Branch
↓
Pull Request
↓
Review
↓
Merge

This gives the team a history of documentation changes.

DOCUMENTATION OWNERSHIP

Documentation	Primary Responsibility
Project documentation	Project Manager
Architecture	Project Manager + Developers
Frontend reference	Frontend Developers
Backend reference	Backend Developer
Database reference	Backend Developer
API reference	Backend Developer
Testing documentation	QA Manager
UI documentation	UI Designer
UX documentation	UX Designer
Data documentation	Data Gatherer
Deployment documentation	Project Manager + Developer
User manual	Project Manager + Design Team

DOCUMENTATION REVIEW

Documentation should be reviewed whenever a change affects System architecture, API behavior, Database structure, User workflow, Authentication, Deployment, External services, or Major UI functionality.

The team should update documentation as part of the change rather than postponing it indefinitely.

DOCUMENTATION QUALITY CHECKLIST

PROJECT

- ☐ README is current.
- ☐ Architecture documentation is current.
- ☐ Technology stack is current.

FRONTEND

- ☐ Routes documented.
- ☐ Major components documented.
- ☐ Important workflows documented.

BACKEND

- ☐ API endpoints documented.
- ☐ Authentication documented.
- ☐ Authorization documented.
- ☐ Error responses documented.

DATABASE

- ☐ Schema documented.
- ☐ Relationships documented.
- ☐ Migrations preserved.

EXTERNAL Services

- ☐ Firebase documented.
- ☐ Hosting documented.
- ☐ Database hosting documented.
- ☐ Required configuration documented.

TESTING

- ☐ Test cases preserved.
- ☐ API collection updated.
- ☐ Regression tests documented.

DEPLOYMENT

- ☐ Environment configuration documented.
- ☐ Deployment procedure current.
- ☐ Rollback procedure current.

TECHNICAL REFERENCE EXAMPLE

```
README
↓
Setup Guide
↓
Architecture
↓
Frontend
↓
Backend
↓
Database
↓
API
↓
Testing
↓
Deployment
↓
Troubleshooting
```

This sequence provides progressively deeper technical information without requiring the developer to read the entire handbook before starting development.

FINAL DOCUMENTATION STRUCTURE

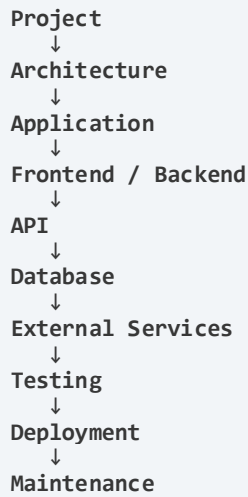
```
OhmSim/
├── README.md
├── docs/
│   ├── architecture.md
│   ├── setup.md
│   ├── frontend.md
│   ├── backend.md
│   ├── database.md
│   ├── api.md
│   ├── authentication.md
│   ├── testing.md
│   ├── deployment.md
│   ├── troubleshooting.md
│   └── changelog.md
├── prisma/
│   ├── schema.prisma
│   └── migrations/
├── tests/
├── .env.example
└── package.json
```

The exact structure should follow the actual repository.

CHAPTER SUMMARY

This chapter establishes the technical documentation standard for OhmSim.

The documentation must explain the project at multiple levels:



The documentation should remain synchronized with the implementation and stored under version control.

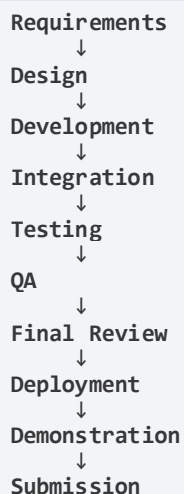
A developer who joins the project after its original development should be able to clone the repository, follow setup documentation, understand the architecture, run the application, test the APIs, understand the database, and deploy the system without depending on undocumented knowledge from the original team.

CHAPTER 20 – FINAL PROJECT STANDARDS AND COMPLETION CRITERIA

OVERVIEW

This chapter defines the final standards required before OhmSim is considered complete and ready for academic submission, demonstration, and production deployment.

The project is considered complete only when its required functionality, interface, backend, database, security, testing, documentation, and deployment requirements have been addressed.



DEFINITION OF DONE

A feature is considered Done only when:

- The feature matches the approved requirement.
- UI implementation is complete.
- Frontend logic works.
- Backend logic works where required.
- Database operations work where required.
- Validation is implemented.
- Authentication and authorization are implemented where required.
- Error and loading states are handled.
- The feature has been tested.
- QA has verified the feature.
- Related documentation has been updated.
- The feature does not introduce unresolved critical defects.

A feature should not be marked complete because its interface exists alone.

FEATURE COMPLETION STANDARD

```
graph TD; Requirement --> UI_Design[UI Design]; UI_Design --> Frontend_Implementation[Frontend Implementation]; Frontend_Implementation --> Backend_Implementation[Backend Implementation]; Backend_Implementation --> Database_Integration[Database Integration]; Database_Integration --> Testing; Testing --> QA_Approval[QA Approval]; QA_Approval --> Documentation; Documentation --> Done;
```

Features that do not require backend or database functionality should skip those stages.

PROJECT COMPLETION CRITERIA

Category	Requirement
Functionality	Core features work correctly
UI/UX	Interface follows approved designs
Backend	Required server functionality works
Database	Data is stored and retrieved correctly
Security	Authentication and authorization work
Testing	Required test cases pass
Performance	Application performs acceptably
Deployment	Production version works
Documentation	Required project documents are complete
Presentation	System is ready for demonstration

CORE FUNCTIONAL REQUIREMENTS

BUYER

- ☐ Registration
- ☐ Login
- ☐ Logout
- ☐ Product browsing
- ☐ Product searching
- ☐ Product filtering
- ☐ Product details
- ☐ BOM management
- ☐ Cart management
- ☐ Checkout
- ☐ Order viewing
- ☐ Order tracking
- ☐ Chat
- ☐ Notifications

- ☐ Profile management

ADMINISTRATOR

- ☐ Administrator login
- ☐ Dashboard
- ☐ Product management
- ☐ Inventory management
- ☐ Order management
- ☐ Order status updates
- ☐ Buyer communication
- ☐ Notifications
- ☐ Other approved administrative functions

The checklist should be adjusted if the approved project scope changes.

AUTHENTICATION COMPLETION CRITERIA

Authentication is considered complete when users can register and log in, invalid credentials are rejected, users can log out, protected pages require authentication, sessions behave correctly, passwords are securely stored, and authentication errors are handled appropriately.

AUTHORIZATION COMPLETION CRITERIA

Authorization is complete when Buyer and Administrator permissions work, unauthorized API requests are rejected, Buyers cannot access administrative functions, users cannot access another user's protected resources, and administrative operations require appropriate privileges. The backend must enforce authorization.

PRODUCT CATALOG COMPLETION CRITERIA

The product catalog is complete when products and information load correctly, images display, search and filtering work where implemented, product details work, stock information is accurate, and unavailable products are handled correctly.

BOM COMPLETION CRITERIA

The BOM feature is complete when Buyers can create BOMs, add and remove products, modify quantities, persist BOM information, reject invalid quantities, handle availability, and use the BOM purchasing workflow where implemented.

CART COMPLETION CRITERIA

The cart is complete when products can be added, quantities changed, products removed, totals calculated correctly, invalid quantities rejected, stock validated, and cart state persisted according to the intended design.

CHECKOUT COMPLETION CRITERIA

Checkout is complete when the Buyer reviews the cart, required information is validated, inventory is checked, the order and order items are created, inventory is updated correctly, confirmation is provided, the order appears in history, and notification behavior occurs. The system must avoid incomplete orders when a critical operation fails.

ORDER MANAGEMENT COMPLETION CRITERIA

Order management is complete when Buyers view their orders, Administrators view authorized orders, statuses follow the approved workflow, invalid transitions are prevented, restricted statuses cannot be modified by Buyers, changes are reflected correctly, and relevant notifications are triggered.

CHAT COMPLETION CRITERIA

Chat is complete when authorized users open conversations, send messages, messages are stored and displayed in the correct conversation, access is restricted, new messages are reflected correctly, and notification behavior works where implemented.

NOTIFICATION COMPLETION CRITERIA

Notifications are complete when permissions, device/browser registration, event triggers, FCM configuration and delivery, in-app notifications, and read/unread behavior work where implemented.

INVENTORY COMPLETION CRITERIA

Inventory is complete when Administrators view and modify stock, Buyers cannot modify it, checkout validates inventory, successful orders change inventory correctly, out-of-stock products are handled, and low-stock conditions work where implemented.

UI COMPLETION CRITERIA

The interface is complete when required pages exist, navigation works, forms and buttons work, loading/empty/error states exist where required, responsive layouts work, text is readable, components remain consistent, and the approved design system is followed.

UX COMPLETION CRITERIA

The user experience should provide clear navigation, understandable actions, consistent terminology, clear feedback, appropriate confirmation, clear errors, predictable workflows, and minimal unnecessary steps. The UX Designer should review major workflows.

BACKEND COMPLETION CRITERIA

The backend is complete when required endpoints exist, authentication and authorization are enforced, validation and business logic work, database operations work, errors are handled, responses follow the defined structure, critical operations use transactions, and sensitive information is not exposed.

DATABASE COMPLETION CRITERIA

The database is complete when tables, relationships, constraints, indexes, Prisma schema, migrations, production configuration, and data integrity are correct.

SECURITY COMPLETION CRITERIA

- ☐ Passwords are securely hashed.
- ☐ Authentication works.
- ☐ Authorization works.
- ☐ Resource ownership is enforced.
- ☐ API inputs are validated.
- ☐ Sensitive credentials are protected.
- ☐ Production secrets are not committed.

- [] Unauthorized requests are rejected.
- [] File uploads are validated where applicable.
- [] Security-related tests pass.

TESTING COMPLETION CRITERIA

```

Unit Tests
  ↓
Integration Tests
  ↓
API Tests
  ↓
UI / Functional Tests
  ↓
Regression Tests
  ↓
Final Smoke Test

```

Testing should cover successful and unsuccessful scenarios.

FUNCTIONAL TEST REQUIREMENTS

For every major feature, QA should test Normal Case, Invalid Case, Boundary Case, Unauthorized Case, and Failure Case.

```

Normal
→ Valid cart → Successful order

Invalid
→ Missing required information

Boundary
→ Quantity exceeds available stock

Unauthorized
→ Buyer attempts administrative action

Failure
→ Database operation fails

```

REGRESSION TESTING

After fixing a defect, QA should verify the original defect is fixed, related functionality still works, and existing workflows were not broken.

```

Cart Bug Fixed
  ↓
Test Cart
  ↓
Test Checkout
  ↓
Test Order Creation
  ↓
Test Inventory

```

PERFORMANCE COMPLETION CRITERIA

The team should verify that pages and APIs load within acceptable time, database queries have no obvious performance problems, images are optimized, large product lists remain usable, and mobile devices remain responsive. Performance requirements should use measurable targets where defined.

BROWSER COMPATIBILITY

The final application should be tested using browsers targeted by the project. At minimum, test current versions of primary browsers used by target users, verifying layout, forms, authentication, catalog, cart, checkout, chat, and supported notifications.

RESPONSIVE DESIGN COMPLETION

The interface should be evaluated across Desktop, Tablet, and Mobile.

Important screens include Home, Product catalog, Product details, Cart, Checkout, Orders, BOM, Chat, and Administrator dashboard. The interface should remain usable rather than simply shrinking desktop layouts.

ACCESSIBILITY REVIEW

Verify readable text, labeled form fields, understandable button names, keyboard navigation where required, appropriate alternative text, understandable errors, and visible interactive states. The project should document its accessibility scope.

DATA INTEGRITY VERIFICATION

```
graph TD
    User --> Order
    Order --> Order_Items[Order Items]
    Order_Items --> Product
```

Verify that orders belong to the correct user, order items reference correct products, inventory changes correctly, BOM items reference correct products, and messages belong to correct conversations.

DEPLOYMENT COMPLETION CRITERIA

Production deployment is complete when the build succeeds, environment variables are configured, the database connection works, migrations are applied, external services work, authentication and core workflows work, and production smoke testing passes.

PRODUCTION SMOKE TEST

```
graph TD
    Open_Website[Open Website] --> Login
    Login --> Browse_Products[Browse Products]
    Browse_Products --> View_Product[View Product]
    View_Product --> Add_to_Cart[Add to Cart]
    Add_to_Cart --> Checkout
    Checkout --> View_Order[View Order]
    View_Order --> Admin_Processes_Order[Administrator Processes Order]
    Admin_Processes_Order --> Buyer_Recives_Notification[Buyer Receives Notification]
    Buyer_Recives_Notification --> Chat
    Chat --> Logout
```

The team should perform additional administrator-specific tests.

DOCUMENTATION COMPLETION CRITERIA

- ☐ README
- ☐ Project handbook
- ☐ Architecture documentation
- ☐ Setup guide
- ☐ Frontend documentation

- ☐ Backend documentation
- ☐ Database documentation
- ☐ API documentation
- ☐ Testing documentation
- ☐ Deployment documentation
- ☐ Troubleshooting documentation
- ☐ User manual
- ☐ Changelog

Only documentation relevant to the final implemented system needs to be included.

REPOSITORY COMPLETION CRITERIA

- ☐ Correct production branch exists.
- ☐ Source code is committed.
- ☐ Required documentation is committed.
- ☐ .env files are excluded.
- ☐ Secrets are not committed.
- ☐ Unnecessary files are removed.
- ☐ Repository structure is understandable.
- ☐ README works as an entry point.
- ☐ Final version is tagged where appropriate.

GIT HISTORY REVIEW

The team should review repository history before final submission. It should show meaningful development activity rather than one final unexplained commit.

```
feat: add product search
feat: implement BOM creation
fix: correct cart quantity validation
fix: resolve order status update
test: add checkout test cases
docs: update deployment guide
```

FINAL CODE REVIEW

Review unused code, duplicate code, debug statements, temporary mock data, hard-coded credentials, unnecessary dependencies, poor naming, broken links, unhandled errors, and incomplete TODO items. Temporary development code should not remain unless intentionally required.

FINAL QA REVIEW

```
Requirement
  ↓
Implementation
  ↓
Expected Behavior
  ↓
Actual Behavior
  ↓
Pass / Fail
```

Each major requirement should have a clear testing result.

CRITICAL DEFECT POLICY

The project should not be ready for final submission when unresolved defects prevent core functionality, including login failure, checkout failure, incorrect orders, corrupted inventory, unauthorized administrative access, an unavailable production application, or critical database failures.

Minor visual defects should be evaluated separately based on usability impact.

FINAL ACCEPTANCE CRITERIA

OhmSim reaches final acceptance when:

- 1. Required functionality has been implemented.
- 2. Major user workflows work end to end.
- 3. Backend and database integration works.
- 4. Authentication and authorization work.
- 5. QA has completed final testing.
- 6. Critical defects are resolved.
- 7. Production deployment works.
- 8. Documentation is complete.
- 9. Repository is organized.
- 10. The team can demonstrate the system confidently.

END-TO-END ACCEPTANCE TEST

BUYER

```
graph TD; Register --> Login; Login --> SearchProduct[Search Product]; SearchProduct --> ViewProduct[View Product]; ViewProduct --> AddComponents[Add Components]; AddComponents --> ReviewBOM[Review BOM]; ReviewBOM --> AddToCart[Add to Cart]; AddToCart --> Checkout;
```

SYSTEM

```
graph TD; ValidateStock[Validate Stock] --> CreateOrder[Create Order]; CreateOrder --> UpdateInventory[Update Inventory]; UpdateInventory --> TriggerNotification[Trigger Notification];
```

ADMINISTRATOR

```
graph TD; ReceiveOrder[Receive Order] --> ReviewOrder[Review Order]; ReviewOrder --> Confirm[Confirm]; Confirm --> Prepare[Prepare]; Prepare --> MarkReady[Mark Ready]; MarkReady --> Complete[Complete];
```

BUYER

```
Receive Notification
↓
View Order
↓
Chat if Necessary
↓
Confirm Completion
```

If the complete workflow succeeds, the team has verified integration across several major system modules.

ACADEMIC SUBMISSION PACKAGE

```
OhmSim Final Submission
|
├─ Source Code
├─ Project Documentation
├─ Database Documentation
├─ API Documentation
├─ Test Documentation
├─ User Manual
├─ Deployment Documentation
├─ Presentation Materials
└─ Demonstration Build / URL
```

The exact submission requirements should follow the instructor or institution's official requirements.

FINAL DEMONSTRATION PREPARATION

Before the presentation, verify production access, test accounts, sample products, inventory, order workflow, notifications, chat, administrator access, and backup demonstration data. Avoid relying on data that has not been tested before the demonstration.

DEMONSTRATION SCENARIO

```
Buyer wants components for a project
↓
Searches for products
↓
Reviews product information
↓
Creates / uses BOM
↓
Adds components to cart
↓
Checks out
↓
Administrator receives order
↓
Administrator prepares order
↓
Order status changes
↓
Buyer receives notification
↓
Buyer communicates through chat
↓
Order completed
```

FINAL PROJECT CHECKLIST

PROJECT

- [] Requirements reviewed.

- ☐ Scope confirmed.
- ☐ Core features completed.

FRONTEND

- ☐ Pages complete.
- ☐ Navigation works.
- ☐ Responsive design verified.
- ☐ Loading states implemented.
- ☐ Error states implemented.
- ☐ Empty states implemented.

BACKEND

- ☐ APIs complete.
- ☐ Authentication implemented.
- ☐ Authorization implemented.
- ☐ Validation implemented.
- ☐ Error handling implemented.

DATABASE

- ☐ Schema finalized.
- ☐ Relationships verified.
- ☐ Migrations preserved.
- ☐ Production database verified.

FEATURES

- ☐ Product catalog.
- ☐ BOM.
- ☐ Cart.
- ☐ Checkout.
- ☐ Orders.
- ☐ Inventory.
- ☐ Chat.
- ☐ Notifications.
- ☐ Administration.

SECURITY

- ☐ Credentials protected.
- ☐ Secrets excluded from repository.
- ☐ Unauthorized access tested.
- ☐ Ownership checks tested.

TESTING

- ☐ Unit tests where applicable.
- ☐ API tests.
- ☐ Integration tests.
- ☐ Functional tests.
- ☐ Regression tests.
- ☐ Production smoke test.

DEPLOYMENT

- ☐ Production build works.
- ☐ Environment variables configured.
- ☐ Database connected.
- ☐ External services connected.
- ☐ Production application verified.

DOCUMENTATION

- ☐ README.
- ☐ Technical documentation.
- ☐ API documentation.
- ☐ Database documentation.
- ☐ Testing documentation.
- ☐ Deployment documentation.
- ☐ User manual.

SUBMISSION

- ☐ Repository finalized.
- ☐ Final release identified.
- ☐ Documentation packaged.
- ☐ Demonstration prepared.
- ☐ Presentation materials prepared.

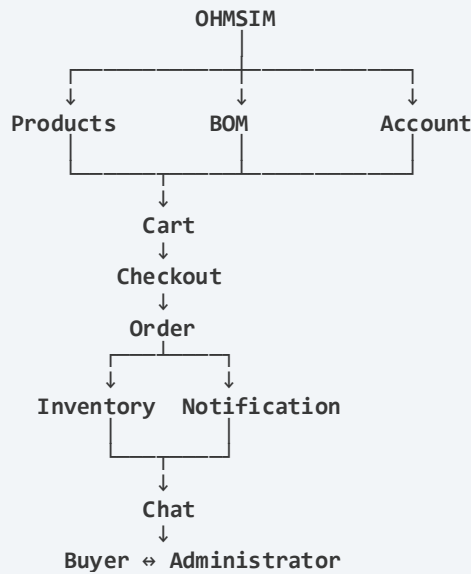
FINAL PROJECT STATUS

Status	Meaning
Not Started	Work has not begun
In Progress	Active development
For Review	Awaiting review
For QA	Ready for testing
Passed QA	Testing completed successfully
Ready for Release	Approved for deployment
Released	Deployed
Completed	Final project requirement satisfied

This prevents the team from treating unfinished development work as completed functionality.

FINAL DEFINITION OF OHMSIM

OhmSim is considered complete when its approved requirements operate as one integrated system rather than as separate individual features.



The value of the system comes from the integration of these functions into a single workflow.

FINAL PROJECT APPROVAL

The Project Manager should coordinate the final approval process.



The final release should be identified clearly so the team knows which version represents the submitted system.

CHAPTER SUMMARY

Chapter 20 establishes the final standard for declaring OhmSim complete.

The project must satisfy five fundamental conditions: The required functionality works; the integrated system works from end to end; the system passes QA and security checks; the production deployment works; and the source code and documentation are ready for handover and submission.

